Agentic SFT Shootout on Snowball: Teacher Traces, Methods and Parents

Marin post-training team (SFT shootout)

First draft, 2026-10-23 (events to 2026-10-23 18:30 UTC)

Contents

1 Introduction **4**
- 1.1 The question . . . 4
- 1.2 The five methods . . . 5
- 1.3 The two parents . . . 5
- 1.4 Terms used throughout . . . 6
- 1.5 Pre-registered rules . . . 7
- 1.6 Rounds of the experiment . . . 8

2 Data **8**
- 2.1 Task pool P . . . 9
  - 2.1.1 Exclusions and leaks . . . 9
  - 2.1.2 Decontamination . . . 10
- 2.2 Teacher trace generation . . . 10
  - 2.2.1 Reasoning effort . . . 10
  - 2.2.2 Selection rule: clean completion . . . 11
  - 2.2.3 Relay generation . . . 11
  - 2.2.4 Generation volume . . . 11
- 2.3 Training rows . . . 12
- 2.4 Contributor rows . . . 13
- 2.5 Validation sets . . . 14
- 2.6 Episode shape of the rows . . . 14
- 2.7 Collection formats, task selection and decontamination coverage . . . 14
  - 2.7.1 Collection format versus evaluation format . . . 14
  - 2.7.2 Our task pool and selection rule . . . 15
  - 2.7.3 Decontamination coverage . . . 16
  - 2.7.4 Trajectory-content overlap check . . . 17
- 2.8 Overlap between training rows and the screen task lists . . . 18

3 Methods and training recipe **19**
- 3.1 Common row representation and objective . . . 19
- 3.2 Plain and resampled demonstrations . . . 19
- 3.3 Relay . . . 20
- 3.4 Recovery . . . 20
- 3.5 Plateau dropping . . . 20
- 3.6 Optimizer, sampler and checkpoints . . . 21
- 3.7 Export audit and release . . . 22

4 Evaluation **22**
4.1 The four screens . . . 22
4.2 Composite score, bounds and unscored trials . . . 23
4.3 Noise and paired comparisons . . . 23
4.4 Serving, placement and records . . . 24
4.5 Evaluation pins . . . 24
4.6 Checks on the TBLite limits . . . 25

5 Results: the initial experiment **25**
5.1 Parents . . . 25
5.2 First round (v5): maximum-effort GLM traces on v0 . . . 26
5.3 Second round (v7): HIGH and LOW rows, two parents, five methods . . . 27
5.4 Paired comparisons . . . 27
5.5 Trajectory shape: what the students copy . . . 29
5.6 The relay result on LOW rows . . . 30
5.7 Review points: interpretation checks for the initial GLM experiment . . . 30

6 Follow-up experiments **31**
6.1 Pilot learning-rate sweep (contributor rows) . . . 31
6.2 Learning-rate check on GLM rows (3e-7 vs 1e-6) . . . 32
6.3 Update-dose check (epoch 2 vs final) . . . 32
6.4 LOW grid: methods and parents on the LOW rows . . . 33
6.5 Contributor-data experiment . . . 33
6.6 BFCL-Parity by category . . . 34
6.7 Style comparison: pilot rows and pilot student . . . 35
6.8 Qwen replication attempt (no completed sweep) . . . 35

7 Costs **36**
7.1 GPU cost by phase . . . 36
7.2 Storage . . . 39

8 Infrastructure events that affected the numbers **39**
8.1 Scoring and harness defects . . . 39
8.2 Timeouts . . . 40
8.3 Sandbox start failures . . . 40
8.4 Evictions and bands . . . 40
8.5 Hub outage (marin #10433) . . . 41
8.6 Storage key requirement (marin #10475) . . . 41
8.7 Storage key rotation (2026-10-23) . . . 41
8.8 ShellSim catalog 403 (marin #10351) . . . 42
8.9 Training data cache failure (2026-10-23) . . . 42
8.10 Manifest check failure of the matched pair (2026-10-23) . . . 42
8.11 Checkpoint save timeout of the contributor-shared run (2026-10-23) . . . 42
8.12 Other findings with no effect on the reported numbers . . . 43

9 Discussion and limitations **43**
9.1 What the comparisons identify . . . 43
9.2 Sampling and missing outcomes . . . 43
9.3 Data and method confounds . . . 43
9.4 Competing explanations for trajectory changes . . . 44

9.5 Runtime and generalization limits . . . 44

10 Conclusions **44**
10.1 Answer to the question . . . 44
10.2 What was measured about the failure . . . 45
10.3 The design doc's expectations . . . 45
10.4 What is open . . . 46

11 Follow-ups added on 2026-10-23, and proposals **46**
11.1 Matched pair . . . 46
11.1.1 What the matched pair measured . . . 47
11.1.2 Hypothesis, narrower but not tested . . . 48
11.2 Plateau on contributor data . . . 48
11.3 Controls not yet run . . . 48
11.4 Cross-format conversion proposal . . . 49
11.5 Checkpoint retention . . . 49
11.6 Feasibility of a crossed-format control . . . 49
11.6.1 What a faithful builder must preserve . . . 50
11.6.2 Effort and cost . . . 50
11.6.3 What the matched pair can answer . . . 51

A Pins and hashes **51**
A.1 Models . . . 51
A.2 Tasks and inputs . . . 51
A.3 Row manifests . . . 51
A.4 Code . . . 51

B Every screened model **53**
B.1 Hugging Face repositories . . . 53
B.2 Screened checkpoints . . . 53
B.3 Training jobs of the runs added on 2026-10-23 . . . 56

C Glossary **56**
C.1 Models and data . . . 56
C.2 Methods and runs . . . 57
C.3 Evaluation . . . 58
C.4 Infrastructure and cost . . . 58

D Per-task paired tables **59**
D.1 Night of 2026-10-22, all comparisons . . . 59
D.2 Transition tables and raw outputs . . . 59

E Training rows and evaluation transcripts, side by side **59**
E.1 SWE-smith task `swesmith-07306` . . . 61
E.1.1 (a) Contributor row (Kimi-2.5, Terminus-2 format) . . . 61
E.1.2 (b) Our GLM LOW row (mini-swe-agent format) . . . 62
E.1.3 (c) Evaluation: SWE-bench r100 under mini-swe-agent (a different task) . . . 63
E.2 Terminal (TMax) task `task_000170_2b09f3ad` . . . 63
E.2.1 (a) Contributor row (Qwen3.6-27B, tool-call format) . . . 63
E.2.2 (b) Our GLM LOW row (Terminus-2 format) . . . 64

E.2.3 (c) Evaluation: TBLite-100 under Terminus-2 (a different task) 65

Abstract

We asked which supervised fine-tuning (SFT) method improves the Snowball language model on agentic coding tasks, where the model works as an agent over many steps. Five methods were compared: plain SFT on one teacher demonstration per task, rejection sampling across epochs (RS-multi), relay (the model starts a task and the teacher finishes it), recovery (synthetic failures inserted into the demonstrations) and plateau dropping (tokens whose loss has stopped changing are dropped from the gradient). Within each round, the methods were matched on tasks, teacher and supervised-token budget. Each trained model was screened on GSM8K, BFCL-Parity, 100 SWE-bench Verified tasks and 100 TBLite terminal tasks; the *composite* is the mean of the SWE-bench and TBLite solve counts. The initial experiment used traces from the GLM 5.3 teacher at three reasoning-effort settings and two parent models (Snowball v0 and v0.1). Every model trained on GLM traces solved fewer SWE-bench and TBLite tasks than its own parent; for the arms on high-effort rows the SWE-bench gap excludes zero. For example, plain SFT on high-effort rows from v0.1 scored a composite of 10.5–11.5 against 24.5–25.0 for the parent. The trained models gained on GSM8K and BFCL-Parity, but their SWE-bench episodes ran into the 32k-token context limit instead of ending with the agent's submit command. Among the methods, only relay on low-effort rows from v0.1 beat plain SFT on the same rows by a 95% task interval that excludes zero (SWE-bench +8 tasks, [+1, +16]); it still scored below its parent. A follow-up trained the same recipe on the traces of two other teachers (Kimi-2.5 for SWE-smith, Qwen3.6-27B for TMax; the *contributor rows*). Those models scored above their parents in point estimate (plain SFT from v0: 30.5 against 26.5–27.0), though every paired interval against the parents contains zero. They clearly beat the GLM-trained plain models (SWE-bench +18 [+10, +26] for the v0 pair). The two changes tested on the GLM rows, a lower learning rate and an earlier checkpoint, did not close the gap. On both parents, switching the whole row set (teacher, harness format, tasks and token budget together) moved the scores more than any method comparison did. The *matched pair*, two plain runs from v0.1 on the same 1,432 tasks, shows that the task set does not explain the gap: on identical tasks the contributor rows beat the GLM low-effort rows by 26 SWE-bench tasks [+17, +36] and 9 TBLite tasks [+1, +17], though with 1.27 times the supervised tokens. Teacher and harness format remain bundled, and which of them matters is not established.

1 Introduction

1.1 The question

Snowball is a 67-billion-parameter mixture-of-experts language model built by the Marin project. Two earlier experiments on Snowball (marin #9920 and #9984) compared plain supervised fine-tuning (SFT), rejection-sampled SFT, on-policy distillation (called MOPD there) and weight merging on non-agentic tasks. Plain SFT and weight merging transferred skills as well as on-policy distillation or better.[1] This experiment moves to *agentic* coding tasks: tasks where the model acts as an agent over many steps (one model reply each), running commands in a sandbox until it has fixed a bug or finished a terminal job.

The question, as posted on marin #10213:[2] *which SFT method, if any, improves Snowball on SWE-bench Verified and the terminal benchmarks over plain SFT at equal supervised tokens, without losing retention on GSM8K, BFCL-Parity and GPQA-Diamond.* "Supervised tokens" (also "loss-bearing tokens") are the target tokens that carry the training loss; context tokens that the model reads but is not trained to produce do not count. "Retention" means keeping the skills the model already had, measured here on math (GSM8K) and function calling (BFCL-Parity).

The evaluation follows the Marin eval policy (marin #10197): a fixed set of agents, task lists, software versions, limits and statistical rules that the project uses to compare models. The policy's full

[1] Google Doc "[Design] Posttraining - SFT Comparison", 2026-10-15, 13duCN6reb7KVLhBi8TszTL1EYKeaV3SEMQhl9IJCZSM. Sections "Background and Motivation" and "Success".

[2] marin #10213, issue body, 2026-10-16 22:05 UTC, section "Question".

comparison runs a model on 23 benchmarks (the *tournament*). This experiment uses a cheaper first stage of four screens (Section 4).

1.2 The five methods

Plain SFT one passing teacher demonstration per task. This is the control.

RS-multi rejection sampling across epochs. The same tasks as plain, but a task that recurs in a later epoch uses a different passing attempt.

Relay the model being trained (the *student*) starts each task; the teacher takes over when a trigger fires (a format error, the context budget, or a claim that the task is done). Only the teacher's turns (its replies) carry loss (marin #10027).

Recovery plain demonstrations in which about 20% get an inserted failure: a synthetic bad turn and the harness's error feedback, written in the style of real errors (for some harnesses only the feedback). The inserted turns carry no loss, so the model sees a failure and is trained on the correct next step.

Plateau dropping plain demonstrations, but tokens whose training loss has stopped changing are removed from the gradient (marin #10028).

Section 3 gives the full definitions.

Collaborators outside this team had proposed relay, recovery and plateau dropping, and had reported gains for them in their own experiments.[1] Those experiments used other traces than ours: plateau dropping on Qwen3.6-27B traces of TMax tasks (terminal jobs), relay on Kimi-2.5 traces of SWE-smith tasks (repository bug fixes; both task families are described below), and recovery on a 40M-token general SFT mix. The size of their reported gains is not recorded in our notes. The design doc chose to regenerate all traces with one teacher, GLM 5.3, so that every method trains on the same demonstrations and matches the earlier experiments. It set two expectations. First, as a control, RS-multi should beat plain SFT. Second, each method's gain should resemble the gain its proposers had reported. It also noted that relay SFT was already merged into Snowball v0 (below), so relay's gain on v0 could read smaller. Section 10 returns to both expectations.

1.3 The two parents

A *parent* is the model a training run starts from. Two parents were used, both released Snowball versions:

Snowball v0 `open-athena/Grug-67B-A2B-Merge-Campaign3-DARE-Linear-02-2026.10.12` at revision `4eeb756b`. It is a *weight merge*: an existing Snowball checkpoint with parts of its expert weights replaced by parts of other fine-tuned models.[3] One of those models was trained by SFT on relay traces (the *allkimi* relay SFT; its training data includes 1,012 of the Kimi-2.5 SWE-smith trajectories used later in this report, Section 2.4), so v0 already carries some relay training.

[3] v0 starts from an earlier Snowball checkpoint named Antidoom Step12. Routed-expert weights were merged into it by DARE (a merge method that keeps a random subset of each donor's weight changes and rescales them): from another checkpoint, Step20, and from the non-agentic expert model of #9920 (called "B" there, not to be confused with the token budget B) in merge campaign 2; and from the allkimi relay SFT in campaign 3, which changed 2.36% of the weights per the merge record. In the provenance graph, the agentic Snowball checkpoints descend from the base model at pretraining step 156,000 through the 2026-09-21 SFT line (Step12's link to it is inferred from tokenizer and template hashes); the #9920 expert descends from the same step through the 2026-09-11 SFT line.

Snowball v0.1 `open-athena/Snowball-Smartie-RL-2026.10.15.32-step14` at revision `6783e974`. It is a different weight merge followed by 14 updates of reinforcement learning.[4] It carries no relay training.

v0.1 is a sibling of v0, not a descendant: both go back to the same earlier Snowball checkpoint (Antidoom Step12). Like every post-trained Snowball model, both descend from the shared pretraining base, the "10T cooldown" at native checkpoint step 156,000 (provenance atlas). Which SFT line Step12 sits on is inferred from tokenizer and template hashes.[3] The two have the same architecture, tokenizer, chat template and generation config (byte-identical files); their `config.json` differs only in `pad_token_id`. Both have about 2 billion active parameters per token (256 experts, 4 used per token; a 2,048-token sliding attention window, with every fourth layer attending to the whole context).

v0 was the parent when the experiment started. Marin #10283 then chose v0.1 as the policy's best candidate, and the second round of traces (v7, below) was trained on both parents. On our screens the two parents are not told apart: v0 minus v0.1 is +7 SWE-bench tasks with a 95% interval of [-2, +16], and -3 [-12, +6] on TBLite, a terminal benchmark (Section 5).

1.4 Terms used throughout

Task family, harness tasks come in two families. SWE-smith tasks are repository bug fixes run under the mini-swe-agent harness; TMax tasks are terminal jobs run under the Terminus-2 harness. A *harness* is the agent program that shows the task to the model, runs its commands in a sandbox and returns the output. Harbor runs harnesses and verifiers; Daytona provides the sandboxes.

Trial, episode, step a *trial* is one run of a harness on one task, ending with a verifier reward (1 = pass). Its recorded conversation is a *trace* and its sequence of model replies an *episode*; one reply (usually one tool call) is a *step* or *turn*. In generation each task gets 8 trials; in a screen, one.

Submit mini-swe-agent's command for "I am done". Episodes that never submit (for example because they hit the 32,768-token context limit) are still scored on the code they left behind.

Row, update a *row* is one training example built from a trace. An *update* is one optimizer step on a batch of 16 rows.

Twin two runs that differ in one factor only, for example the same rows on two parents, or plain and a method on the same rows.

Paired interval the 95% interval of a per-task difference between two models on the same tasks, from resampling the tasks (Section 4).

Screens the four evaluations run on every trained model. *SWE r100* is a fixed random 100 of the 500 SWE-bench Verified tasks, run under mini-swe-agent; *TBLite-100* is the 100-task OpenThoughts TBLite terminal benchmark, run under Terminus-2; *BFCL-Parity* is 123 function-calling tasks; *GSM8K* is grade-school math, scored with the Evalchemy evaluation harness. Terminal-Bench 2.1 (*TB2.1*) is a related terminal benchmark used only for decontamination here. Section 4 gives the details.

HF Hugging Face, and its standard model file format; an *HF export* is a trained model saved in that format for serving and release.

[4] v0.1 is 14 updates of RLOO-N (a policy-gradient method with a leave-one-out baseline; learning rate 4e-6, 512 groups of 16 samples) on the Campaign2 DARE-TIES-02 merge, whose base is also Antidoom Step12, trained on 12k NVIDIA Nemotron tasks and 57k "structural-closure" tasks (model card). Which donors that merge used is not recorded in our notes (unknown).

Operations a *serve* is the 8-GPU inference server that hosts a model during its screens; a *screen group* is the scheduler job that runs a model's screens, and a *record* is the stored result of one screen. Jobs ran on two CoreWeave H100 clusters, `cw-rno2a` and `cw-us-east-02a`, through the Iris scheduler and its central service (the *hub*). A *band* is a job's scheduling priority (batch, interactive, production, from lowest), and an *eviction* is the scheduler stopping a job to place a higher-priority one. A training run's GPUs form a *gang* (4 tasks of 8 GPUs). GPU cost is in H100-hours (H100-h). The *owner* is the person directing the experiment.

Appendix C lists every term.

1.5 Pre-registered rules

These rules were fixed before the results they apply to.

1. **Token matching.** B is the supervised-token count of the plain arm per epoch: the first passing attempt per task, one row per task. Every arm trains on B agentic supervised tokens plus a *replay slice* of general (non-agentic) SFT data of one quarter of B, which guards retention. Plateau dropping is matched on updates instead, and was to report the tokens that kept gradient; that total was not aggregated for any run (Section 3).

2. **Composite.** Our screening mean, the *composite*, is (SWE solved + TBLite solved)/2 on the two 100-task sets. When trials are *unscored* (no verifier outcome because of an infrastructure error), the composite is a range: unscored counted as failed (lower end) and as passed (upper end). The range is a bound, not a confidence interval.

3. **Learning-rate rule.** Four rates (3e-7, 1e-6, 3e-6, 1e-5) from v0 for 3 epochs on *pilot data*: existing traces from two other teachers (Section 2.4). Rates with a decisive retention loss against v0 are excluded. Among the rest, the best composite wins and the smaller rate is preferred within noise; if several rates tie and the smallest is among them, the choice goes to the owner. The chosen rate is then confirmed once on the real plain rows and frozen.

4. **Arm promotion.** An arm passes when it has no decisive loss against plain on the retention screens and a decisive win over plain on at least one agentic benchmark, with no decisive loss on the others. "Decisive" is the policy's rule: a gap above 1.96 times the combined task-level standard error, each standard error scaled by $1/\sqrt{0.9}$. Passing arms, plain and the parent go to the tournament.

5. **Relay fallback.** If the relay pool for a task family ends below its token target, plain is cut to the relay pool's size for that comparison. As executed, this built a separate run of *plain SFT matched to the relay budget* (run name `plain-matched-relay`): plain SFT whose SWE-smith rows are cut so that its supervised tokens equal the relay arm's. The other arms kept the full budget.

What was not done as planned:

- The learning rate was chosen by the owner under the tie clause of rule 3: the 3e-7, 1e-6 and 3e-6 finals tied, and the owner confirmed 1e-6 for 3 epochs. It was not confirmed on the real plain rows before the arms trained. A check at 3e-7 on the GLM rows ran later (Section 6).

- The policy's decisive test of rule 4 was not computed for the arms. The paired task bootstrap of Section 4 was used instead. On it, only relay on LOW rows (rows from low-effort GLM traces; see Rounds below) from v0.1 beat its plain twin by an interval that excludes zero (SWE-bench), and that arm stayed below its parent. No arm was promoted, and the tournament was not run.

- GPQA-Diamond and Terminal-Bench 2.1 were not run as screens. The four screens were those of the pilot sweep: GSM8K, BFCL-Parity, a fixed 100 SWE-bench Verified tasks and the 100-task TBLite set. The record does not give a reason for leaving out the other two (unknown).

1.6 Rounds of the experiment

The experiment ran in rounds, listed in Table 1. "v5" and "v7" are the version labels of the GLM trace pools; v6 was a proposal that was never built. "HIGH" and "LOW" name the GLM reasoning-effort setting used to generate a v7 pool ("high" and "low"); v5 used the default, "max". "Contributor rows" are rows rebuilt from the pilot data's traces. The later rounds have names of their own:

LOW grid the remaining methods trained on the LOW rows, for both parents.

LR check plain SFT on LOW rows at 3e-7 against the same run at 1e-6.

Dose check a screen of an earlier checkpoint (after 2 of 3 epochs) of a finished run.

Matched pair two runs on the same 1,432 tasks, one on LOW rows and one on contributor rows.

Plateau on contributor rows plateau dropping trained on the full contributor rows.

Table 1: Rounds of the experiment, in order. Dates are 2026, UTC. Read down the Traces column: each round changed the training traces. The recipe (learning rate 1e-6, 3 epochs) stayed the same, except in the pilot sweep and the LR check.

Round	Dates	Traces	Parent(s)	Runs
Pilot LR sweep	10-16/10-18	pilot (Kimi-2.5 SWE, Qwen3.6-27B TMax)	v0	4 rates
v5	10-16/10-19	GLM 5.3, max effort	v0	plain, RS-multi, recovery
v7 HIGH	10-19/10-22	GLM 5.3, high effort	v0.1 and v0	plain on both; 4 methods on v0
v7 LOW	10-20/10-23	GLM 5.3, low effort	v0.1 and v0	plain, relay; then the LOW grid
LR and dose checks	10-22/10-23	GLM 5.3, low and high	v0, v0.1	plain at 3e-7; epoch-2 checkpoint
Contributor experiment	10-22/10-23	contributor rows	v0.1 (and v0)	plain, RS-multi, recovery; plain v0
Qwen replication	10-19/10-21	pilot, in Qwen's template	Qwen3-30B-A3B	not trained
Matched pair	10-23/10-24	GLM LOW and contributor, shared tasks	v0.1	2 plain runs
Plateau on contributor	10-23/10-24	contributor rows	v0.1	plateau

The HIGH methods ran on v0 only. After both HIGH plain runs scored below their parents, the recommendation to the owner was to train the remaining arms on v0, the parent with the higher agentic scores; with no reply by the announced time (2026-10-21 17:00 UTC), that default applied. It was an operational default, not a finding that v0 is the better parent. So the HIGH relay rows for v0.1, and plain matched to them, were built but never trained.

2 Data

This section describes the task pool, how the teacher traces were generated, and the training rows built from them. Trial, trace, episode, step and row are defined in Section 1. Terms added here:

Task one problem with a verifier: a SWE-smith repository bug or a TMax terminal environment.

Clean pass a passing trial that ended with the agent's own finish: the mini-swe-agent submit command for SWE-smith, Terminus-2's `task_complete` signal for TMax.

Row cap every row is cut off at 32,768 tokens, the eval policy's context limit; longer passing traces are not used.

Padded rows the trainer fills the last batch of an epoch with filler rows so that every update has 16 rows; steps per epoch = padded rows / 16.

2.1 Task pool P

The pool has two task families, each run under the agent harness that the eval policy uses to score the matching benchmark:

TMax 2,015 terminal tasks (101 held out) from `allenai/TMax-SFT-16.5K@848e2182` (`tasks.zip`), converted to Harbor tasks with TMax's own converter (`hamishivi/tmax@6d3d606c`). Run under Terminus-2, the agent the policy uses for the terminal benchmarks.

SWE-smith 2,100 repository bug-fix tasks (105 held out) across 28 repositories, sampled from `DCAgent/swesmith-sandboxes-with_tests-oracle_verified_120s@78b0d879` (9,439 packages from 38 repositories, none a SWE-bench Verified repository). Run under mini-swe-agent 2.1.0, the agent the policy uses for SWE-bench.

Selection was outcome-blind, with one exception (the pilot sweep's held-out tasks were included first); Section 2.7 gives the rule. Held-out tasks are 5% per family; no arm trains on them.

Pool v5 manifest: `7f3d6f7dbef1ed80d302f251540c72a5c548edb871fd0afed729653d7b6def68`, public at open-athena/Snowball-SFT-Shootout-Pool-2026.10 (revision `d30b3063`). Every row builder checks this hash.

2.1.1 Exclusions and leaks

The pool went through five versions (v1 to v5, 2026-10-16 to 10-17). Each removal is one class of problem: during the agent phase the sandbox holds a readable clean copy of the code the agent must fix, or the verifier cannot give a clean pass or fail. Removed SWE-smith tasks were replaced ("topped up") from other repositories with no pass/fail information; removed TMax tasks were not replaced. Table 2 lists the removals.

Table 2: Tasks removed from pool P, by version and reason. Most removals are leak classes (a readable clean copy of the code); the v5 removals are verifiers that cannot give a clean result.

Version	Repository or task	Removed	Reason
v1	181 TMax environments	181	hidden verifier assets in the image
v2	`agronholm/typeguard`	57	clean copy vendored in setuptools
v2	`swesmith-01094` (`ariga/atlas`)	1	broken verifier (unpatched tree scores 1)
v3	`pygments`, `pandas`, `tenacity`	448	clean copy in mini-swe-agent's tool environment
v3	`mahmoud/boltons`	21	clean copy in the conda package cache
v4	`chardet`, `tomli`, `inflect`, `dataset`	73	clean copies vendored or installed as an egg
v5	7 `paramiko` tasks	7	broken verifier (null agent scores 1)
v5	2 TMax tasks	2	broken verifier (null agent scores 1)
v5	2 TMax tasks	2	verifier hangs on failure

Leak classes and fixes:

1. **Git history (marin #10212).** The upstream SWE-smith images keep the repository history ("Initial commit", "Bug Patch", "Remove F2P Tests"), so `git diff HEAD main` prints the fix. In the Kimi-2.5 pilot traces, 860 of 5,237 passing rows (16%) read the history; the pilot's own filter drops them (`reads_git_history`). The fixed packages delete `.git`, commit the task tree as a single new commit, and expire the reflog. The verifier's test restore (which upstream does with `git checkout HEAD~1`) now copies the restored test files from `tests/f2p_restore/`, which Harbor uploads only for the verifier. Checked on 20 tasks from 20 repositories: no history refs; the gold patch (the *oracle*) scored 1 on 20/20; the unpatched tree scored 0 on 19/20 (the 20th is the broken `ariga/atlas` verifier above).

2. **Clean copies in the environment (marin #10223).** Some repositories' own code is installed elsewhere in the image (vendored in pip or setuptools, the conda cache, an egg) or in mini-swe-agent's tool environment. Those repositories were removed. The row builder also drops any trace that touches the tool environment or reads a library copy of its task's repository.

3. **Hidden TMax verifier assets.** 221 of 2,200 TMax images contain verifier assets (reference programs, ground-truth files). The 181 whose instruction never names the asset were removed; the 40 that name it are kept and flagged.

4. **Integrity pass.** A null agent (changes nothing) scored 0 on 2,015 of 2,019 TMax and 2,093 of 2,100 SWE-smith v4 tasks; the SWE-smith gold patch scored 1 on all of them. The exceptions were removed in v5. TMax has no runnable reference solutions, so it has no oracle check.

Requirements for runs on the pool: no internet in mini-swe-agent's agent phase (this closes the public `github.com/swesmith` mirrors) and pinned mini-swe-agent dependencies.

2.1.2 Decontamination

Decontamination checks that no training task shares text with an evaluation task. Every pool task was checked with 13-word n-gram matching against all of SWE-bench Verified, Terminal-Bench 2.1 and TBLite: 0 tasks were flagged, and 10 TMax tasks came near the threshold (the highest overlap 0.474 against a 0.5 threshold). The contributor rows inherit this result for their tasks in pool v5; their other 1,515 SWE-smith tasks got two new passes, which found one hit, excluded anyway. Section 2.7 gives the method and the coverage of every training source (Table 9), and Section 2.8 checks the training rows against the screen task lists directly.

2.2 Teacher trace generation

The teacher is GLM 5.3, served through the internal inference service. Each task was attempted k = 8 times. Settings:

- SWE-smith under mini-swe-agent 2.1.0 with a 30,000-token generation window (the eval window is 32,768).
- TMax under Terminus-2 with a 65,536-token input limit, a 16,384-token output limit and an 1,800 s agent timeout (the TBLite screen itself runs at a 32,768-token context; Section 4).
- Harbor runs the trials in Daytona sandboxes, launched from cw-us-east-02a.
- A *pool gate* scans every trace for leaked credentials before any row is built; every production pool passed with 0 hits (Section 2.7).

2.2.1 Reasoning effort

GLM 5.3's chat template exposes three reasoning-effort settings: low, high and max. ("medium" renders as max.) The clients of the first pool sent no effort, so the template default, max, applied. That is the v5 pool. Its rows caused two defects in the first trained arms (Section 5), so a pilot measured the other two settings on 200 tasks per family, 2 samples each.

On this 200-task pilot, TMax pass rates were within 1.4 points of each other, and SWE-smith pass rates ranged from 61.1% (max) to 68.9% (high), with a standard error of about 2.4 points per cell. The setting changed the shape of the demonstrations far more than the pass rate. High was chosen for the main v7 pool (2026-10-19 23:50 UTC); a second, plain-only pool at low effort followed (decided 2026-10-20 about 03:30 UTC). These are the HIGH and LOW pools.

Table 3: GLM 5.3 reasoning-effort pilot (200 tasks per family, 2 samples). Pass rates barely move; look at the SWE-smith clean-pass and window-cutoff rows, which change several-fold.

Measure	max	high	low
TMax pass rate (%)	77.6	78.3	76.9
TMax clean pass (%)	75.9	75.5	75.4
TMax reasoning tokens per turn, p50	709	409	81
TMax reasoning tokens per turn, p99	16.4k	10.6k	6.7k
TMax turns over 8k reasoning tokens (%)	5.5	2.2	0.7
SWE-smith pass rate (%)	61.1	68.9	65.5
SWE-smith clean pass (%)	31.1	55.6	64.2
SWE-smith trials cut by the 30k window (%)	65.6	32.8	6.6
SWE-smith completion tokens per pass, p50	8.6k	4.5k	1.4k

2.2.2 Selection rule: clean completion

In the v5 (max-effort) pool, about half of GLM's passing SWE-smith trials ended at the window without a submit, and the v5 rows inherited them (Section 5.2). From v7 on, an attempt counts only if it is a clean pass, so every v7 row ends with the agent's own finish.

2.2.3 Relay generation

In relay generation the student (a parent model) runs the task first. The teacher continues after a trigger: a format error, the context budget, or a completion claim. The context-budget trigger fires at 75% of the 30k window, that is after 22,500 student tokens. SWE-smith tasks run up to six relay rounds, TMax tasks two. Relay rows keep the student's turns as context with no loss; only GLM's turns are trained. Relay pools were generated with both parents as students.

A first relay pool was generated for v5 (max effort, v0 as student; 130 SWE-smith and 36 TMax rounds). Only its TMax part was built into rows, as a component check; the v5 relay arm was never built or trained, because the v5 results (Section 5) led to the v7 regeneration.

At high effort, the SWE relay pools ended below their token targets. Many of GLM's passing relay continuations ended without a submit: only 47% (v0.1 student) and 55% (v0 student) of them submitted, because the student's opening turns had used part of the window. A test at a 0.5 handoff on one shard did not raise the submit share, so the 0.75 handoff was kept. At low effort, the student kept 81-84% of each episode, and on a one-shard pilot the clean share of relay passes was 74% (v0.1) and 71% (v0), against 52% and 45% for the same shard at high effort; both LOW SWE relay pools reached their targets.

2.2.4 Generation volume

Table 4: GLM 5.3 generation volume for the v7 pools. Look at the prompt-token column: relay costs about as many GLM tokens as plain. Sandbox-hours are Harbor trial wall time, a proxy for Daytona time, not billing. The LOW relay sets reached their budget on 6.31B GLM prompt tokens; the HIGH relay sets used 7.42B and still fell short. Relay student GPU time is in Section 7.

Set	Trials (attempts)	GLM prompt tokens	GLM completion tokens	Sandbox-hours
HIGH, four sets (plain and relay)	68,379	13.96B	545M	5,127
LOW plain (both families)	34,353 (a)	3.76B	178M	1,479
LOW relay (both students)	34,186	6.31B	170M	2,885

(a) Attempts and input tokens from the 2026-10-20 13:35 UTC snapshot, after LOW plain had

finished; 92% of the input tokens were *prefix-cached* (a repeated conversation prefix the service did not recompute). The final LOW plain shard files hold 16,775 SWE-smith and 18,483 TMax traces (35,258); the 905 difference from the snapshot is not explained in our notes. The v7 HIGH plain shard files hold 16,760 and 17,811. Generation volume for the v5 pool was not totalled (unknown).

A disk-full scan of all 79,482 v7 attempts that existed on 2026-10-20 (plain and relay, both efforts, while relay was still running) found 10 attempts with incomplete traces; they are excluded from every production build by file (`traces-v7/exclusions-enospc-20261020.jsonl`, sha256 prefix `8fa67c25`).

2.3 Training rows

B is plain's agentic supervised-token count per epoch. Every arm also trains on a replay slice of general SFT data with a target of R = round(B/4) supervised tokens; the last replay row is kept whole, so the realized replay total is slightly above R (Section 3). All arms cut it from the same sample of general SFT data (`nemotron_sft_v3` with reasoning kept, the "continuation v2" sample; manifest prefix `32e6e20b`). Table 5 gives B for each plain row set and Table 6 the method row sets that differ from plain in size. One optimizer step takes 16 rows; steps per epoch = padded rows per epoch / 16.

Table 5: Plain row sets by round. B is per epoch. Eligible = tasks with at least one usable pass (a clean pass from v7 on) that fits the 32,768-token cap, out of non-held-out tasks. Look at B: it falls from v5 to HIGH to LOW while the eligible task counts stay similar, because the episodes get shorter.

Pool	SWE eligible	TMax eligible	B SWE	B TMax	B total	Steps per epoch
v5, max effort	1,398 / 1,995	1,482 / 1,914	13,594,868	15,338,712	28,933,580	264
v7 HIGH	1,317 / 1,995	1,531 / 1,914	7,543,248	11,874,998	19,418,246	235
v7 LOW	1,468 / 1,995	1,578 / 1,914	4,213,235	7,383,004	11,596,239	225
contributor	unknown	unknown	8,851,457	4,050,783	12,902,240	226

The v7 HIGH pool has a lower B than v5 because HIGH trajectories are shorter: a clean TMax pass carries a median of 6,320 supervised tokens at high effort against 9,413 at max. (Eligibility in v5 required any pass and in v7 a clean pass; counted by clean passes, HIGH has more SWE-smith tasks than v5, 1,416 of 2,100 against 1,002.) The LOW pool's B is 40% below HIGH for the same reason. Pass rates in the LOW pool were 63.5% (SWE-smith) and 77.3% (TMax), within 2 points of HIGH.

Quality of the v7 pools against v5 (from the full-pool dry build on 2026-10-20):

- SWE passes ending in a submit: 81.0% (HIGH) and 97.8% (LOW), against 52.3% for v5.
- SWE trials cut by the window: 35.1% (HIGH) and 7.7% (LOW), against 66.0%.
- TMax reasoning tokens per turn, median: 315 (HIGH) and 62 (LOW), against 614.

Plain rows for v0.1 and v0 are byte-identical within a pool (equal per-epoch row hashes); only the manifests differ, because they name the parent. The same holds for the RS-multi, recovery and validation rows. Only relay rows differ by parent, because the parent was the relay student.

The HIGH relay SWE pools ended at 0.72 times (v0.1) and 0.65 times (v0) the plain SWE budget of 7,543,248, so the fallback built plain SFT matched to the relay budget (`plain-matched-relay`) for both parents. It is cut to approximately the relay pool's total: 16,761,141 against 16,750,605 for v0, and 17,280,056 against 17,286,007 for v0.1. Only the v0 relay arm and its matched plain run were trained. The LOW relay pools met the LOW budget in both families with no fallback.

Other method rows:

RS-multi per-epoch supervised tokens within 1.5% of B, with the 3-epoch mean within 1% (the verifier tolerance was widened from 1% per epoch on 2026-10-21, because the sampler draws one random

Table 6: Method row sets that differ from plain in size. Steps are per epoch. Relay rows need more steps than plain for the same supervised tokens, because the student's turns are context only.

Row set	Parent	B SWE	B TMax	Steps	Updates
HIGH relay	v0.1	5,404,160	11,881,847	307	not trained
HIGH relay	v0	4,873,795	11,876,810	308	924
HIGH plain-matched-relay	v0.1	(total 17,280,056)		212	not trained
HIGH plain-matched-relay	v0	(total 16,761,141)		206	618
LOW relay	v0.1	4,217,069	7,383,308	321	963
LOW relay	v0	4,216,141	7,385,080	311	933

clean pass per task per epoch and does not target B). HIGH v0.1: epochs -1.10%, -0.77%, +0.86%. LOW: -0.03%, +0.64%, +1.26%.

Recovery failures injected in about 20% of rows per family: HIGH 20.0% SWE and 19.3% TMax (dry build); LOW 294 of 1,468 SWE rows and 297 of 1,578 TMax rows. B equals plain's.

Plateau dropping trains on the plain rows; Section 3 describes the masking.

Updates are steps per epoch times 3. The v0.1 HIGH relay rows and their matched plain rows were built and verified but never trained, because the remaining HIGH arms trained on v0 only.

2.4 Contributor rows

The contributor rows come from the traces of the pilot learning-rate sweep: Kimi-2.5 on SWE-smith, recorded in Terminus-2 format, and Qwen3.6-27B on TMax, with native tool calls. The collaborators' relay and plateau-dropping experiments used these teachers (Section 1).[5] A converter reproduced the pilot's 7,393 training trajectories and 161 held-out tasks exactly, into a separate pool (`pool-t202-contrib/`, manifest prefix `ecebf85a`).

The two sources use the opposite harness formats to our rows: the Kimi-2.5 SWE-smith traces are in Terminus-2's JSON command format, and the Qwen3.6-27B TMax traces use mini-swe-agent-style bash tool calls (Table 7; excerpts in Appendix E).

Overlap with our pool and with v0:

- Of the 1,995 pilot SWE-smith training tasks, 480 are in pool v5, 104 were excluded from it, and 1,411 were never in it.
- 1,012 of the 1,995 are in the allkimi relay SFT data, with the same trial as the pilot trajectory. allkimi is the SFT run merged into v0, so v0 has seen half of these tasks. This is why the contributor methods ran on v0.1, with plain on v0 only as a bridge to the pilot sweep.

Exclusions, for parity with pool v5: 58 tasks (128 trajectories), file `exclusions-20261022b.jsonl` (sha256 prefix `06a99e03`), by class in Table 8.

The 64 SWE-smith tasks excluded from pool v5 for mini-swe-agent's tool environment were kept, because that copy exists only under mini-swe-agent and these traces are Terminus-2. The 7 near-threshold TMax tasks were kept and flagged.

Row rule: the first passing clean trajectory per task (the v7 rule). The plain rows have B = 12,902,240 supervised tokens per epoch (SWE 8,851,457, TMax 4,050,783), replay 3,235,023, 43,930,783 context tokens, and 3,616 padded rows = 226 steps per epoch (678 updates for 3 epochs). Unlike the GLM rows, the SWE family carries more of the budget than TMax. RS-multi varies the TMax

[5] Google Doc "[Design] Posttraining - SFT Comparison", section "Dataset / Env" and "Alternatives Considered".

attempts only, because the Kimi SWE set has one trajectory per task. Recovery injects Terminus-2 parser feedback into 379 of the 1,954 Kimi SWE rows (19.4%) and the bash-tool format error into 205 of the 1,024 Qwen TMax rows (20.0%). Builder `task/t202-arms@8c1d3fb11b` (includes the per-harness recovery commit `d57cecdfec`).

The pilot sweep itself (Section 6) trained on a larger, earlier cut of the same traces: 8,768 rows and 34.4M supervised tokens.

2.5 Validation sets

The trainer's in-loop validation is off: in the pilot sweep it failed when the context was split across GPUs (marin #10232), so held-out loss is computed offline from saved checkpoints.

- v7 HIGH: a validation split exists for each parent (manifests `422fa71f...` for v0.1 and `acea6293...` for v0), built from the held-out tasks.
- v7 LOW: the inventory lists no validation split for the LOW rows.
- Contributor: 105 SWE-smith and 55 TMax held-out tasks (the pilot had 56 TMax; one held-out trajectory lacked a clean completion). Manifests `da051da0...` (v0.1) and `af6115af...` (v0).
- Matched pair (2026-10-23): no held-out loss for either run.

Held-out loss was not computed for the HIGH, LOW or contributor arm runs: their launcher passed no validation input to the trainer. The pilot sweep's held-out losses were computed offline and are reported in Section 6.

2.6 Episode shape of the rows

The *shape* of an episode is its number of steps, its tokens per step, and its context length at the last step. On SWE-smith, the median row has 25 steps of 120 tokens for HIGH, 14 steps of 90 tokens for LOW, and 14 steps of 246 tokens for the Kimi-2.5 pilot rows. Every row ends with the agent's own finish. Section 5.5 compares these shapes with the parents' own episodes and with the trained students (Table 23).

TMax rows at high effort are short episodes with long steps: median 5 steps (p90 9), 513 tokens per step (p90 3,161), and 11,087 tokens of context at the final step. In 1,530 of 1,531 TMax rows an earlier turn also sets `task_complete`, consistent with Terminus-2 asking the agent to confirm.

2.7 Collection formats, task selection and decontamination coverage

A *harness* is the program that presents the task, interprets model actions, executes tools and records feedback. Harbor orchestrates sandboxes and verifiers; its agent implementations include mini-swe-agent and Terminus-2. These agents use different action protocols. Shared terms such as *episode* and *turn* are defined in 1. An optimizer *update* is a training step and is unrelated to an episode's action count.

2.7.1 Collection format versus evaluation format

Every source is ultimately stored as `input_ids` and `loss_weights` in the v0 chat template, with tokenizer revision `4eeb756b404db25bc04e4daf49dfb9817bceb741`. The renderer preserves message role, content, reasoning, tool-call IDs and tool-call structure, then serializes them through that template. Reasoning appears inline in the token sequence. This common outer encoding does not make Terminus-2 JSON actions equivalent to native bash tool calls. In Terminus-2 rows, earlier reasoning is retained

Table 7: Collection protocols and their closest evaluation protocol. A match here means the action protocol, not identical prompts, histories, tasks or model behavior. Compare the final column: both contributor families switch protocol at evaluation.

Data / family	Collection agent	Stored action content before tokenization	Corresponding evaluation	Protocol agreement
GLM HIGH / SWE	mini-swe-agent 2.1	Bash tool calls, tool replies, explicit submit command	SWE r100: mini-swe-agent 2.1	Yes, bash/submit protocol
GLM LOW / SWE	mini-swe-agent 2.1	Same protocol; lower teacher reasoning effort	SWE r100: mini-swe-agent 2.1	Yes, bash/submit protocol
GLM HIGH / terminal	Terminus-2	JSON analysis/plan/commands; keystrokes; task completion field	TBLite-100: Terminus-2 via Harbor	Yes, JSON command protocol
GLM LOW / terminal	Terminus-2	Same protocol; lower teacher reasoning effort	TBLite-100: Terminus-2 via Harbor	Yes, JSON command protocol
Relay HIGH/LOW / SWE	mini-swe-agent 2.1	Student prefix plus teacher bash-tool continuation; prefix masked	SWE r100: mini-swe-agent 2.1	Yes, action protocol; conditioning differs
Relay HIGH/LOW / terminal	Terminus-2	Student prefix plus teacher JSON-command continuation; prefix masked	TBLite-100: Terminus-2 via Harbor	Yes, action protocol; conditioning differs
Contributor Kimi / SWE	Terminus-2	JSON commands and `task_complete`; parser feedback retained	SWE r100: mini-swe-agent 2.1	No: JSON terminal protocol versus bash tool
Contributor Qwen / terminal	mini-swe-agent bash-tool harness	Native `bash` tool calls; tool replies; explicit submit command	TBLite-100: Terminus-2 via Harbor	No: bash tool versus JSON terminal protocol
All training sources / BFCL transfer	No BFCL collection arm	Their respective protocols above	BFCL-Parity: `pi` agent via Harbor	No direct matched collection protocol

even though the collection harness does not resend it in later requests; protocol agreement therefore does not imply history agreement.

SWE r100 is the frozen 100-task SWE-bench Verified screen. Its completion action is `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`. TBLite-100 uses Terminus-2 and its `task_complete` signal. BFCL-Parity uses the `pi` agent, version 0.87.0, on tool-use tasks; its exact provider-side tool-call serialization was not independently inspected for this table and remains unknown here. It is not a fifth demonstration source. GSM8K is a direct question-answering evaluation and has no collection-harness match.

2.7.2 Our task pool and selection rule

Our GLM collection uses the frozen pool v5, containing 2,100 SWE-smith tasks from 28 repositories and 2,015 TMax tasks. It holds out 105 and 101 respectively. SWE-smith packages originate from `DCAgent/swesmith-sandboxes-with_tests-oracle_verified_120s@78b0d879`, which supplied 9,439 packages across 38 repositories. The initial allocation was proportional to repo package counts, using largest remainders and random selection within each repo with seed 20261016. Repository identity against SWE-bench Verified excluded no source repo. The pilot's 105 held-out SWE tasks were inserted first; because these were Kimi passes, that inherited subset was not outcome-blind. Later removals and top-ups preserved kept tasks and did not use model success to choose replacements.[6]

The terminal source is the 2,200 environments in `allenai/TMax-SFT-16.5K@848e218270672080a92a94791c028ee4` converted with `hamishivi/tmax@6d3d606c`. The pool excludes 181 environments with hidden verifier assets, then two with broken verifiers and two whose verifier hangs on failure, leaving 2,015. Forty environments with instruction-visible verifier assets were kept and marked. The separate TMax 15K RL set was not included. No runnable TMax oracle solutions were available; null-agent checks test only whether an unchanged environment can pass.

For our GLM collection, SWE package repair removes the accessible Git history and remote, then reconstructs a single task-tree commit. Fail-to-pass tests are restored only during verification. Additional checks remove readable clean copies in tool environments, vendored packages and base images. Generation restricts agent-phase network access, and trace filters reject reads of forbidden clean copies. These protect against answer access within a task; they are distinct from overlap with an evaluation benchmark. These collection-time protections were not retroactively applied to the original Kimi/Qwen episodes; those inputs use recorded trace filters and task exclusions.[6]

These 58 task exclusions remove 128 contributor trajectories. A separate clean-completion filter removes 64 TMax trajectories without the required submit, leaving 13 additional TMax tasks without an eligible pass. Contributor plain ultimately uses 1,954 SWE tasks and 1,024 TMax tasks. The contributor pool and the final selected training rows must therefore not be treated as the same task set.

[6] Task-pool builder; `mopd2/pool/README.md` and `manifest.json`, pool-v5 SHA-256 `7f3d6f7dbef1ed80d302f251540c72a5c548edb871fd0afed729653d7b6def68`. History repair is tracked in the history-leak issue.

Table 8: Contributor task exclusions. These are a different population from the pool-v5 removals and from the later trace-level disk-failure exclusions. Most exclusions concern answer access or verifier validity, not benchmark overlap.

Family	Exclusion class	Tasks
SWE-smith	Image-level clean copy	30
SWE-smith	Vendored clean copy	7
SWE-smith	Broken verifier	3
SWE-smith	N-gram overlap rule	1
TMax	Hidden verifier assets	15
TMax	Broken verifier	2
Total		58

2.7.3 Decontamination coverage

Decontamination here searches for shared text between training-task material and benchmark material. It is separate from checking whether a training sandbox exposes its own answer. It also does not establish whether a pretrained parent has seen a task before.

The pool driver uses DataKit `decon_to_parquet` with 13-word n-grams, formed from whitespace-split words. A paragraph's overlap is its matching-feature fraction; the reported maximum is over paragraphs. The threshold is 0.5 with at least two distinct matched features, subject to the implementation's exact-short-document exception. Matched features are n-gram hashes detected by the filter; confirmed features are those also found in its evaluation hash index. Near-threshold reporting covers 0.2–0.5 and does not itself trigger exclusion. Evaluation documents are SWE-bench Verified problem, gold-patch and test-patch text, and terminal benchmark instructions, tests and solutions. Training documents are instructions plus verifier tests: restored fail-to-pass tests for SWE-smith, and `tests/test_final_state.py` for TMax. Terminal evaluation `environment/` files are excluded because they vendor upstream projects and caused a license-text hit. The checked sets are all 500 SWE-bench Verified tasks, 89 Terminal-Bench 2.1 tasks and 100 TBLite tasks. Thus the SWE coverage is broader than the r100 screening subset.

Table 9: Checked task text and inherited coverage. A zero hit is limited to these documents, benchmark pins and thresholds; it is not a trajectory-text audit. In the third column, Qwen TMax relies on inherited records, with no new pass.

Training source / family	SWE-bench Verified	TBLite-100 and TB2.1	Findings and action
Our GLM pool / SWE-smith	Repo identity; instruction plus restored tests, n-gram pass	Same full-document n-gram pass	Zero tasks flagged by the rule; no decon removals
Our GLM pool / TMax	Instruction plus verifier tests, n-gram pass	Same full-document n-gram pass	Zero tasks flagged by the rule; near-threshold tasks kept/flagged
Contributor Kimi / SWE tasks in pool v5	Inherited pool-v5 task record	Inherited pool-v5 task record	480 pilot training tasks have records
Contributor Kimi / SWE without pool-v5 record	New instruction-only and full-parity passes	Both passes include these benchmarks	1,515 checked; one shared hit excluded
Contributor Qwen / TMax	Inherited pool-v5 task record	Inherited pool-v5 task record; no new Qwen-specific pass	1,037 covered; 17 pool-excluded tasks removed
Original contributor pilot	SWE repo-identity filter only	No pilot n-gram pass	Later task checks do not retroactively change pilot inputs
Selected agentic trajectory text	Checked separately below	Checked separately below	Replay and additional RS-multi passes are outside that check
General-data replay	Coverage not established in this check	Coverage not established in this check	Separate from the agentic task-document checks

The added contributor SWE check first used instruction text alone, then repeated with instructions plus restored tests (called *full parity* with the pool-v5 document method). Both checked 1,515 training tasks and flagged `swesmith-00735` at overlap 0.5088 with 29 matched and confirmed features. Its match was reused xarray issue-template checklist text against SWE-bench Verified, rather than shared task-specific solution text. It was nevertheless excluded under the rule. `swesmith-10059` at 0.25 was retained and flagged. The full-parity run reported no package-rebuild failures.[7]

[7] Reports under `s3://marin-us-east-02a/marin/experiments/sft-shootout/pool-t202-contrib/notes/`: `decon-hits.json`, `decon-full.json` and `bookkeeping.json`. Evaluation pins: SWE-bench Verified `c104f840`, TB2.1 `c5ee500c`, TBLite `f075e463`. The shared driver is `sft_shootout_pool/decon.py` at `04adf519c5`. The contributor-specific invocation script/commit is not yet identified in this check; the stored reports identify its document scope and shared method.

Of the pilot's 1,054 TMax training tasks, 1,037 have a pool-v5 record, 17 were excluded from that pool, and none was outside its original source. All 17 are removed from the contributor rows. Thus the retained Qwen TMax task coverage against TBLite is inherited from the pool-v5 instruction-plus-tests pass. The later trajectory check is reported below. The converter attaches that record by matching family and task ID; this check has not established byte identity between the contributor prompt and the audited pool document. The inherited record shows that a task with that identity was checked. It does not show that every field of the contributor episode was checked again.

Seven pilot TMax tasks are among pool-v5's ten near-threshold tasks and remain flagged. The largest pool overlap below threshold is 0.474. No separate terminal repository- or instance-identity screen is established by the checked code.

The generation `pool_gate` does not perform evaluation decontamination. It checks shard completion markers, relay-generation stop markers, rebuilt trace files and capability-token leaks (secret credentials embedded in proxy URLs) in trace/request files. Its report of zero token hits means no detected secret tokens; it does not mean no benchmark-text overlap. `ARMS_VERIFIED` is the success message emitted by the row verifier after checking the built rows, held-out exclusions, replay identity and epoch/budget invariants. It is not an evaluation score or decontamination certificate.

2.7.4 Trajectory-content overlap check

The trajectory check flagged three rows, all against TBLite reference material. Two flags occur in trained row sets; the third is in the untrained HIGH relay v0.1 set, included to cover the generated relay data for both parents. Inspection of the matched paragraphs found two Python import blocks and one generic line-reading loop. These matches do not establish answer leakage. No row was changed or excluded on this evidence. A lack of detected overlap does not establish absence of contamination.[8]

The check decoded the selected first-epoch agentic rows with the pinned training tokenizer, including prompts, observations and masked student prefixes. It used the same selected examples as the first training epoch; masked prefixes supply context but receive no direct training loss. Every row was checked against both reference collections. The check used the existing DataKit paragraph-level 13-gram rule: overlap at least 0.5 and at least two matched features. Features are consecutive whitespace-separated words, with case and punctuation preserved; paragraphs split at blank lines. The overlap denominator is the number of features in the training paragraph, not in the reference document. The implementation's short-record and no-long-paragraph fallbacks were retained. An exact hash set replaced the Bloom filter. Row/index file hashes were compared with their source manifest, whose own hash was recorded. Replay, padding, additional RS-multi passes (resampled successful episodes), recovery-specific injections and unused generation trajectories were outside the scope. The excluded resampled episodes and synthetic recovery errors were used by their respective training methods; this check does not cover all of those methods' content. The reference text comprised gold and test patches for the frozen SWE-bench r100 tasks, and 399 readable files under tests/ and solution/ for all 100 frozen TBLite tasks. Instructions and environment files were excluded from the reference side.

The HIGH plain flag is `task_000395_7d722152` against `supply-chain-fulfillment`: an import block containing `defaultdict`, `Path` and `Any` (overlap 0.5). The HIGH relay v0.1 flag is `task_000709_7bd8a98a` against `sales-data-csv-analysis`: imports of pandas, numpy, scipy.stats and json (overlap 0.667). The Qwen flag is `task_002073_a89d5ed6` against `malicious-package-forensics`: a loop reading a file, stripping lines and skipping empty lines (overlap 0.5). Each flag has two matched features. The interpretation as generic code is based on these matched paragraphs; this check does not test semantic equivalence, model pretraining exposure or overlap below its threshold.

[8] Reproduction script and per-row hit IDs: `mopd2/arms/trace-content-decon-20261023.py` and `mopd2/arms/trace-content-decon-20261023.json`. Reference pins: SWE-bench Verified `c104f840` and TBLite `f075e463`. The scan completed at 2026-10-23 23:13 UTC; matching paragraphs were reviewed on 2026-10-24. The frozen task-ID file hash and source manifest hashes are in the JSON.

Table 10: Selected trajectory rows checked. Flags occur only against TBLite; they are text-overlap candidates, not confirmed leakage.

Row set	SWE rows	Terminal rows	SWE flags	TBLite flags
GLM HIGH plain	1317	1531	0	1
GLM LOW plain	1468	1578	0	0
HIGH relay v0.1 (untrained)	1891	2110	0	1
HIGH relay v0	1909	2104	0	0
LOW relay v0.1	2469	2113	0	0
LOW relay v0	2398	2015	0	0
Contributor Kimi SWE	1954	0	0	0
Contributor Qwen terminal	0	1024	0	1

2.8 Overlap between training rows and the screen task lists

Could the training rows contain the screened tasks, or near copies of them? We checked three kinds of overlap for every training row set against the two agentic screen lists: identical task ids, shared source repositories, and near-duplicate text. The text check is Marin's DataKit n-gram decontamination: each training task's document (its instruction plus its verifier's test files) is compared with each evaluation document by 13-word n-gram overlap measured per paragraph, and a task is flagged at an overlap of 0.5 with at least 2 matched features (the reference pipeline's thresholds). It was run against all 500 SWE-bench Verified tasks (problem statement, gold patch, test patch), all 100 TBLite 2.0 tasks and the 89 Terminal-Bench 2.1 tasks (instruction plus tests and solution). Both screen lists are subsets of the first two. The check compares task documents, not the recorded trajectories: the Qwen TMax tasks inherit their pool v5 result by task id. The recorded trajectories were checked separately, after this check (Section 2.7.4).

Table 11: Overlap of each training row set with the screen task lists. Task counts cover the training and held-out tasks after exclusions. Read across each row: no row set shares a task id or a repository with the screens, and the n-gram check flagged one boilerplate hit, which was removed before training.

Training rows	Source pool	Tasks	Task-id overlap	Repository overlap with SWE-bench r100 | 10 repos|	n-gram check |13-gram, 0.5|
GLM HIGH, LOW and relay rows: SWE-smith	pool v5, 28 repositories	2,100	0	none; same GitHub org only: `pysem-dev/iniconfig` (r100 has `pysem-dev/pysem`)	0 flagged
GLM HIGH, LOW and relay rows: TMax	pool v5 | TMax-SFT-16.5K|	2,015	0 (TBLite-100)	not applicable	0 flagged; 10 near threshold in the manifest (0.22–0.47; the README says 11), eval match not recorded
contributor rows: Kimi-2.5 SWE-smith	pilot set, 38 repositories	2,059 (1,954 train + 105 held out)	0	none; same org only: `pysem-dev/iniconfig`	1 flagged and excluded (`swesmith-00736`, survey issue-template text, 0.51); 1 near threshold (0.25)
contributor rows: Qwen3.6-27B TMax	pilot set |TMax-SFT-16.5K|	1,093 (1,037 train + 56 held out; 1,024 have training rows, 55 held-out rows)	0 (TBLite-100)	not applicable	inherits pool v5: 0 flagged; 8 of the 10 pool near-threshold tasks are in this set (7 in training)

Table 12: Tasks newly solved (gained) and no longer solved (lost) by the contributor plain students against their parents, SWE-bench by repository. Compare the gained column with the screen's own repository mix (django 46 of 100, sympy 14).

Pair	Screen	Gained	Lost	Gained tasks by repository / task
contributor plain v0.1 vs v0.1	SWE-bench	12	9	django 8, sympy 2, astropy 1, sphinx 1
contributor plain v0 vs v0	SWE-bench	13	11	django 5, sympy 5, sphinx 2, pytest 1
contributor plain v0.1 vs v0.1	TBLite	14	10	14 unrelated tasks (e.g. `amuse-install`, `broken-python`, `mlflow-register`, `okhttp-trailers-crash`)
contributor plain v0 vs v0	TBLite	12	6	12 unrelated tasks (e.g. `acl-permissions-inheritance`, `grid-pathfinding`, `sympy-bug-fix`)

Reading (Tables 11 and 12). No training row set shares a task id or a repository with the screens, and the n-gram check of the training task documents found no screened content (the one hit was issue-template boilerplate and was removed). The separate trajectory check (Section 2.7.4) flagged three TBLite rows with generic code and no SWE rows. The contributor students' gains are spread over the screen's own repository mix and over unrelated TBLite tasks, with the only possible link at organisation level (one `pytest-dev/pytest` task gained by contributor plain v0, whose rows include `pytest-dev/iniconfig`). On the checks run, the gains do not concentrate where the training tasks overlap the screens.

Limits. The n-gram records do not name the eval document behind the 10 TMax near-threshold

scores; the check is lexical and does not detect paraphrased tasks; trajectory text is covered only by the separate check in Section 2.7.4. Separately, parent v0 was built with an earlier SFT graft (relay-format training on contributor traces) that included 1,012 of the contributor SWE-smith trajectories; that is a property of the parent v0, not an overlap with the screens.

3 Methods and training recipe

The arms change which demonstration tokens contribute to supervised fine-tuning (SFT). A demonstration is a recorded task episode, including model replies and tool feedback. The model predicts the next token; the loss mask selects which predictions train it. The common recipe makes comparisons within a data set interpretable. At a fixed token budget, different episode lengths produce different row counts, which change the number of fixed-row-batch updates. Parent and data-source definitions are in 1 and 2; task filters and harness protocols are in 2.7.

3.1 Common row representation and objective

An episode becomes one tokenized row in the v0 chat template. Both parents use the v0 tokenizer revision `4eeb756b404db25bc04e4daf49dfb9817bceb741`. Reasoning is rendered inline between the template's thinking delimiters. The row carries `input_ids` and `loss_weights`: weight at position i selects the prediction of token $i + 1$. Teacher assistant turns are supervised; system, user and tool messages are context. Student prefixes in relay and injected failed replies in recovery have zero loss. Rows are not packed together, and the loader does not insert an additional end token.

Replay supplies additional general supervised examples alongside agent episodes, using a seeded slice of DataKit's pinned `nemotron_sft_v3` sources rendered in the parent template. The arm slice is the frozen *continuation v2* artifact: a larger sampled replay set that extends the pilot's general-data sample using the same source partitions and quotas. Here *continuation* names a data-sampling artifact, not resumed model training. Its purpose is to retain broader instruction-following behavior; this experiment does not include a no-replay ablation.

Let B be the plain arm's agentic supervised-token budget per epoch, summed over the software-repair and terminal families. A frozen general-data replay prefix targets $R = \text{round}(B/4)$ supervised tokens. The last replay row is retained whole, so the realized replay total can exceed the target. Replay is approximately one fifth of the combined supervised tokens. Related method arms keep the plain arm's replay cut; plain SFT matched to the relay budget also keeps that cut when its agentic budget falls.

3.2 Plain and resampled demonstrations

Plain SFT selects the first eligible passing attempt for each task, ordered by attempt sample index. Eligibility includes the recorded clean-completion rule and source-specific filters. Its selected episodes recur across epochs in seeded permutations. Passing means that the task verifier accepted the episode; clean completion additionally requires the collection harness's explicit completion signal.[9]

RS-multi resamples a passing demonstration for each of the plain arm's task slots across epochs. It preserves the task slots and replay rows, while the chosen demonstrations can change the supervised-token total. The production verifier accepts each epoch within 1.5% of plain's agentic budget and the mean within 1%. In the contributor experiment, only TMax changes: the Kimi software-repair source supplies one passing episode per task. Thus the contributor label *RS-multi-TMax* denotes the same builder with variation confined to the terminal family, rather than a different training objective.

[9] Builder and renderer: `experiments/post_training/sft_shootout_rows/` at `8c1d3fb11b`; frozen rows and manifests: `sft-shootout/data-inventory.md`.

3.3 Relay

Relay lets the student begin the task and a teacher continue from the resulting state. The student-written prefix remains in context but contributes no supervised loss. Only teacher continuation turns train the model. The builder samples usable relay episodes separately by family toward that family's plain budget. This changes the visited states and the distribution of continuation lengths as well as the loss mask. It is therefore a comparison of the whole relay recipe.[9]

The HIGH relay pool exhausted its software-repair episodes before reaching the target budget for either parent. The registered fallback builds a separate plain SFT run, `plain-matched-relay`, by cutting its software-repair family to the achieved relay budget. It does not trim the other HIGH arms. LOW relay met both family budgets and is compared with ordinary LOW plain. LOW relay v0.1 takes 963 updates against plain's 675; HIGH relay takes 924 updates on v0, against full plain's 705. The planned HIGH relay v0.1 run would take 921 updates but was not trained. HIGH v0 plain matched to relay takes 618 updates. The v0.1 plain matched to relay was planned at 636 updates but was not trained in that HIGH wave. Equal supervised-token budgets do not imply equal row counts, context exposure or updates.

3.4 Recovery

Recovery adds synthetic, harness-specific format failures to approximately 20% of eligible plain rows. It inserts one or two failure points, with one to three failed attempts at a point, away from the first and last teacher turns. The original teacher tokens remain the supervised targets. Added replies and feedback are context only; injections are reduced or omitted when they would exceed the context limit.

The recovery history follows each collection harness. Terminus-2 retains the malformed assistant reply and feedback from its JSON parser. The GLM software-repair harness rejects a malformed reply before adding it to history, so its default recovery variant adds the feedback without that reply. The contributor Qwen bash-tool harness retains both the failed reply and its literal format-error message. That message demands exactly one bash call and identifies the completion command. Error styles were drawn from the measured v0 error recipe; they should not be read as an estimate of Qwen's error-style distribution.[10]

3.5 Plateau dropping

Plateau dropping keeps the plain rows but removes stable tokens from the loss. For each supervised token it stores the losses at its previous two visits. Starting at visit three, it drops a token if both successive absolute relative changes are below 0.1, using a 0.05-nat floor for each earlier-loss denominator. A dropped token stays in the input; its loss is measured again and it returns to training if the criterion fails. The objective averages over the supervised tokens still kept. With an epoch-exact three-epoch traversal, dropping ordinarily first applies in the final epoch.

The history is keyed by a hash of the row's tokens and loss mask, independent of row order. Resume requires both the native training state and its matching `plateau-updates-N.npz` sidecar. Preserving only model weights cannot reproduce the plateau state. Archived sidecars copied out during storage cleanup are analysis records; their relocation alone does not make a training root resumable.[11]

The implementation records `plateau_kept_tokens` and `plateau_dropped_tokens`. The fraction of supervised tokens retaining gradient should be computed as summed kept tokens divided by summed kept plus dropped tokens across the run, including replay. That aggregate is unknown in this report for

[10] `recovery.py` at `8c1d3fb11b`; source error observations in `mopd2/arms/contributor-launch-plan-20261022.md` and the builder's provenance constants.

[11] `mopd2/arms/checkpoint-retention-20261023.md` and `mopd2/arms/storage-delete-20261021.md`, plateau sidecar preservation.

HIGH plateau v0 and LOW plateau v0/v0.1; the final update's fraction would not substitute for the run-wide quantity.

3.6 Optimizer, sampler and checkpoints

Table 13: Common arm recipe. The lower-learning-rate follow-up changes only its rate. The fixed row batch leaves update counts free to differ with each arm's row count.

Setting	Value
Learning rate	10^{-6}, constant, no warmup
Epochs	3
Optimizer	AdamW; weight decay 0
Adam moments	0.9 / 0.999
Adam epsilon	10^{-8}
Gradient-norm clip	1.0
Global batch	16 rows
Context per row	32,768 tokens
Padding	Zero supervised loss
Objective	Mean over active supervised tokens
Routing bias	Frozen
Selection	Final checkpoint after the prescribed epochs
In-loop held-out validation	Disabled

The arm loader concatenates separately materialized epoch files through a scheduled mixture with blocks of 16 rows. It reads each file in its stored order without an additional shuffle, and checks every cached row against its source tokens and mask. This differs from the earlier pilot LR sweep's shuffled mixture with nominal block-2048 epochs. The pilot and the arm grid therefore do not have identical samplers.

The arm launcher supplied no validation paths or validation data to its export adapter, so per-arm held-out loss was not computed. HIGH and contributor validation artifacts exist, but LOW has no prebuilt validation set; see 2 for the splits. The matched pair likewise disables validation on both sides rather than evaluating LOW training against HIGH-effort validation episodes. Final checkpoint selection does not use held-out loss. The pilot measured held-out loss separately, offline.[12]

Training uses Levanter in Marin on gangs of 32 H100 GPUs. The arm launcher pins include `d23b8c9a94` for the contributor grid and the named earlier pins in the launch ledger. The row builder for contributor data is `8c1d3fb11b`, including the harness-aware recovery change `d57cecdfec`. Production scheduling was used from about 21:30 UTC on 2026-10-20; the five v0 arms were resubmitted at production on 2026-10-21 after interactive attempts. Interactive scheduling became the default again for the LOW grid and contributor runs on 2026-10-22, with own-root resume. Recovery exceptions and their costs remain in the attempt ledger. Scheduling bands are operational settings, not method arms.

The two HIGH plain runs exported HF weights after epoch two and at the final checkpoint. Subsequent arm launches used `--hf-export-schedule final-only` to avoid intermediate conversion/upload time. Native training checkpoints were still requested at epoch boundaries. They contain optimizer and data-position state for resume, unlike HF weights. The proposed rolling-retention change in 11 was not used for the reported runs.

[12] `mopd2/arms/shared-task-budget-20261023.json` and `mopd2/arms/submit-shared-controls-20261023.py`; LOW validation absence confirmed in the launcher and row inventory.

3.7 Export audit and release

The final BF16 HF exports have 39 shards and 502 tensors, with 134,157,765,632 tensor bytes. Audits check shard presence, tensor inventory and shapes, safetensor bounds, tokenizer and chat-template identity, and the 26 routing-bias tensors against the corresponding parent after the export cast. These are structural and selected-tensor checks; they are not checksums of every learned weight or inference-equivalence tests. Screens are handed the exact audited path and independently recheck required files. Appendix B records published model repositories and revisions.

4 Evaluation

Every checkpoint in this report was scored on the same four screens. The screens are short, fixed evaluations meant to rank training arms quickly, not to estimate absolute capability. Two screens test general skills that fine-tuning should not damage (GSM8K and BFCL-Parity). Two test the agentic coding skills the training data targets (SWE-bench Verified and TBLite). The agentic pair decides the comparison.

Episode, step and trial are used as defined in 1 (C); every task gets one trial. **Unscored** trials are defined below. Table 14 lists the screens.

4.1 The four screens

Table 14: The four screens. Read the first column for what each tests and the limits column for what can end an episode early. One trial per task.

Screen	Tasks	Agent / harness	Limits per trial	Metric
GSM8K 0-shot	1,319	Evalchemy (no agent)	thinking disabled	accuracy, flexible-extract
SWE-bench Verified r100	100	mini-swe-agent 2.1.0	1,800 s; 32,768-token context	tasks solved / 100
TBLite-100	100	Terminus-2	1,800 s; 32,768-token context	tasks solved / 100
BFCL-Parity	123	`pi` agent 0.87.0	1,800 s; verifier 300 s	tasks solved / 123

- **GSM8K** is grade-school arithmetic word problems. The model answers with no examples in the prompt and with its thinking mode switched off (`enable_thinking: false` in the config). It runs in Evalchemy, an evaluation runner built on the LM Evaluation Harness. The score is the share of 1,319 test problems whose extracted final number is correct (the `flexible-extract` answer filter). It checks that fine-tuning did not damage basic reasoning.
- **SWE-bench Verified r100** is a fixed random 100-task subset of SWE-bench Verified: real GitHub issues in Python repositories, each with hidden tests[13]. The model runs as the mini-swe-agent coding agent inside a sandbox with the repository. It works by issuing bash commands and ends the episode by issuing the submit command (`echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`). A task is solved when the hidden tests pass on the final working tree. The verifier grades the working tree whether or not the agent submitted, so a task can be solved without a submit.
- **TBLite-100** is OpenThoughts TBLite 2.0, all 100 tasks: terminal tasks (system administration, data processing, small programs) solved through a tmux shell with the Terminus-2 agent. The

[13] SWE-bench ids: `DCAgent2/swebench-verified-random-100-folders` @ `0c553bd6d05d451907afb8db31dba84759ccc993`, mapped to the policy registry. TBLite: all 100 tasks of `openthoughts-tblite` 2.0. Both frozen in `mopd2/sweep/screen-task-ids.json` (SHA-256 `baf99a92ded65a32175e69b6aa0e3e4147f804eff93cc57c5b6cad004598ea52`), also at `s3://marin-us-east-02a/marin/experiments/task203-sweep/screens/screen-task-ids-20261017.json`.

episode ends when the agent marks the task complete or the 1,800 s agent limit expires. A task's verifier gives a reward between 0 and 1. A task counts as solved only at reward 1; partial credit is not counted.

- **BFCL-Parity** is a 123-task port of the Berkeley Function Calling Leaderboard to the Harbor harness, run with the `pi` agent (a minimal tool-calling agent). Each task asks for one or more correctly formed tool calls, or (in the `irrelevance` categories) for the model to decline because no offered tool fits. Section 6.6 splits it by category. We count BFCL-Parity as a **retention** screen, one that should not drop, because it is not the training target; every student in fact gains on it, mostly by learning to decline.

All three agentic screens run through Harbor, an open-source agent evaluation framework that starts each task in a remote sandbox (Daytona), runs the agent, and runs the task's verifier. They run one trial per task, with up to 10 automatic retries of a trial that fails on a known infrastructure error. The frozen task lists are hashed in `screen-task-ids.json` (SHA-256 `baf99a92ded6...`)[13].

4.2 Composite score, bounds and unscored trials

The **composite** is the screening score used throughout: the mean of the two agentic screens on fixed 100-task denominators,

$$\text{composite} = \tfrac{1}{2}\,(\text{SWE solved} + \text{TBLite solved}).$$

It is our screening convention, not a published metric. A trial is **unscored** when no verifier result exists because of an infrastructure failure (sandbox start timeout, tmux transport error, a verifier that never finished). Unscored trials are counted both ways: as failures (lower bound) and as successes (upper bound). A composite written as `20.00-21.00` therefore means two unscored agentic trials. The range is a bound for missing outcomes, not a confidence interval.

Timeouts are scored failures, not unscored trials. When a TBLite agent hits its 1,800 s limit (`AgentTimeoutError`), the verifier still runs on the final state and the trial is scored, almost always as unsolved. The tables report timeouts separately because they explain most of the TBLite losses (Section 5).

Submits. For SWE-bench, "submits" counts episodes whose mini-swe-agent exit status is `Submitted`, i.e. the agent issued its submit command. Episodes that hit the 32k context limit end with exit status `ContextLimit` and are still scored on their final diff, so solved can exceed submits.

Infrastructure retries. After a screen's model server ("serve") finishes, any trial left unscored by an error in Harbor's retry taxonomy (its list of known, retryable infrastructure errors) is retried with the launcher's built-in option (`--retry-unscored-harbor-trials`), which re-runs only those trials into the original results archive. Final counts are read from that archive. Errors outside the taxonomy (`DaytonaError`, `NotFoundError`) fail the record and are not retried.

Verifier loops. A few tasks repeatedly failed in their verifier while holding the whole 8-GPU serve. Two BFCL screens were cancelled by decision once the loop was clear; their looping task counts as unscored and their BFCL score is a trial-level count without a record file (55/121 and 58/122). BFCL is not part of the composite. Details and the SWE-bench case (`psf__requests-2317`, scored unsolved after its retry) are in 8.1.

4.3 Noise and paired comparisons

One trial per task makes every screen noisy. The only direct repeat is parent v0 on SWE-bench r100 under two harness installs: 33/100 both times, with 25 tasks solved in both runs, 8 only in the first and 8 only in the second. Single-task outcomes therefore flip often, and differences of a few tasks between two checkpoints are within noise.

Comparisons between two checkpoints use a **paired per-task test** on the frozen task ids. For each task, a run either solved it or not. The paired difference is $\sum_t (s_t^A - s_t^B)$ over tasks scored in both runs, where $s_t = 1$ if task t is solved. The 95% interval comes from 10,000 bootstrap resamples of tasks (seed `20261022`). A **sensitivity range** over all 100 tasks counts one run's unscored tasks as failures and the other's as successes, then the reverse. We call a difference established only when its 95% interval excludes 0. Per-task transition tables (solved, submitted, context limit) are in Appendix D. Script: `t238/paired.py`; night-of-2026-10-22 batch: `t238/run_pairs_night.py`[14].

4.4 Serving, placement and records

Each checkpoint was served by vLLM (an open-source inference server) on one node of 8 H100 GPUs (data-parallel 8, one model copy per GPU), from the regional object-storage copy of its Hugging Face (HF) export, with `max_input_tokens` 32,768 and `max_output_tokens` 8,192 per call. All four screens of a checkpoint share one serve. All screen serves ran on the CoreWeave cluster `cw-rno2a`. Their scheduling priority (Iris "band") changed twice: production band from 2026-10-20, batch band from 2026-10-22, interactive band from 2026-10-23. Lower bands are evicted more often by other jobs. An eviction pauses the screen; the harness resumes from the trials already saved and re-runs only trials that were in flight. 8 lists the evictions that touched the numbers.

Each screen writes a record: a FineStore dataset (Marin's store for evaluation outputs) plus a Harbor results archive. Retried trials leave earlier attempts in the archive, so final counts take each task's scored attempt from the archive (`t238/final_counts.py`, the same loader as the paired test), and GSM8K is read through FineStore's `ReadView`, which hides superseded rows. All serving was on H100 GPUs (8 explains why not TPU)[15].

4.5 Evaluation pins

Table 15: Evaluation code pins, in the order they were adopted. Look at the third column: after `ba5b739d24` moved Harbor from `016dd78b` to `38742b0f`, no commit changes scoring (policy `4d45932e39`, Evalchemy `c1d5c62c`); the later commits change submission and serving only.

Worktree commit	Used for	Change against the previous commit
`9af01aa66e`	v5 arm screens (2026-10-18)	arm screen configs; quieter GPU-communication logs (NCCL) on serves
`ba5b739d24`	v5 SWE-bench reruns	Harbor pin `016dd78b` to `38742b0f` (mini-swe-agent install fix)
`d9c5646b6d`	from 2026-10-19 08:20 UTC	GPU serves submitted non-preemptible
`dbb92ac80b`	v0.1 reference and all v7 screens	v0.1 reference served from its regional copy
`3d9fb7544a`	relaunches from 2026-10-22 20:22 UTC (a)	backport of #10475: forward the submitter's storage keys
`caba0fa38b`	from 2026-10-23 03:22 UTC; current pin	backport of #10351: tool-package catalog fetched at tool time (8)

(a) The screens relaunched from 2026-10-22 20:22 UTC ran from `dbb92ac80b` with this backport applied as an uncommitted change to `lib/iris/src/iris/cluster/types.py`; the identical change was committed as `3d9fb7544a` at about 03:12 UTC on 2026-10-23, together with `caba0fa38b`. Table 15 lists all pins.

The policy pin is the commit of Marin's evaluation-policy branch, which fixes the benchmark versions, limits and scoring rules. The two 2026-10-23 backports change job submission and serve start-up only; scoring code and the three pins are unchanged. The eval worktree was not rebased onto current main: its policy branch is 226 commits off main, main now vendors Harbor and Evalchemy, and a trial rebase

[14] Scripts live in the screens operator's scratch area (`t238/paired.py`, `t238/final_counts.py`, `t238/run_pairs_night.py`); outputs are copied into `mopd2/arms/screens.md` and `t238/pairs-night/`.

[15] Marin issue #10250: a TPU-served Snowball checkpoint (Step12) diverges from the same checkpoint on H100 past token 2,048.

conflicted in the Evalchemy runner and the scoring code on its first commit. A rebase would therefore change scoring and break comparability with the screens in this report[16].

SWE-bench install fix (8.1). Every SWE-bench number from the v5 reruns onward is from Harbor `38742b0f`; the pilot-sweep screens ran earlier on `016dd78b`, before the break, and parent v0 scored 33/100 on both[17].

4.6 Checks on the TBLite limits

Parent v0.1 times out on 49 of 99 TBLite trials (v0: 8). Two single-run checks on v0.1 tested whether the 1,800 s limit or the 32k context, not the model, cause its TBLite score (Table 16). Neither moved the solved count by more than 3 tasks, and the per-task overlap shows as much churn as gain. A GLM 5.3 reference row on the same task lists solved 79 TBLite tasks under the same limits, so the limits do not prevent a concise strong model from solving most tasks.

Table 16: TBLite limit checks on parent v0.1, and a GLM 5.3 reference row (not a screen). Look at the overlap column: raising the time or context limit swaps which tasks are solved about as much as it adds; on these single runs the limits do not account for v0.1's score.

Run	TBLite solved	Timeouts	Per-task overlap with the 1,800 s / 32k run
v0.1, 1,800 s, 32k (reference)	23 / 99	49	
v0.1, 5,400 s, 32k	24 / 100	27	14 both; 10 only at 5,400 s (3 needed > 1,800 s); 9 only at 1,800 s
v0.1, 1,800 s, 65,536-token context	26 / 100	30	11 both; 15 only at 64k; 12 only at 32k
GLM 5.3, high effort (teacher)	79 / 97	5	not a screen; SWE-bench r100 78 / 99, 55 submits

5 Results: the initial experiment

Takeaway. Every student trained on GLM 5.3 traces scores below its own parent on both agentic screens, while GSM8K and BFCL-Parity rise. On SWE-bench the students nearly stop issuing the submit command and run into the 32k context; on TBLite most of their losses are timeouts. Among the methods, only relay on the LOW rows from v0.1 beats its plain twin on SWE-bench with an interval that excludes 0 (+8 tasks). No arm in this experiment improves its parent.

Parents, arms, row sets (v5, HIGH, LOW) and methods are defined in 1 and C; each arm is compared with its **plain twin** (plain SFT on the same rows and parent) and with its parent. **Plain SFT matched to the relay budget** (run name `plain-matched-relay`) is plain SFT cut to the relay arm's smaller SWE pool (2). Submits and the composite: 4.

All numbers below are from the four screens of Section 4. "Solved (submits)" for SWE-bench gives tasks solved and episodes ending in a submit; "solved (timeouts)" for TBLite gives tasks solved and trials that hit the 1,800 s limit.

5.1 Parents

The difference between the parents on the agentic screens is not resolved. Paired on the frozen tasks (Table 17), v0 minus v0.1 is +7 SWE-bench tasks [-2, +16] (both solve 19; only v0 solves 14; only v0.1 solves 7) and -3 TBLite tasks [-12, +6] on 98 complete pairs (both 11; v0 only 9; v0.1 only 12). v0.1 timed out on 49 TBLite tasks; v0 solved 3 of those. 55 of the 100 TBLite tasks were never solved by either parent under any condition we ran, so most of v0.1's timeouts are on tasks neither parent solves. v0 submits more often on SWE-bench (56 vs 36 episodes). Because neither parent was clearly better,

[16] Decision recorded in `mopd2/arms/screens.md` ("Eval pin"), 2026-10-23.

[17] Marin issue #10262 and the project's incident record 789. Harbor `38742b0f` installs mini-swe-agent 2.1.0 together with mcp 2.3.0.

Table 17: The two parents on the four screens. Composite: our screening mean of SWE-bench and TBLite solved (4). v0 leads on SWE-bench, v0.1 on TBLite, and v0.1 times out on 49 TBLite tasks against v0's 8. Every later table repeats the parent's row as the reference.

Parent	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
v0	85.2	33 (56)	20 / 99 (8)	27 / 122	26.50-27.00
v0.1	91.1	26 (36)	23 / 99 (49)	32 / 123	24.50-25.00

HIGH plain was trained on both parents in the v7 round; the other HIGH methods then ran on v0 only (1).

5.2 First round (v5): maximum-effort GLM traces on v0

Table 18: v5 arms: GLM 5.3 traces at maximum reasoning effort, parent v0, LR 1e-6, 3 epochs (792 updates, final step `step-791`; 28.93M supervised tokens per epoch). Look at the submits and timeouts columns: the arms almost never submit and time out on most TBLite tasks.

Model	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
parent v0	85.2	33 (56)	20 / 99 (8)	27 / 122	26.50-27.00
v5 plain	90.0	15 (0)	8 / 94 (67)	46 / 123	11.50-14.50
v5 RS-multi	89.7	4 (0)	3 / 90 (67)	59 / 123	3.50-8.50
v5 recovery	89.3	13 (1)	8 / 98 (63)	53 / 123	10.50-11.50

All three arms gain 4-5 points on GSM8K and 19-32 tasks on BFCL, and lose most of their agentic score (Table 18). Two distinct failures explain the loss[18]:

1. **No submit on SWE-bench.** Counting every recorded trajectory (100 tasks plus retried attempts), the arms issued the submit command in 0 of 102 (plain), 0 of 102 (RS-multi) and 1 of 103 (recovery); v0, on the same basis (every recorded trajectory including retries), did in 70 of 110. Every episode except recovery's one submit ran into the 32k context. The arms' solves are correct diffs left in place when the context ran out. They are not verbose on SWE-bench: plain takes 37 steps per episode at a median of 117 completion tokens per step, against v0's 37 at 148.

2. **Too slow for TBLite.** 63-67 of about 95 TBLite trials hit the 1,800 s limit (v0: 8). On trials that finished in time, plain solves 7 of 27, close to v0's 20 of 91. Plain takes 38 steps per trial (v0 14) at a median of 1,075 tokens per step with a long tail (p99 23,736 vs 10,231), at the same decoding speed (9.2 s per 1,000 tokens).

Both failures match properties of the training data (2 gives the v5 data statistics). In the v5 generation, 47.6% of GLM's 9,629 passing SWE-smith trials passed without a submit (the 30k generation window cut them off and the verifier scored the working tree), and "first passing attempt" selection put those into 56% of the SWE rows. 72% of the trained TBLite-family (TMax) tokens were reasoning, because GLM wrote the pool at maximum reasoning effort. The v5 comparison therefore cannot separate the methods, and the data was regenerated (v7): clean completions only, at HIGH and LOW reasoning effort, on both parents.

Table 19: v7 students of parent v0.1. HIGH plain: 705 updates (final `step-704`) on 19.42M supervised tokens per epoch; LOW plain: 675 updates on 11.60M; LOW relay: 963 updates on the same 11.60M budget. All at LR 1e-6, 3 epochs, final checkpoint. Compare each row's composite with the parent's first row, and the submit counts of the HIGH and LOW students.

Model	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
parent v0.1	91.1	26 (36)	23 / 99 (49)	32 / 123	24.50-25.00
HIGH plain v0.1	92.3	13 (6)	8 / 98 (41)	67 / 123	10.50-11.50
LOW plain v0.1	91.7	17 (27)	12 / 93 (42)	55 / 121	14.50-18.00
LOW relay v0.1	92.9	25 (25)	15 / 98 (64)	64 / 123	20.00-21.00

5.3 Second round (v7): HIGH and LOW rows, two parents, five methods

LOW plain v0.1's BFCL score is a trial-level count without a record file: one task looped on verifier timeouts, the screen was cancelled by decision, and 2 tasks are unscored.

Table 20: v7 students of parent v0 on the HIGH rows. Plain SFT matched to the relay budget is cut to the relay arm's smaller SWE pool (618 updates, because its SWE share is cut to the relay pool's 4.87M tokens while TMax stays at 11.88M); relay v0 ran 924 updates; the others 705. Compare every method with HIGH plain v0 (second row): none is above it on the composite.

Model	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
parent v0	85.2	33 (56)	20 / 99 (8)	27 / 122	26.50-27.00
HIGH plain v0	87.7	17 (3)	9 / 91 (unknown)	44 / 122	13.00-17.50
plain-matched-relay v0	91.1	13 (6)	11 / 99 (70)	54 / 123	12.00-12.50
RS-multi v0	89.9	11 (6)	10 / 99 (54)	41 / 123	10.50-11.00
recovery v0	88.5	12 (4)	9 / 99 (68)	48 / 123	10.50-11.00
relay v0	89.0	13 (1)	14 / 99 (68)	48 / 123	13.50-14.00
plateau v0	89.8	14 (6)	13 / 100 (54)	49 / 123	13.50

HIGH plain v0's TBLite row is the record's count (91 scored). The paired loader counts 92, because one trial that the record treats as an infrastructure failure still left a verifier reward; Table 21 uses 92. Its timeout count after two retries is unknown (43 of the first 86 scored trials timed out).

The v5 pattern repeats on clean, shorter v7 rows. Every v7 student gains on GSM8K (+0.6 to +5.9 points over its parent) and on BFCL (+14 to +35 tasks) and loses on both agentic screens. The HIGH students submit on SWE-bench in 1-6 of 100 episodes; the LOW students of v0.1 still submit in 25-27. The methods do not change the picture on the HIGH v0 rows: all five arms sit within or below HIGH plain v0's composite range (Tables 19 and 20).

5.4 Paired comparisons

Measured:

- **Methods against plain.** Relay on the LOW v0.1 rows is the only method whose SWE-bench difference from its plain twin excludes 0 (+8 [+1, +16]; 5.6). On the HIGH v0 rows no method differs from plain on either screen: SWE-bench -3 to -6, TBLite 0 to +3, every interval containing 0.
- **Students against parents.** Every v7 student is below its parent on TBLite (-7 to -15) and, except LOW relay v0.1, on SWE-bench (-9 to -22). Most intervals exclude 0. The two closest are LOW relay v0.1 on SWE-bench (-1 [-10, +8]) and LOW plain v0.1 on SWE-bench (-9 [-19, +0]).

[18] `mopd2/arms/screens.md`, "SWE r100 failure anatomy (plain vs v0)", the rs-multi and recovery anatomy paragraphs, and "TBLite verbosity breakdown"; script `t238/swe_anatomy.py` and `t238/tblite/*.py`.

Table 21: Paired per-task differences (student minus reference, tasks scored in both), with 95% task-bootstrap intervals and sensitivity bounds over all 100 tasks. Bold rows: interval excludes 0. Only one method row (relay vs plain on LOW rows) is bold; almost every student-vs-parent row is.

Comparison	Screen	Solved (A vs B)	Scored in both	Gained / lost	Difference [95%]	Sensitivity
LOW relay v0.1 vs LOW plain v0.1	SWE	25 vs 17	100	12 / 4	**+8 [+1, +16]**	+8 to +8
LOW relay v0.1 vs LOW plain v0.1	TBLite	15 vs 12	91	6 / 5	+1 [-6, +7]	-4 to +5
LOW relay v0.1 vs parent v0.1	SWE	25 vs 26	100	10 / 11	-1 [-10, +8]	-1 to -1
LOW relay v0.1 vs parent v0.1	TBLite	15 vs 23	97	11 / 18	-7 [-18, +3]	-9 to -6
plain-matched-relay v0 vs HIGH plain v0	SWE	13 vs 17	100	6 / 10	-4 [-12, +4]	-4 to -4
plain-matched-relay v0 vs HIGH plain v0	TBLite	11 vs 9	91	6 / 4	+2 [-4, +8]	-6 to +3
RS-multi v0 vs HIGH plain v0	SWE	11 vs 17	100	5 / 11	-6 [-14, +2]	-6 to -6
RS-multi v0 vs HIGH plain v0	TBLite	10 vs 9	92	6 / 5	+1 [-5, +8]	-7 to +2
recovery v0 vs HIGH plain v0	SWE	12 vs 17	100	6 / 11	-5 [-13, +3]	-5 to -5
recovery v0 vs HIGH plain v0	TBLite	9 vs 9	92	6 / 6	+0 [-7, +7]	-8 to +1
relay v0 vs HIGH plain v0	SWE	13 vs 17	100	5 / 9	-4 [-11, +3]	-4 to -4
relay v0 vs HIGH plain v0	TBLite	14 vs 9	91	5 / 2	+3 [-2, +8]	-3 to +6
plateau v0 vs HIGH plain v0	SWE	14 vs 17	100	5 / 8	-3 [-10, +4]	-3 to -3
plateau v0 vs HIGH plain v0	TBLite	13 vs 9	92	7 / 4	+3 [-3, +9]	-4 to +4
plain-matched-relay v0 vs parent v0	SWE	13 vs 33	100	2 / 22	**-20 [-29, -11]**	-20 to -20
plain-matched-relay v0 vs parent v0	TBLite	11 vs 20	98	5 / 14	**-9 [-17, -1]**	-10 to -8
RS-multi v0 vs parent v0	SWE	11 vs 33	100	3 / 25	**-22 [-32, -13]**	-22 to -22
RS-multi v0 vs parent v0	TBLite	10 vs 20	99	5 / 15	**-10 [-19, -2]**	-11 to -9
recovery v0 vs parent v0	SWE	12 vs 33	100	3 / 24	**-21 [-31, -12]**	-21 to -21
recovery v0 vs parent v0	TBLite	9 vs 20	99	6 / 17	**-11 [-20, -2]**	-12 to -10
relay v0 vs parent v0	SWE	13 vs 33	100	3 / 23	**-20 [-30, -11]**	-20 to -20
relay v0 vs parent v0	TBLite	14 vs 20	98	8 / 15	-7 [-16, +2]	-7 to -5
plateau v0 vs parent v0	SWE	14 vs 33	100	3 / 22	**-19 [-28, -10]**	-19 to -19
plateau v0 vs parent v0	TBLite	13 vs 20	99	7 / 14	-7 [-16, +2]	-8 to -7
HIGH plain v0.1 vs parent v0.1	SWE	13 vs 26	100	6 / 19	**-13 [-23, -4]**	-13 to -13
HIGH plain v0.1 vs parent v0.1	TBLite	8 vs 23	97	4 / 19	**-15 [-24, -6]**	-16 to -13
HIGH plain v0 vs parent v0	SWE	17 vs 33	100	4 / 20	**-16 [-25, -7]**	-16 to -16
HIGH plain v0 vs parent v0	TBLite	9 vs 20	92	7 / 17	**-10 [-20, -1]**	-12 to -3
LOW plain v0.1 vs parent v0.1	SWE	17 vs 26	100	7 / 16	-9 [-19, +0]	-9 to -9
LOW plain v0.1 vs parent v0.1	TBLite	12 vs 23	93	7 / 17	**-10 [-19, -1]**	-12 to -4

Table 21 lists all 30 comparisons. The v5 arms were not paired against v0; their gaps (18-29 SWE tasks) are far outside the noise of Section 4.3.

5.5 Trajectory shape: what the students copy

The students reproduce the per-step style of their training rows but not the episode shape. We measured each SWE-bench episode on the agent's own steps: steps per episode, completion tokens per step, context length at the final step, whether the last step is a submit, and the step mix (each step classed by its strongest bash command: submit > edit > test > run > probe > other). Rows are the SWE-smith training rows (passing episodes only); students are every SWE-bench r100 trial with a complete trajectory in the record read for this table, which for three students is one or two fewer than the final paired count after retries[19]. **ATIF** is Harbor's per-step trajectory format.

Table 22: SWE-bench episode shape: HIGH training rows (the rows built for parent v0.1; the v0 rows are byte-identical) against every v7 student. Compare tokens per step (students match the rows) with steps, final context and last-step submit (students do not).

Episodes	Count	Steps med / p90	Tokens per step med / p90 / mean	Final context med / p90	Final step > 20k / 25k / 30k	Last step = submit	Step mix edit / probe / run / test / other
HIGH rows (passing only)	1,317	25 / 38	120 / 503 / 221	15,858 / 27,022	33% / 17% / 2%	100%	not measured
LOW plain v0.1	100	54 / 74	121 / 474 / 219	32,087 / 32,590	82% / 79% / 74%	27%	27 / 33 / 30 / 7 / 3
LOW relay v0.1	99	53 / 69	127 / 539 / 235	31,975 / 32,524	84% / 80% / 77%	25%	31 / 30 / 28 / 8 / 2
HIGH plain v0.1	100	50 / 64	126 / 571 / 247	32,228 / 32,628	97% / 96% / 95%	6%	33 / 33 / 25 / 7 / 3
HIGH plain v0	98	51 / 65	120 / 606 / 250	32,275 / 32,638	99% / 98% / 96%	3%	34 / 33 / 22 / 8 / 3
plain-matched-relay v0	100	49 / 65	135 / 586 / 247	32,071 / 32,560	98% / 97% / 93%	6%	30 / 33 / 27 / 7 / 3
RS-multi v0	100	55 / 66	115 / 553 / 232	32,309 / 32,626	97% / 96% / 94%	6%	30 / 34 / 25 / 7 / 4
recovery v0	100	47 / 65	123 / 614 / 255	32,247 / 32,602	97% / 97% / 94%	4%	32 / 34 / 23 / 9 / 3
relay v0	100	48 / 59	133 / 685 / 290	32,266 / 32,606	100% / 100% / 97%	1%	31 / 41 / 21 / 5 / 3
plateau v0	99	55 / 67	112 / 538 / 225	32,292 / 32,620	99% / 98% / 97%	6%	33 / 33 / 21 / 10 / 3

Measured:

- Every student keeps the rows' tokens per step (median 112-135 against the rows' 120) and takes about twice the rows' steps (47-55 against 25; Table 22).
- 93-97% of every HIGH-row student's episodes end with a final step above 30k context; at most 6% end with a submit. The rows end with a submit in 100% of episodes, at a median final context of about 15.9k.
- The LOW students of v0.1 submit in 25-27% of episodes. HIGH plain v0.1 has the same parent and submits in 6%, so in these runs the submit gap goes with the training rows (LOW vs HIGH), not with the parent or the method. Relay on the LOW rows keeps LOW plain's shape and submit rate and solves more.
- The LOW rows themselves have a median of 14 SWE steps; the LOW plain student takes 54. Its command mix matches the LOW rows within 3 points per class (bash commands that run code: rows 44.2%, student 43.7%; read: 24.1% and 26.9%; edit: 16.4% and 16.1%; `mopd2/arms/screens.md`, "v7 SWE per-step style"). Three context-limited trials read turn by turn show many short probe turns (one `grep` or one Python probe each) and few edits, until the context runs out near step 55-60.

Measured: parent v0 reasons at length in nearly every step (a think block in 99.8% of steps; 62 reasoning and 36 action tokens at the median). Parent v0.1 writes no think block in 66% of its steps. Neither serve inserts the think-block opening token into the prompt on the model's behalf, so these are steps where the model chose not to reason. Every row set is action-heavy per step, so no training data

[19] `t238/swe_shape.py` (students, from the ATIF agent steps; final context = prompt tokens of the last agent step) and `t238/rows_shape.py` (rows: one supervised span = one turn). The edit class counts any output redirect, so it slightly overcounts edits.

Table 23: Parents' own SWE-bench episodes against the training row sets. Pilot rows are the contributor SWE-smith traces used in the learning-rate sweep (6.1). Reasoning share: generated tokens inside the think block. Solved-only parent values in brackets. Look at the last two columns: v0 reasons in every step, v0.1 rarely, and every row set spends most of each step on the action.

Episodes	Count	Steps med / p90	Tokens per step med	Final context med	Finish at last step	Reasoning share	Reasoning / action tokens per step (med)
parent v0, all [solved 33]	100	37 / 49 [34]	148	29,579	56% submit	68.6% [63.1%]	62 / 36 [59 / 39]
parent v0.1, all [solved 26]	100	42 / 57 [39]	75	31,724	36% submit	26.4% [19.6%]	0 / 45 [0 / 52]
pilot rows (Kimi-2.5, Terminus-2)	1,995	14 / 23	246	12,984	100% `task_complete`	28.2%	45 / 171
HIGH rows v0.1 (GLM)	1,317	25 / 38	120	15,857	100% submit	39.0%	21 / 69
LOW rows v0.1 (GLM)	1,468	14 / 31	90	6,384	100% submit	34.5%	0 / 65

matches v0's per-step style; the HIGH rows are the nearest to it (Table 23). The LOW rows' many no-reasoning steps resemble v0.1's.

Caveats: rows hold passing episodes only, the parents all episodes (the solved-only values move little); rows are SWE-smith tasks and the parents ran SWE-bench Verified; tokens per step is the supervised span for rows and completion tokens for parents.

Hypothesis, not tested: the students learn the rows' local habits (step length, command mix, closing format), which is all GSM8K and BFCL need, but not the rows' decision of when to edit and when to stop, so on long agentic episodes they drift from the parent's own policy without acquiring the teacher's. 6.7 compares this with the pilot student (the learning-rate sweep's 1e-6 checkpoint, trained on contributor Kimi and Qwen traces), which kept most of its parent's submit rate.

5.6 The relay result on LOW rows

Relay gives opposite outcomes on the two row sets. On the HIGH v0 rows (a SWE relay pool smaller than the plain budget, 2) relay v0 submits in 1 of 100 SWE-bench episodes and is not distinguishable from plain. On the LOW v0.1 rows (token-matched to plain; about 70% of the final pool's passing relay episodes end cleanly, no fallback needed) LOW relay v0.1 submits in 25 and solves 8 more SWE-bench tasks than LOW plain v0.1 (+8 [+1, +16]); in this experiment it is the only arm above its plain twin on all four screens (composite 20.00-21.00 vs 14.50-18.00; three arms of the later LOW grid also are, Table 28), and still not above its parent (composite 24.50-25.00; SWE-bench -1 [-10, +8]). Relay is a whole recipe here: the student's own opening turns are masked from the loss and the arm trains for more updates at the same supervised token budget (963 vs 675), so the gain cannot be attributed to any one of these parts. One run per arm.

5.7 Review points: interpretation checks for the initial GLM experiment

The initial GLM rows did not produce an observed agentic improvement over the corresponding parent in the tested arms. This statement is scoped to the GLM experiment; the contributor follow-ups are reported separately. Similar losses under HIGH plain SFT on both parents do not establish that the parent has no effect. The parent comparison and its paired intervals leave room for interactions with the data and method.[20]

LOW relay v0.1 improves SWE by 8 solves over LOW plain, with a task-bootstrap interval of [+1, +16]. It has 25 trajectories ending with an explicit submission, versus plain's

1. These count trajectories, not the number of submit commands. Thus its improvement

does not arise from a higher aggregate submit count. The interval describes task resampling of the observed trials; it does not include repeated-generation variability or a correction for the full set of method comparisons.

[20] `sft-shootout/review-20261022-owner-summary.md`; `mopd2/arms/screens.md`, paired per-task tables and final retry-adjusted rows.

The policy's formal decisive-improvement test was not computed for the arms, and no arm was promoted under that rule. The paired SWE bootstrap result alone cannot substitute for that uncomputed test, which has its own standard-error criterion. An observed method-vs-plain improvement and a formal promotion decision are separate claims.

HIGH and LOW differ in reasoning effort, selected passing episodes, lengths and token budget. Their agentic budgets are 19,418,246 and 11,596,239 tokens per epoch respectively. The plain comparison therefore cannot isolate an effect of shorter or more decisive traces. Relay further changes the conditioning states, masked prefixes and number of updates. HIGH plain SFT matched to the relay budget is separate; other HIGH arms were not cut down to the relay pool.

Fewer explicit submissions and more context-limit endings describe the students' behavior, but do not identify its cause. The rows themselves contain completion signals. A student that solves less effectively may spend longer acting and reach the context limit before submitting. Conversely, training may alter stopping behavior directly. The present results do not separate these explanations.

Genuine agent timeouts are scored outcomes; sandbox or transport failures without a score are unscored. Composite ranges formed by treating unscored tasks as fail or pass are coverage bounds, not confidence intervals. Neither a reduction in timeout count nor a similar point score establishes equivalence between parents or runtime limits.

6 Follow-up experiments

Takeaway. Within the tested ranges, the training rate, the number of updates and the method do not account for the students' losses. Changing the data source does change the outcome: plain SFT on the contributors' Kimi-2.5 and Qwen3.6-27B traces gives students above their parents in point estimate on both agentic screens (paired intervals contain

1. and clearly above the GLM-trace students, under the same recipe and evaluation. The

sources differ in more than the teacher (task set, harness format, episode shape), so this does not identify which property of the data matters.

6.1 Pilot learning-rate sweep (contributor rows)

Question: which learning rate and epoch count should the arms use? Before the arm runs, it was chosen by a sweep on the **pilot set**, an earlier SFT data set built from external contributors' agent traces (the **contributor rows** below are the same traces converted to the arm format): the contributors' traces (Kimi-2.5 on SWE-smith in Terminus-2 format, Qwen3.6-27B on TMax with native tool calls), 8,768 rows (the converter described in 2.4 reproduces its 7,393 agentic training trajectories) and 34.4M supervised tokens, parent v0, four rates, 3 epochs each. Held-out loss is on a held-out slice of the same task families (Table 24; screens in Table 25).

Table 24: Pilot sweep, held-out loss (pooled over TMax and SWE-smith) after each epoch. Lower is better. 3e-7 keeps falling, 1e-6 is flat, 3e-6 and 1e-5 rise after epoch 1.

LR	Epoch 1	Epoch 2	Epoch 3
3e-7	0.289	0.278	0.272
1e-6	0.272	0.268	0.273
3e-6	0.271	0.276	0.308
1e-5	0.297	0.312	0.349

The composites of the three epoch-3 finals at 3e-7, 1e-6 and 3e-6 (25.00-26.50, 31.00-32.00 and 29.50-30.00) differ by less than the recipe's comparison threshold: 1.96 times the composite's standard error,

Table 25: Pilot sweep screens (parent v0). Epoch 3 is the final checkpoint. Composite: our screening mean of SWE-bench and TBLite solved (4). Look at the composite: the 3e-7, 1e-6 and 3e-6 finals overlap within noise; 1e-5 is lower.

Checkpoint	SWE solved / scored	TBLite solved / scored	GSM8K (of 1,319)	BFCL-Parity	Composite
v0	33 / 100	20 / 99	1,124	27 / 122	26.50-27.00
3e-7 epoch 2	30 / 100	15 / 98	1,164	73 / 123	22.50-23.50
3e-7 epoch 3	33 / 100	17 / 97	1,186	70 / 123	25.00-26.50
1e-6 epoch 2	29 / 99	25 / 97	1,215	70 / 123	27.00-29.00
1e-6 epoch 3	33 / 99	29 / 99	1,222	73 / 123	31.00-32.00
3e-6 epoch 2	27 / 100	23 / 99	1,221	81 / 123	25.00-25.50
3e-6 epoch 3	33 / 100	26 / 99	1,235	68 / 123	29.50-30.00
1e-5 epoch 2	22 / 100	10 / 93	1,175	46 / 123	16.00-19.50

about 9 points on 100-task screens[21]. That is an unpaired test; no paired per-task test was run on the sweep. The 1e-5 checkpoint was screened only at epoch 2, where it is the lowest of all checkpoints on SWE-bench, TBLite and BFCL. The frozen recipe's tie rule is: "Prefer the smaller LR within noise, but if multiple rates remain tied with the smallest, report the ambiguity to the owner before final selection"[22]. Three rates tied with the smallest, so the choice went to the owner, who confirmed LR 1e-6 for 3 epochs on 2026-10-18. This is a practical selection, not a unique statistical winner. The 1e-6 final is the "pilot student" used as a reference below.

6.2 Learning-rate check on GLM rows (3e-7 vs 1e-6)

The sweep chose the rate on contributor rows, not on GLM rows. To test whether 1e-6 was too aggressive for the GLM rows, plain SFT on the LOW rows from parent v0 was trained at 3e-7 next to its 1e-6 twin, same rows and schedule (675 updates).

Table 26: Learning-rate check: plain SFT on LOW rows, parent v0. The lower rate is worse on every column.

LR	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
1e-6	89.30	18 (31)	13 / 97 (37)	50 / 123	15.50-17.00
3e-7	87.04	11 (26)	7 / 100 (40)	39 / 123	9.00

The gentler rate is worse on every screen (Table 26). Paired, 3e-7 minus 1e-6 is -7 SWE-bench tasks [-13, -1] and -6 TBLite tasks [-14, +1]. A lower rate does not recover the parent's agentic score.

6.3 Update-dose check (epoch 2 vs final)

The HIGH plain v0.1 run also exported its epoch-2 checkpoint (`step-469`, after 470 of 705 updates). If the students lost the agentic skills by over-training, the earlier checkpoint should score higher.

Table 27: Dose check: HIGH plain v0.1 after 2 and 3 epochs. Both checkpoints sit far below the parent (last row) and submit rarely.

Checkpoint	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
epoch 2 (`step-469`)	91.43	18 (3)	9 / 98 (57)	68 / 123	13.50-14.50
epoch 3 (`step-704`)	92.3	13 (6)	8 / 98 (41)	67 / 123	10.50-11.50
parent v0.1	91.1	26 (36)	23 / 99 (49)	32 / 123	24.50-25.00

[21] `mopd2/sweep/README.md`, correction of 2026-10-18 14:30 UTC (line 212).
[22] `mopd2/sweep/recipe.md`, line 72.

Epoch 2 solves 5 more SWE-bench tasks (Table 27), but the paired interval contains 0 (+5 [-4, +13]; 12 gained, 7 lost). TBLite and BFCL are flat. The epoch-2 checkpoint already submits in only 3 of 100 episodes, so the submit collapse is present by epoch 2. Fewer updates do not restore the parent.

6.4 LOW grid: methods and parents on the LOW rows

Because relay helped only on the LOW rows (Section 5.6), the methods were re-run on the LOW rows for both parents: plain, relay and plateau on v0; plateau, RS-multi and recovery on v0.1 (LOW plain and relay v0.1 are from the initial experiment).

Table 28: LOW grid. All arms on the same LOW rows (11.60M supervised tokens per epoch), LR 1e-6, 3 epochs. Compare each method with the plain row of the same parent.

Parent	Arm	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
v0	plain	89.30	18 (31)	13 / 97 (37)	50 / 123	15.50-17.00
v0	relay	88.78	24 (19)	18 / 99 (64)	50 / 123	21.00-22.00
v0	plateau	91.28	25 (31)	14 / 98 (39)	63 / 123	19.50-20.50
v0.1	plain	91.7	17 (27)	12 / 93 (42)	55 / 121 (a)	14.50-18.00
v0.1	relay	92.9	25 (25)	15 / 98 (64)	64 / 123	20.00-21.00
v0.1	plateau	91.81	18 (20)	15 / 97 (44)	74 / 123	16.50-18.00
v0.1	RS-multi	91.74	17 (20)	11 / 96 (45)	67 / 123	14.00-16.00
v0.1	recovery	93.03	24 (30)	15 / 98 (43)	58 / 122 (a)	19.50-20.50

(a) Trial-level count without a record file: one BFCL task looped on verifier timeouts, the screen was cancelled by decision and the task counts as unscored. LOW relay v0 solved 24 of 99 scored SWE-bench tasks (one unscored), hence its composite range.

Table 28 gives the rows. Paired against the LOW plain arm of the same parent, no method's interval excludes 0 on either screen in this grid: relay v0 +6 [-2, +14] SWE and +4 [-3, +11] TBLite; plateau v0 +7 [-1, +15] and +1 [-6, +9]; recovery v0.1 +7 [-1, +15] and +4 [-3, +11]; plateau v0.1 +1 [-7, +9] and +3 [-5, +11]; RS-multi v0.1 0 [-7, +7] and -1 [-7, +5]. LOW relay v0.1 (+8 [+1, +16], Section 5.4) remains the only established method gain. Against the HIGH-row twin of the same method on v0, the LOW arms are clearly ahead on SWE-bench: relay +11 [+3, +19], plateau +11 [+4, +19]. LOW plain v0 against HIGH plain v0 is +1 [-6, +8]. Every LOW arm of v0 is below its parent in point estimate on SWE-bench; only LOW plain v0's gap is established (-15 [-23, -7]); LOW relay v0 (-8 [-16, +0]) and LOW plateau v0 (-8 [-17, +2]) are not.

6.5 Contributor-data experiment

The pilot student (contributor traces, 1e-6) scored above its parent in point estimate (composite 31.00-32.00 against 26.50-27.00; screened on the earlier Harbor install `016dd78b`). Paired on the frozen tasks it is level on SWE-bench (0 [-7, +7], 33 vs 33) and ahead on TBLite (+8 [-2, +18], 29 vs 20), both intervals containing 0. Every GLM-trace student scored below its parent. To test the data source directly, plain SFT was re-run on the contributors' traces converted to the arm row format, with the arm recipe and both parents, plus RS-multi (resampling across epochs on TMax only; the Kimi SWE set has one trajectory per task) and recovery (injected, masked failures in each harness's own error format) on v0.1. The converter reproduced the pilot's 7,393 training trajectories and 161 held-out tasks; 58 tasks were excluded for parity with the v7 pool. Each run: 12,902,240 supervised tokens per epoch, 226 steps per epoch (678 updates over 3 epochs), LR 1e-6. One caveat applies to the v0 rows of this table: parent v0 itself was built with an earlier SFT graft that trained on 1,012 of these 1,995 contributor SWE-smith trajectories, the same trajectories (2.4). Parent v0.1 has no such graft, which is why the method arms use v0.1.

Table 29: Contributor-data arms (Kimi-2.5 SWE-smith + Qwen3.6-27B TMax rows), grouped by parent. Compare each composite with its parent's row directly above the group.

Arm	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
parent v0	85.2	33 (56)	20 / 99 (8)	27 / 122	26.50-27.00
contributor plain v0	90.90	35 (35)	26 / 100 (23)	68 / 123	30.50
pilot student (sweep 1e-6, v0)	92.6	33 / 99 (40)	29 / 99	73 / 123	31.00-32.00
parent v0.1	91.1	26 (36)	23 / 99 (49)	32 / 123	24.50-25.00
contributor plain v0.1	94.01	29 (32)	28 / 99 (26)	71 / 123	28.50-29.00
contributor RS-multi-TMax v0.1	94.54	29 (37)	24 / 100 (35)	69 / 123	26.50
contributor recovery v0.1	92.04	29 / 99 (29)	29 / 100 (30)	81 / 123	29.00-29.50

Table 30: Contributor arms, paired per-task differences (A minus B) with 95% intervals. Bold: interval excludes 0. Only the rows against GLM-trace students are bold.

A	B	SWE-bench	TBLite
contributor plain v0	parent v0	+2 [-8, +12]	+6 [-2, +14]
contributor plain v0	HIGH plain v0	**+18 [+10, +26]**	**+16 [+9, +24]**
contributor plain v0	pilot student	+2 [-6, +10]	-3 [-11, +5]
contributor plain v0.1	parent v0.1	+3 [-6, +12]	+4 [-6, +14]
contributor plain v0.1	HIGH plain v0.1	**+16 [+7, +25]**	**+19 [+12, +27]**
contributor plain v0.1	LOW plain v0.1	**+12 [+3, +22]**	**+13 [+5, +21]**
contributor RS-multi-TMax v0.1	parent v0.1	+3 [-5, +11]	+1 [-8, +9]
contributor RS-multi-TMax v0.1	contributor plain v0.1	0 [-8, +8]	-4 [-14, +6]
contributor recovery v0.1	parent v0.1	+4 [-4, +12]	+6 [-4, +17]
contributor recovery v0.1	contributor plain v0.1	+1 [-8, +10]	+1 [-8, +10]

Measured (Tables 29 and 30):

- All four contributor students score above their parent on the composite, and every paired difference against the parent is positive on both screens (+1 to +6), with every interval containing 0: above in point estimate, not established (Table 30).
- Every contributor plain arm is clearly above the GLM-trace plain arm of the same parent: +12 to +19 tasks, every interval excluding 0.
- Contributor plain v0 reproduces the pilot student within noise (+2 and -3).
- RS-multi-TMax and recovery against contributor plain v0.1 are not resolved: 0 and +1 on SWE-bench, -4 and +1 on TBLite, every interval containing 0.
- The contributor students keep submitting on SWE-bench (29-37 of 100 episodes; parents 36 and 56), unlike the HIGH-row GLM students (1-6).

The comparison is not matched on tokens or tasks: the contributor rows carry 12.90M supervised tokens per epoch against 19.42M (HIGH) and 11.60M (LOW), and cover different SWE-smith tasks. A matched pair on the 1,432 tasks both sources cover (466 SWE + 966 TMax) is running (11).

6.6 BFCL-Parity by category

Every student scores far above its parent on BFCL-Parity. The split shows which part moves. The 85 tool-call tasks need well-formed calls; the 37 irrelevance tasks need the model to decline because no offered tool fits; one live-relevance task needs a call where one is warranted.

Table 31: BFCL-Parity by category group: solved / scored. The decline column jumps for every student; the tool-call column separates the students.

Model	Tool calls (85)	Decline (37)	Live-relevance (1)	Total
parent v0	15 / 85	12 / 36	0 / 1	27 / 122
parent v0.1	28 / 85	4 / 37	0 / 1	32 / 123
HIGH plain v0.1	42 / 85	25 / 37	0 / 1	67 / 123
LOW plain v0.1	30 / 83	25 / 37	0 / 1	55 / 121
LOW relay v0.1	40 / 85	24 / 37	0 / 1	64 / 123
contributor plain v0.1	46 / 85	25 / 37	0 / 1	71 / 123
contributor plain v0	41 / 85	27 / 37	0 / 1	68 / 123

Table 32: Tool-call categories in detail: solved (of n; scored = n unless shown). Most of the contributor arm's edge is in live-multiple.

Category	n	v0	v0.1	HIGH plain v0.1	LOW plain v0.1	LOW relay v0.1	contrib. plain v0.1	contrib. plain v0
live-multiple	35	7	13	14	9 / 34	12	21	14
live-simple	9	2	4	4	7 / 8	6	4	5
simple-python	13	4	5	8	7	6	7	11
simple-java, simple-javascript	5	0	1	2	0	0	2	0
multiple	7	1	1	5	3	5	3	3
parallel	7	0	2	2	1	5	4	4
parallel-multiple	7	1	1	6	2	6	4	4
live-parallel, live-parallel-multiple	2	0	1	1	1	0	1	0

Measured: against the parents, both parts improve (Table 31). Declining rises from 4/37 (v0.1) and 12/36 (v0) to 24-27/37 for every student, about equally for every training source. Tool calls rise from 28 (v0.1) and 15 (v0) to 30-46. Among the students, the contributor rows' edge is in tool calls, not declining: contributor plain v0.1 makes 46 correct calls against 42 (HIGH plain v0.1) and 30 (LOW plain v0.1) with the same 25/37 declines, most of it in live-multiple (21 vs 14 and 9; Table 32). These are single runs with small categories; no paired interval was computed for this split. Most of the students' BFCL gain over v0.1 is in declining, a behaviour every training source teaches, which is why BFCL works here as a check that tool-calling did not break rather than as a measure of the target skill.

The Qwen3-30B-A3B replication attempt is described in 6.8.

6.7 Style comparison: pilot rows and pilot student

The pilot student kept most of its parent's submit rate on SWE-bench (40 vs 56 episodes), although its SWE rows were in Terminus-2 format and never contained mini-swe-agent's submit command. The HIGH and LOW GLM rows end with that exact submit command in every row, yet their students submit in 1-6 (HIGH) and about 25 (LOW) episodes. The pilot SWE rows are short (14 steps, about 13k context at the finish) with longer, action-heavy steps (246 tokens per step, 28% reasoning share). The pilot student takes 49 steps per episode at 73 tokens per step and reaches the 30k context in 66% of episodes (v0: 49%).

Hypothesis, not tested: training rows in the evaluation harness's own format (mini-swe-agent for the GLM SWE rows) overwrite the parent's harness-specific decision policy more than rows in another harness's format do. State shift between teacher and student trajectories is the competing explanation. Neither is established; the matched pair (11) holds the task set fixed but not the format.

6.8 Qwen replication attempt (no completed sweep)

The design doc proposed repeating the comparison on a Qwen model, to see whether the method ranking holds outside Snowball. The model chosen was `Qwen/Qwen3-30B-A3B-Thinking-2507` at revision

`144afc2f`. No Qwen learning rate was trained to completion, so there are no Qwen results.

What was done:

Data the pilot sweep rows re-rendered in the Qwen chat template, with the same rows and held-out ids; 216 rows were dropped by the 32k cap, mostly replay rows from the finance part of the general SFT slice (manifest `d2acc807...`).

Weights staged and hash-checked against the HF files (16 shards, 61,066,575,656 bytes).

Conversion check the original logit-parity gate failed. The accepted replacement criterion used pooled supervised-token negative log-likelihood (NLL): HF 0.823530981 versus Levanter 0.824872359, a +0.162881% difference, on four prefixes up to 4,096 tokens. Both forwards used BF16; cross-entropy used FP32. Earlier routing evidence compared Levanter BF16 to HF FP32: near-tie routing changes supported a numerical explanation but did not prove conversion correctness. A genuine FP32 comparison failed on GPU kernel shared-memory limits. The work found a loader bug (FP32 overrides ignored on BF16 checkpoints, marin #10266). Acceptance did not establish full-32k, gradient or fused-training-loss parity.

Training attempts the first attempt at 1e-6 (2026-10-20 20:10 UTC) failed on a 64.35 GiB temporary allocation in the training step. Two fixes followed (token-local expert dispatch; a router auxiliary loss aggregated as in HF, which cut that term from 1.40 to 0.009 at step 0). A smoke configured for 20 updates reached steps two and three at about 355.6 s per update; it did not complete the configured updates. A separate bounded profile measured two host-side training intervals averaging 353.449 s. Nearly all that time was in executable execution/waiting, but CUPTI failed to capture GPU events, so compute, collectives and device waiting could not be separated. The work was stopped on 2026-10-21 at about 00:35 UTC with a written handoff; it was not resumed.

Cost about 16 H100-h of preflight and about 60 H100-h of attempts, smokes and profiles.

7 Costs

GPU cost is reported in H100-hours (H100-h): the number of H100 GPUs a job held, times the hours it held them. All training and eval serving ran on CoreWeave H100 clusters (cw-rno2a and cw-us-east-02a). GLM 5.3 ran on an internal inference service whose GPU time is not measured here; it is reported in API tokens (Table 4). Daytona sandbox time is CPU and is reported in sandbox-hours. Neither is added to H100-h.

7.1 GPU cost by phase

The relay student serves are the parent models served on 8-GPU servers during v7 relay generation, so that they could start each relay episode (Section 2); they are not the relay training runs.

The relay-generation row splits into HIGH 1,357 H100-h, LOW 721, and at least 59 on evicted interactive students, about 2,137 in all. GLM teacher use and sandbox CPU are not included.

How the ledgers count:

- *Allocated* H100-h sum each placed task attempt's start-to-end interval times its GPU count. Queue time and CPU preprocessing are excluded.
- *Retained lineage* is the part of the allocation on the attempts whose results we keep, including their startup, saves, exports and idle time. It is a bookkeeping convention, not a measure of useful work. *Overhead* is the rest: attempts that were evicted, cancelled or failed and not reused.

Table 33: H100-hours by phase. The scopes do not overlap. Look at the type column: the ledgers are measured from scheduler timestamps, the "~" rows are rounded estimates. The total is a documented partial subtotal, not an invoice.

Phase	H100-h	Type
Pilot learning-rate sweep (4 runs, screens), all-in	~560	estimate
v5 arm training (plain, RS-multi, recovery)	~200	estimate (73.6 + 55.8 + about 72)
v5 arm screens (v5 total with training: ~463)	~263	estimate (77 + 82 + 104)
Qwen preflight and Qwen sweep attempts (no rate trained)	~76	estimate (about 16 + 60)
v0.1 reference screens	~30	estimate
LOW plain v0.1 screens (main serve)	~42	estimate
HIGH plain v0.1 and v0 screens (main serves)	~68.8	estimate (32.8 + about 36)
v7 training, nine results, all attempts	746.39	ledger
2026-10-22 screen wave, six arms, with retries	373.54	measured, 15 attempts
Night of 2026-10-22/23: eleven training runs, screens, dose check	1,282.33	ledger and measured
Three runs added on 2026-10-23, training only	184.91	ledger (49.05 of it the failed save run)
Screens of those three runs, with TBLite retries	106.4	measured, 6 serves, queue excluded
Training, evaluation and diagnostics subtotal	~3,930	
Relay student serves for v7 generation: HIGH 1,357, LOW 721, plus at least 59 evicted	~2,137 or more	scheduler durations, to the minute
Documented subtotal	~6,070	not complete

Table 34: v7 training cost, nine results. Look at the overhead column: most of it is the LOW relay v0.1 run, killed by the hub outage and resumed twice (Section 8).

Training result	All attempts	Retained lineage	Overhead
LOW plain v0.1	63.64	53.60	10.03
HIGH plain v0.1	47.31	47.31	0.00
HIGH plain v0	86.72	86.72	0.00
HIGH plain-matched-relay v0	56.73	50.14	6.60
HIGH RS-multi v0	70.08	60.20	9.88
HIGH recovery v0	66.82	63.75	3.08
HIGH relay v0	70.72	70.72	0.00
HIGH plateau v0	72.30	72.30	0.00
LOW relay v0.1	212.06	88.89	123.17
Total	746.39	593.64	152.75

HIGH plain v0 cost 86.72 H100-h against 47.31 for its v0.1 twin at similar per-step speed. Their HF cadence was epoch two plus final, not every epoch. The v0 epoch-two HF export took 32 minutes, overlapping native saving for 9m10s; its exposed tail was about 12.2 H100-h. Later arms omit that intermediate HF export. The tail estimates a possible saving under that run's storage conditions, not a measured constant saving per arm. The duplicate terminal native save remained in place.

Table 35: Night of 2026-10-22/23, all on cw-rno2a. Look at the overhead column: the LOW grid and the checks trained without loss; the contributor overhead is their evictions, and the screen overhead is the startup failures and two retry serves. Those two serves' cached trials may or may not survive in the final records, hence the ranges; they are attribution bounds, not confidence intervals.

Scope	Allocated	Retained lineage	Overhead
LOW grid training (6 runs)	396.42	396.42	0.00
Contributor training (4 runs)	255.66	234.09	21.56
LR check training and dose-check serves	117.16	117.16	0.00
Other screens and serves	513.10	488.59-506.44	6.66-24.51
Total	1,282.33	1,236.26-1,254.10	28.23-46.07

Per training run that night (allocated H100-h): LOW plateau v0.1 49.99, LOW plateau v0 47.38, LOW plain v0 51.95, LR check (LOW plain v0 at 3e-7) 52.23, LOW relay v0 110.14, LOW RS-multi v0.1 69.92, LOW recovery v0.1 67.04; contributor plain v0.1 59.57, plain v0 64.07, RS-multi v0.1 63.09, recovery v0.1 68.93. Serve time that produced no output: 6.66 H100-h (the ShellSim failures 6.42 and the dead-key retry 0.24).

The three runs added on 2026-10-23 (allocated / retained-lineage H100-h): plateau on contributor rows 54.15 / 54.15, LOW-shared 34.89 / 34.89, contributor-shared 95.87 / 46.82. The 49.05 H100-h of overhead is the failed contributor-shared `-r2` run (8). The plateau run's first cache failure and both manifest failures of the pair used no GPU time. Their screens cost 106.4 H100-h in all: LOW-shared 38.2, plateau on contributor rows 35.5 and contributor-shared 32.7, each a main serve plus one TBLite retry serve, with no evictions. With training, the three runs cost 291.3 H100-h all-in.

Rough per-arm figures for planning: a LOW or contributor training run costs about 50-70 H100-h, a LOW relay run about 110, and one four-screen serve group about 25-70 including retries. These are run-level allocations including startup, saves and exports; they should not be converted into a universal per-update throughput estimate.

Not counted in Table 33:

- retries of the earlier LOW and HIGH plain screens;
- the v0.1 TBLite checks at 5,400 s and at 64k context;
- generation serves and retries outside the v7 cost windows, including the student serves of the v5 relay generation;
- unrecorded diagnostics;
- the separate learning-rate test of on-policy distillation (MOPD) from marin #9984, which is not part of this experiment.

Because historical estimates can include queue time while the ledgers exclude it, the subtotal is not a strict lower bound.

7.2 Storage

Each training run writes two kinds of checkpoint to the CoreWeave bucket `marin-us-east-02a`:

- Native checkpoints (Levanter's format, with optimizer state, needed to resume): about 805 GB per ordinary epoch save and 1.61 TB at the terminal step, where the state is written twice (once by the epoch hook, once by the trainer's final save), about 3.22 TB per run. With a single terminal save it would be about 2.4 TB.
- HF export (BF16 safetensors, used for serving and release): 39 shards, 134,157,765,632 tensor bytes (134.2 GB) per export.

On 2026-10-21 at 20:17 UTC, `marin/checkpoints` held 26.9 TB, 22.7 TB of it from this project. Trace pools under `experiments/sft-shootout/` held about 120 GB and the production rows 3.6 GB.

Table 36: Storage deletions, all approved by the owner and verified by an independent listing. Every final HF export was kept. Native checkpoints (batches 1, 2 and 4) are most of the bytes.

Batch	What	When (UTC)	Bytes
1	18 native step directories of six finished runs	10-21 20:25-20:27	19,319,352,585,450
Dry runs	73 dry-run row roots	10-21 20:26	21,989,890,252
2	12 native step directories of the pilot sweep's four runs (a)	10-21 20:28-20:29	12,879,505,399,427
3	checkpoints of the MOPD learning-rate test (not this experiment)	10-22 16:28	3,622,768,461,623
4	18 native checkpoint directories: 17 versions with own verified exports plus one superseded failed version	10-23 02:51-02:53	55,745,452,592,750
4b	5 unused mid-training HF exports	10-23 15:57	670,875,441,795
5	native checkpoints of the three runs added on 10-23, and of the failed contributor-shared run (no export of its own)	10-23 20:32-22:57	10,464,967,266,119
Total			102,724,911,637,416

(a) Under `experiments/task203-sweep/`, outside the `marin/checkpoints` census above.

The shootout's own deletions (all except batch 3) freed 99,102,143,175,793 bytes, about 99.1 TB. Before batch 4, the 9 plateau-dropping loss-history files were copied out of the native directories and hash-checked (9/9). The HIGH plain v0.1 epoch-2 export is kept because it is the dose-check model. Batch 5 deleted the native checkpoints of the three runs added on 2026-10-23 after their export audits, under a standing approval. Before that, the plateau run's three loss-history files were copied beside its export and hash-checked (3/3). The failed contributor-shared run had no export and uncommitted saves; its checkpoints were deleted separately on the same authorization, keeping its metrics. A retention change (one rolling native save, export at the end, delete native after a verified export) is proposed in Section 11.

Six exports were uploaded to the Hugging Face Hub as private repositories (Appendix B); each was verified file by file against its bucket export.

8 Infrastructure events that affected the numbers

This section lists only events that changed a reported number, a run's cost, or which results exist. Other operational events are in the event log of `sft-shootout/data-inventory.md`. Bands, evictions, serves, groups, records and the hub are defined in Section 1.

8.1 Scoring and harness defects

litellm release broke mini-swe-agent (marin #10262) litellm 1.106.0 was released on 2026-10-18 at 05:49 UTC. Harbor `016dd78b` installs mini-swe-agent 2.1.0 fresh in every sandbox, and 2.1.0 requires only litellm 1.75.5 or later. Under 1.106.0 its first model call raises `ModuleNotFoundError: No module named 'mcp'` and sends no request. Every SWE-bench trial of the first three arm screens therefore scored 0 with 0 tokens (for example plain's record `...-swebench-verified-100-b4e0`, 0 of 99). The SWE screens were rerun on Harbor `38742b0f`,

which pins the fix. v0 scored the same 33/100 on both Harbor commits, so the rerun numbers are comparable with the sweep's. Our generation driver was pinned and unaffected.

Record assembly and re-scoring a duplicated `_gen` column broke record assembly with the older reader (fixed on the policy branch, pin moved to `4d45932e39`). The proxy that routes eval requests to the serve returned HTTP 502 during a serve replacement and Evalchemy recorded those as final answers (#10273). The evaluator skipped re-scoring when a scored result existed (#10274).

Looping tasks under the retry cap a task whose verifier fails or times out on every attempt holds the whole 8-GPU serve until Harbor's 10-retry cap. Three cases changed results:

- LOW plain v0.1 BFCL: `bfcl-live-multiple-83-38-0` looped on `VerifierTimeoutError` (named in the record; `mopd2/arms/screens.md`, LOW BFCL note) (about 4.7 H100-h per attempt); the group was cancelled on 2026-10-21 at 04:51 UTC and the task counted as unscored (55/121).
- Plateau v0 SWE: `psf__requests-2317` failed its verifier on all 11 attempts and held the serve for 1h45m before TBLite could start; scored unsolved after the retry.
- LOW recovery v0.1 BFCL: `bfcl-live-irrelevance-698-227-1` looped on `VerifierTimeoutError`; cancelled on 2026-10-23 at 06:11:54 UTC, counted as unscored (58/122).

A retry count alone does not bound cost; a per-task budget that releases the group is needed.

8.2 Timeouts

TBLite trials that hit the 1,800 s agent timeout are scored as failures, as the policy scores them; they are never counted as unscored. Timeouts make up a large share of the GLM-trained arms' TBLite trials: 63-67 of about 95 for the v5 arms, 54-70 of 99 for the HIGH v0 method arms, about 40 for the plain runs on HIGH and LOW rows, and 64 for LOW relay v0.1 (Section 5). Checks at a 5,400 s limit and at a 65,536-token context on v0.1 did not move its score beyond sampling noise (Section 4.6).

8.3 Sandbox start failures

The Daytona sandbox key is shared by our generation lanes, our eval screens and other users' evals. On 2026-10-21 between 09:02 and 09:32 UTC, 10 of the 101 trials that HIGH plain v0.1's first SWE-bench pass recorded for its 100 tasks were unscored (the source does not say why 101) because the sandbox never started (`EnvironmentStartTimeoutError` at 1,800 s). At 10:14 UTC the key had 261 live sandboxes (79 ours) and 443 in an error state, mostly from other runs going back to August. A planned split of the LOW relay lanes was cancelled rather than add about 220 sandboxes. The unscored trials were rerun with the launcher's built-in retry (HIGH plain v0.1 ended at 13/100 SWE, 8/98 TBLite). During LOW relay generation, sandbox start timeouts hit 63 of 6,802 attempts (0.93%).

8.4 Evictions and bands

- 2026-10-18: the first arm gangs (32 H100 each) starved on cw-us-east-02a behind 8-GPU backfill; resubmitted on cw-rno2a, where they were admitted within minutes. The v5 recovery arm was evicted once and resumed from step 528.
- 2026-10-20 about 21:30 UTC: a priority wave on cw-rno2a evicted the interactive-band jobs (the LOW plain v0.1 run at update 178, and the TBLite timeout check's serve). The LOW run was relaunched at the production band; from then on every training launch resumes from its own checkpoint root. Eval serves ran at the production band from then until 2026-10-22.

- 2026-10-21 after 19:08 UTC: three of the five v0 arm gangs were evicted within 25 minutes by another user's production-band serves. All five were cancelled and resubmitted at the production band at 21:21 UTC as new jobs (named with an `-r2` suffix, "second root"). RS-multi and recovery lost their first attempts from step 0; plain matched to the relay budget resumed from its committed epoch-1 checkpoint.
- 2026-10-22: all six screen serves of the v0 arms and LOW relay v0.1 were evicted exactly once, even at the production band and although the screen config requests a non-preemptible serve; why that request did not prevent the evictions is not recorded. From the owner's decision that afternoon, screens ran at the batch band.
- 2026-10-22 18:37-19:15 UTC: with eleven of our gangs on cw-rno2a, three contributor gangs were evicted five times in all (plain v0.1 once, recovery twice, RS-multi twice). Each restarted from update 0. The contributor overhead in the ledger is 21.56 H100-h.
- Screens of 2026-10-22 evening to 2026-10-23 morning: across all 39 serve groups, 8 batch-band serve evictions (the dose check 3 times; five other serves once each), counted as the hub's preemption counter minus the normal end-of-run kill. No training gang was evicted after 19:15 UTC. An earlier report, at about 20:05 UTC, gave 4 evictions for the dose-check serve; the two counts use different methods and the difference is not resolved. The dose serve resumed cleanly each time, losing only in-flight trials.
- From 2026-10-23 about 17:40 UTC: eval screens run at the interactive band (owner decision).

8.5 Hub outage (marin #10433)

On 2026-10-21 from 17:14 to 19:08 UTC the Marin hub lost its heartbeat to cw-rno2a. No job of any user started there for two hours, and jobs whose controlling process (the *coordinator*) could not reach the cluster were failed, with their GPU gangs killed. Ours: the LOW relay v0.1 gang, after its last update but before its HF export, and a TBLite retry serve. The LOW relay v0.1 run's epoch-2 checkpoint had its payload but no commit marker, so the resume went back to step 320 (end of epoch 1). The same-root resume then failed its terminal save against the stale partial arrays (marin #10453), so a second resume into a clean output version (`2026.10.21.4`) finished the run. Its training cost 212.06 H100-h, against 110.14 for the uninterrupted LOW relay v0 run. The outage also delayed the five v0 arms' start.

8.6 Storage key requirement (marin #10475)

Marin #10475, merged on 2026-10-22 at 18:18 UTC, requires the submitter's CoreWeave storage keys in every job's environment. The eval worktree's client did not forward them, so the first four screens of that evening were rejected at handoff and never placed (20:02:50-20:15:40 UTC: LOW plateau v0, LOW plateau v0.1, LOW plain v0 and the 3e-7 check). From 20:22 UTC the screens were relaunched with an eval client that forwards the submitter's keys from the shell environment, never on a command line. Those relaunches ran from `dbb92ac80b` with the backport as an uncommitted change to the Iris client; the identical change was committed as `3d9fb7544a` at about 03:12 UTC on 2026-10-23 (Appendix A). A names-only scan found no key values in the records written after the change. Training runs submitted before the change already carried the keys; none needed a resubmit.

8.7 Storage key rotation (2026-10-23)

The storage key in the launch environment expired at about 01:55-01:59 UTC on 2026-10-23, in a planned organisation-wide rotation. Because of #10475, every new task attempt uses the submitter's stored key, so:

- one screen retry failed at 02:00:14 UTC (`InvalidAccessKeyId`; 0.24 H100-h);
- seven serves still running on the dead key could not write their records and were cancelled at 02:32-02:43 UTC.

Every record written before about 01:58 UTC had landed. What was lost and rerun: BFCL for four screens (LOW plateau v0.1, LOW recovery v0.1, LOW relay v0, contributor recovery v0.1), TBLite for LOW relay v0, and several infrastructure retries. A first replacement was an API token (the wrong key type); an Object Storage key added at 02:39:31 UTC passed a bucket listing at 02:39:50 UTC. Record listings with the new key confirmed which records existed before any relaunch.

8.8 ShellSim catalog 403 (marin #10351)

From 02:56:52 to 03:08:51 UTC on 2026-10-23, every new serve failed at startup (`RemoteInferenceStartupError`): five failed startups, and one cancel at 03:11:09. The serve's tool dashboard fetched a package catalog from an outside service (ShellSim, at `shellsim.rjp.io`) at startup, and that URL now returned HTTP 403. The six groups cost 6.42 H100-h. The fix was to backport #10351, which defers the catalog fetch to tool-execution time; the screens never call that tool route. The eval worktree's head became `caba0fa38b`: the base `dbb92ac80b` plus two backports, `3d9fb7544a` (for marin #10475) and this one. The policy, Harbor and Evalchemy pins did not change. A first group (smoke test) registered its endpoint at 03:18:52 UTC; all 12 relaunch groups had registered endpoints by 03:59:50 UTC; 11 succeeded and 1 was cancelled by decision (the BFCL loop above).

8.9 Training data cache failure (2026-10-23)

The first plateau run on the contributor rows failed at 16:02:32 UTC on 2026-10-23, in the CPU cache stage before any GPU allocation (TensorStore: malformed S3 List response). The probable cause, not confirmed because the raw HTTP body was not logged, is that TensorStore's default credential chain sent unsigned requests when credential files were mounted; marin #10571 (`3eb121f2ee`, merged 16:35 UTC) addresses it. The launcher was rebased onto main `4a0b967c24`, which contains #10571, and the run was resubmitted at 18:00:52 UTC. No GPU time was lost. The failure went unseen for 1h50m, because no automatic training watcher had run since 04:20 UTC, when the watchers were stopped to relieve memory pressure on the operator's workstation.

8.10 Manifest check failure of the matched pair (2026-10-23)

Both runs of the matched pair failed on 2026-10-23 (errors logged at 18:01:29 and 18:02:02 UTC), in the CPU stage before any GPU allocation. The launcher's manifest verification expected an optional field, `test_inputs`, that the manifests of the shared-task row sets do not have, and raised a `KeyError`. The fix (`548bd67c41`, manifest check only) leaves the resolved training config byte-identical. Both runs were resubmitted under it. No GPU time was lost.

8.11 Checkpoint save timeout of the contributor-shared run (2026-10-23)

The contributor-shared run of the matched pair (`-r2`) failed at 20:23:02 UTC on 2026-10-23, during its epoch-2 save after step 213 of 320. The trainer exited with a JAX coordination error: `DEADLINE_EXCEEDED` on the checkpoint key `tensorstore_checkpoint_0` after 30 minutes. Its epoch-1 save at step 106 had been attempted but never committed, and the step-213 save was waiting on it. With no committed checkpoint, the run could not resume. It was rerun from update 0 as `-r3` at the same commit, with the rule that a second hang at a save would stop the work. `-r3` committed all three saves (steps 106, 213 and 320, the last at 21:58:12 UTC) with no failure or preemption. The

LOW-shared run's saves were clean throughout. The cause of the hang is not established. The failed attempt cost 49.05 H100-h, the overhead in 7.

8.12 Other findings with no effect on the reported numbers

- Iris `remote_inference` kept a stale address after a student serve was rescheduled (marin #10237). Affected relay rounds were rerun or excluded before any row build.
- The Iris capability-URL path into cw-us-east-02a limited generation at about 900 concurrent trials (marin #10318); agents were moved to the proxy's local address.
- Artifact provenance records stored the launcher's argv, including secret values passed with `iris -e` (marin #10440). 20 of this project's records were rewritten with the values redacted.
- TPU serving of Snowball diverges from H100 past the 2,048-token sliding window (marin #10250). All screens here were served on H100.

9 Discussion and limitations

9.1 What the comparisons identify

The GLM experiment shows that the tested demonstration-training recipes can reduce agentic performance relative to their parent. The source/configuration comparisons in 5 and 6 differ more in their point scores than the HIGH method variants. These are bundled data comparisons, not isolated teacher effects. The results support further data controls before choosing a more elaborate training method. They do not establish that off-policy SFT generally damages decision-making, or that an on-policy replacement would improve these models.[23]

The lower-learning-rate and earlier-checkpoint checks test particular alternatives to the chosen recipe. Their outcomes constrain those alternatives on these rows. They do not rule out all learning-rate, exposure, optimizer or checkpoint-selection effects. The pilot's tied rates also do not guarantee a unique optimum on a new teacher's data.

9.2 Sampling and missing outcomes

Each agentic screen has 100 fixed tasks and generally one generated trajectory per task. The paired task bootstrap estimates uncertainty from resampling those observed task pairs. It does not measure training-seed variation or repeated evaluation trajectories. Method comparisons share tasks and parents, and many contrasts were inspected. The intervals are descriptive, without a multiple-comparison correction. Missing trials can change both denominators and the paired subset, so the coverage bounds should be read alongside the paired intervals.

Infrastructure retries replace only identified failed trials, preserving scored outcomes. This avoids paying for complete repeated screens, but the resulting rows combine trials from different times and sometimes different serve attempts. Provenance permits auditing that merge; it cannot turn the result into an independent replication.

9.3 Data and method confounds

Teacher identity, collection harness, task pool, clean-completion filtering and trace shape change together between GLM and contributor data. Within GLM, HIGH versus LOW also changes supervised exposure.

[23] `sft-shootout/review-20261022-owner-summary.md`; `mopd2/arms/screens.md`, paired comparisons and follow-up results.

Parent comparisons on byte-identical plain rows hold data fixed, but they do not establish that parent differences are irrelevant for relay or other methods. Relay is student-dependent and has its own passing-pool coverage.

Budget matching counts supervised tokens. It does not match the context tokens needed to predict them, the number of rows, optimizer updates, or number of unique task states. Plain SFT matched to the relay budget addresses the exhausted HIGH family pool but leaves student-conditioned context and episode geometry different. The matched pair holds task identity fixed while intentionally retaining teacher-dependent lengths.

9.4 Competing explanations for trajectory changes

One hypothesis is that teacher demonstrations alter the parent's action and stopping policy in states it would otherwise handle well. Another is that training rows present the wrong distribution of contexts: long teacher trajectories, different reasoning history, and successful states that the student seldom reaches. A third is a loss of task-solving ability that secondarily leads to repeated actions and context exhaustion. The observed trajectory summaries are compatible with more than one mechanism.

The format comparison suggests a further hypothesis. Kimi software-repair demonstrations use Terminus-2, while software-repair evaluation uses mini-swe-agent; GLM software-repair demonstrations use the same command protocol as that evaluation. Same-harness training could overwrite an existing harness-specific policy more directly, whereas a different format could leave it less changed. Teacher, tasks and lengths are also different, so this remains a hypothesis rather than a format ablation.[24]

Collection/evaluation protocol agreement is weaker than equality of model input history. In particular, the recorded Terminus-2 rows retain earlier reasoning inline even though the generation harness does not resend that reasoning in later requests. This is a documented train/inference mismatch and an alternative to a stopping-policy explanation.

9.5 Runtime and generalization limits

TBLite evaluates behavior under a wall-clock limit as well as a token context limit. Genuine timeouts are failures under that protocol, but changing the limit also changes the resource budget. The longer-time and longer-context probes did not produce a large observed aggregate gain; they do not prove that the limits never affect a task or that the compared settings are equivalent. Task-level churn and infrastructure failures remain part of their interpretation.[25]

These screens cannot establish generalization across all coding repositories, terminal environments or tool schemas. Decontamination checks cover specified fields and sources; a zero-hit result does not certify absence of every answer leak. Parent training overlap, including the contributor software-repair tasks represented in v0's earlier relay-graft training, further limits causal interpretation of the contributor bridge. That bridge tests the source on v0 but cannot establish generalization to unseen training tasks. The lineage and overlap evidence are in 1 and 2.4. The data section reports decontamination coverage separately from runtime package integrity.

10 Conclusions

10.1 Answer to the question

The question was which SFT method, if any, improves Snowball on the agentic benchmarks over plain SFT at equal supervised tokens (Section 1). On what was run, the answer has three parts.

[24] `mopd2/ingestion/README.md`, reasoning-span/history decision; `experiments/post_training/sft_shootout_rows/traces.py` and `recovery.py` at `8c1d3fb11b`.

[25] Time-limit check; `mopd2/arms/tblite-timeout-check-20261020.md`.

1. **On GLM 5.3 traces, every tested arm scored below its parent on the agentic screens.** Every model trained on GLM traces solved fewer SWE-bench and TBLite tasks than its parent; for the arms on HIGH rows the SWE-bench gap excludes zero. Against plain SFT on the same rows, only relay on LOW rows from v0.1 differed by an interval that excludes zero: SWE-bench +8 [+1, +16], TBLite +1 [-6, +7]. That arm still scored below its parent (composite 20.0-21.0 against 24.5-25.0). The policy's formal promotion test (rule 4) was not computed, and no arm was promoted.

2. **On contributor rows, plain SFT scored above the parents in point estimate; method differences remain unresolved.** Plain SFT from v0 scored a composite of 30.5 against the parent's 26.5-27.0, and from v0.1 28.5-29.0 against 24.5-25.0; every paired interval against the parents contains zero. RS-multi and recovery differ from contributor plain by SWE-bench 0 and +1 tasks; their paired intervals contain zero. Recovery also has a higher composite point estimate than plain, so these results should not be described as no effect. Plateau dropping on these rows also matched plain (SWE-bench +0 [-7, +7], TBLite +2 [-7, +11]). Relay was not run on these rows. One contributor run did clear its parent: the contributor side of the matched pair, on SWE-bench (+10 [+1, +19]; TBLite -5 [-15, +4]).

3. **The contributor-versus-HIGH-GLM plain comparisons have larger SWE point differences than the v7 method-versus-plain comparisons.** Contributor plain scored above GLM plain from the same parent by SWE-bench +18 [+10, +26] (v0) and +16 [+7, +25] (v0.1). This is a comparison of whole row sets that differ in teacher, harness format, tasks and token budget together (Section 9). The matched pair then held the tasks fixed (1,432 tasks, same parent and recipe): the contributor rows still beat the GLM LOW rows by SWE-bench +26 [+17, +36] and TBLite +9 [+1, +17], with 1.27 times the supervised tokens. So the task set does not explain the gap, but the comparison still does not say which of teacher, format and length matters.

Three changes that reduced exposure to the GLM rows did not rescue the result. A lower learning rate (3e-7 instead of 1e-6) was worse: SWE-bench -7 [-13, -1]. An earlier checkpoint (after 2 of 3 epochs) was not resolvably better: SWE-bench +5 [-4, +13], composite 13.5-14.5 against the parent's 24.5-25.0. The GLM side of the matched pair, with 39% of the LOW plain budget, scored below the full LOW plain arm in point estimate (SWE-bench -7 [-14, +0]) and below its parent (-16 [-24, -8]). These checks do not rule out other learning rates, training doses or their interactions with the data, and none measures how the damage scales with exposure.

10.2 What was measured about the failure

Measured on SWE-bench: the v7 GLM-trained students have per-step lengths similar to the rows (about 120 tokens per step) but take about twice the rows' steps and run into the 32k-token context limit in most SWE-bench episodes. Students trained on HIGH rows end with a submit in 1-6% of episodes, those trained on LOW rows in 19-31%, against 56% (v0) and 36% (v0.1) for the parents (Section 5.5). They gain on GSM8K and BFCL-Parity.

Not established: why. Section 9 lists the competing explanations. One of them, that SFT in an evaluation's own harness format overwrites the parent's behavior in that format, is consistent with the action protocols of the rows (the GLM rows use each evaluation's own protocol, the contributor rows the other one; Section 2.7) but has not been tested.

10.3 The design doc's expectations

- *RS-multi should beat plain.* No tested round establishes that gain: v5 composite 3.5-8.5 against plain's 11.5-14.5; on HIGH v0 rows and on LOW v0.1 rows its paired SWE-bench difference from

plain contained zero (LOW: 0 [-7, +7]); on the contributor rows 0 on SWE-bench and -4 on TBLite.

- *Each method's gain should resemble its proposers' reported gain.* This cannot be checked against numbers, because the reported gains are not recorded in our notes. Qualitatively, only relay on LOW rows showed a resolvable gain over plain; recovery, RS-multi and plateau dropping did not on any row set where they ran.

- *Relay's gain could read smaller on v0, which already carries relay training.* On LOW rows, relay beat plain by SWE-bench +8 [+1, +16] from v0.1 and +6 [-2, +14] from v0. The direction matches the expectation, but the difference between those gains was not tested. Comparing whether each individual interval excludes zero does not establish a parent-by-method interaction.

10.4 What is open

Which property of the contributor rows helps is the open question. The matched pair showed that the task set does not explain the gap; the teacher and the harness format remain bundled (Section 11). The proposed follow-ons include a conversion intended to vary the GLM action format while preserving episodes. Whether actions, observations and reasoning histories can be mapped faithfully remains to be audited; format-only isolation is a design aim, not an established fact.

11 Follow-ups added on 2026-10-23, and proposals

11.1 Matched pair

The matched pair restricts existing LOW GLM and contributor plain rows to exactly the same 1,432 selected training tasks: 466 software-repair tasks and 966 TMax tasks. Each side retains its whole selected episode for each task. There is no new generation, relay graft or row-level subsampling. Both start from v0.1. Their smaller budgets are reported explicitly rather than equated to the full contributor arm.[26]

Table 37: Matched pair inputs. Agentic tokens exclude replay; row counts include replay. The task set is identical, but contributor traces carry more tokens and updates.

Input	SWE tokens	TMax tokens	Agentic tokens	Replay tokens	Rows	Padded rows	Steps/epoch	Updates
LOW shared	887,535	3,626,719	4,514,254	1,132,112	1,628	1,632	102	306
Contributor shared	1,949,625	3,781,833	5,731,458	1,440,055	1,699	1,712	107	321

The contributor side has about 27% more agentic supervised tokens, but about 4.9% more updates. Its software-repair token total is 2.20 times the LOW side's, and its TMax total is 1.04 times as large. Replay is cut independently under the same budget rule and therefore also differs. The comparison holds task identities fixed; remaining differences include teacher, format, lengths and exposure. Both use the same final-checkpoint recipe and have no held-out loss measurement, for the reasons in 3.

Padding rows are the zero-loss rows added to complete a global batch. The table's *padded rows* column counts all rows after adding that padding, as defined in 2.

[26] Row root: `s3://marin-us-east-02a/marin/experiments/sft-shootout/rows/shared-v01-20261023/` with children `lowshared` and `contribshared`. Manifest SHA-256 values are `ba03f0f0b3df50929b770f01219502bb9a7b0d699d05105c12398e5a7519ed8c` and `db79d475595e4cac22513660b0f441e731e9633729ec5ac9c`. The shared-task-list SHA-256 is `992958dd0126ca70d38b160b370470f04f700a76c97ad51a8804e5f0fff36469`. Builder `f1ae2762e6` preserves source row tokens and masks and records the seeded epoch order in each manifest. Source and sensitivity figures are in `mopd2/arms/shared-task-budget-20261023.json`.

The contributor plateau run used the rebased launcher `264434a6ce` after its cache failure. The matched pair's final runs used `548bd67c41`: that rebase plus the manifest-only fix described next. The configuration-equality check and the credential fix are in 8.

The initial matched pair failed before GPU allocation on an optional manifest field; the resubmissions use manifest-only correction `548bd67c41` without changing rows or resolved training settings, as detailed in 8.

The corrected roots were accepted at 18:20:18 and 18:20:51 UTC (`-r2` job-name suffixes). The LOW-shared run finished at 19:54 UTC, with committed saves at steps 101, 203 and 305. The contributor-shared `-r2` run failed at 20:23 UTC at its epoch-2 save, and its epoch-1 save had never committed, so it could not resume. It was rerun from update 0 as `-r3` at the same commit and finished at 21:58 UTC with all three saves committed (the failure is in 8). Both exports passed the structural audit: 39 shards, 502 BF16 tensors, and shapes, tokenizer, template and router biases checked against v0.1. That audit is not a full weight checksum or an inference test.

Cost of the matched pair and the plateau run together: 184.91 H100-h of training (49.05 of it the failed contributor-shared `-r2` run) and 106.4 H100-h of screens, 291.3 H100-h all-in (7).

Table 38: Screens of the three runs added on 2026-10-23, with the references they are compared to. Read the two matched-pair rows against each other: same 1,432 tasks, same parent, same recipe. The composite is our screening mean, (SWE + TBLite)/2 out of 100; a range counts unscored trials first as failed, then as passed.

Model	GSM8K	SWE solved (submits)	TBLite solved (timeouts)	BFCL-Parity	Composite
plain-lowshared v0.1	92.8 (1,223 / 1,318)	10 (12)	9 / 98 (39)	62 / 123	9.50-10.50
plain-contribshared v0.1	92.9 (1,225 / 1,319)	36 (46)	18 / 100 (25)	80 / 123	27.00
plateau-contributor v0.1	93.9 (1,238 / 1,319)	29 (41)	30 / 100 (25)	74 / 123	29.50
parent v0.1	91.1	26 (36)	23 / 99 (49)	32 / 123	24.50-25.00
LOW plain v0.1	91.7	17 (27)	12 / 93 (42)	55 / 121	14.50-18.00
contributor plain v0.1	94.0	29 (32)	28 / 99 (26)	71 / 123	28.50-29.00

LOW-shared has one GSM8K infrastructure error and 2 TBLite trials still unscored after retry.

Table 39: Paired per-task differences (A minus B) for the added runs, with 95% task-bootstrap intervals. Bold: interval excludes 0. Read the first row: it is the matched pair.

A	B	SWE-bench	TBLite
plain-contribshared v0.1	plain-lowshared v0.1	**+26 [+17, +36]**	**+9 [+1, +17]** (sensitivity +7 to +9)
plain-contribshared v0.1	parent v0.1	**+10 [+1, +19]**	-5 [-15, +4]
plain-contribshared v0.1	contributor plain v0.1	+7 [-1, +15]	-10 [-20, +0]
plain-lowshared v0.1	parent v0.1	**-16 [-24, -8]**	**-14 [-22, -6]**
plain-lowshared v0.1	LOW plain v0.1	-7 [-14, +0]	-4 [-11, +2] (sensitivity -10 to -1)
plateau-contributor v0.1	contributor plain v0.1	+0 [-7, +7]	+2 [-7, +11]
plateau-contributor v0.1	parent v0.1	+3 [-7, +13]	+6 [-3, +15]

11.1.1 What the matched pair measured

On the same 1,432 tasks, from the same parent with the same recipe, plain SFT on the contributor rows beat plain SFT on the GLM LOW rows by 26 SWE-bench tasks [+17, +36] and 9 TBLite tasks [+1, +17]. The GLM side scored below the parent on both screens (SWE -16 [-24, -8], TBLite -14 [-22, -6]). The contributor side scored above the parent on SWE-bench (+10 [+1, +19]), the first contributor-versus-parent difference whose interval excludes zero, and within noise on TBLite (-5 [-15, +4]). So the drop from GLM rows and the gain from contributor rows both occur on the same tasks; the task set does not explain the gap. Three limits stay with this result. The pair is matched on tasks, not tokens:

the contributor side has 1.27 times the supervised tokens and about 5% more updates. The contributor subset loses TBLite against the full contributor rows (-10 [-20, +0]), so the small-budget pair is read for its gap, not for absolute levels. And each arm is a single run screened on 100 tasks.

The GLM side used 39% of the LOW plain budget (4,514,254 of 11,596,239 supervised tokens per epoch) and did not land closer to the parent. Its point estimate is below the full LOW plain arm (SWE -7 [-14, +0], an interval that touches zero). 85 of its 100 SWE-bench episodes ran into the context limit, and it submitted in 12. With the learning-rate check (3e-7 worse than 1e-6) and the dose check (the epoch-2 checkpoint not resolvably better), this is the third result in which less exposure to the GLM rows did not bring the student closer to its parent. None of the three measures how the damage scales with exposure.

Submitting again follows the rows: 46 of 100 SWE-bench episodes end with a submit on the contributor side, 12 on the GLM side, against 36 for parent v0.1.

11.1.2 Hypothesis, narrower but not tested

With the task set no longer an explanation, two bundled differences remain between the row sets. The first is the teacher (GLM 5.3 against Kimi-2.5 and Qwen3.6-27B), including how long its episodes run and how it acts. The second is the harness the traces were recorded in: the GLM rows use each evaluation's own action protocol, and the contributor rows use the other one (2.7). The pattern in which rows in the evaluation's own format overwrite the parent's evaluation-time policy (episodes twice as long, submitting stops) fits every result so far. It has not been tested; the conversion in 11.6 would separate format from teacher for the software-repair half.

11.2 Plateau on contributor data

The contributor plateau arm uses the same full contributor plain v0.1 rows as its plain SFT baseline: 226 steps per epoch and 678 updates. It tests whether dropping stable token losses helps on the contributor source, where the initial GLM method comparisons need not transfer. During training, its plateau-state files sat where the resume loader expects them. After the export was verified, they were copied to `plateau-state/` beside the export and hash-checked, and the native root was deleted, so a resume from this run would have to point at the copy. The initial cache-stage failure and resubmission are infrastructure events, not plateau outcomes.

The run saved its final step (677) at 20:20 UTC and its export passed the same structural audit at 22:55 UTC. Before its native checkpoints were deleted, its three plateau-state files were copied beside the export and their hashes checked. It scored the same as contributor plain SFT: SWE +0 [-7, +7], TBLite +2 [-7, +11], composite 29.50 against 28.50-29.00. On the contributor rows, none of the three methods tried besides plain SFT (RS-multi-TMax, recovery, plateau dropping) differs from it by an interval that excludes zero.

11.3 Controls not yet run

A teacher-only control matched to relay's supported tasks and exposure would help separate the student-conditioned prefix from passing-pool selection. It requires either additional teacher passes on common tasks or a restricted relay subset. No result from that proposed control is available. A direct harness-format intervention on fixed teacher episodes would address a different question from the current matched pair.

11.4 Cross-format conversion proposal

A format-only conversion of the GLM LOW rows is proposed; it was briefly withdrawn on 2026-10-23 on a mistaken reading of the formats and reinstated the same evening.[27]

The proposed intervention converts existing GLM LOW software-repair episodes from mini-swe-agent bash calls to Terminus-2 JSON, and terminal episodes in the opposite direction. Its aim is to hold the teacher, task identities and recorded episode content fixed while changing the action format. Whether that isolation is feasible remains unverified: commands, observations, reasoning history and loss masks may not map one-to-one. The faithful-conversion audit must identify any unavoidable changes before this can be described as varying format alone (Section 11.6). Matching an action protocol also does not establish identical evaluation input history.

The proposed plain-SFT arm starts from v0.1. Its provisional training-and-screen budget is about 125 H100-h (range 100-150); this is a planning estimate, derived in Section 11.6. The owner has not decided, and no converted rows have been built or training launched. The current matched pair cannot replace this intervention because teacher, format and trace lengths still vary together in that pair.

With the matched-pair result in, the decision before the owner has three options:

- **A. Cross-format conversion** of the existing GLM LOW rows, as above: one plain SFT arm from v0.1, about 125 H100-h (100-150) plus an estimated 8-16 engineering hours. Plain shell calls map across; persistent terminal state, command batches and control keys do not (Section 11.6). Unsupported terminal episodes would be dropped and counted, not invented, so the software-repair family is the cleaner half of the test.
- **B. Relay generation on the contributor task pools**, about 900-1,100 H100-h. It would not test anything the matched pair did not.
- **C. Stop and write up.**

The coordinator recommended A. The owner chose C: the experiment stopped here and this paper is the write-up.

11.5 Checkpoint retention

The checkpoint-retention implementation task proposes one rolling committed native checkpoint, final-only HF export, an independent verified-release stage, and native cleanup only after that verification. Plateau history must remain discoverable for resume and be preserved in the released record. An explicit keep override is needed for continuations. The implementation must not delete the last resumable state before a replacement commits, nor treat a partial checkpoint directory as committed. This policy was approved for implementation but is not part of the pins used for the reported experiments.

11.6 Feasibility of a crossed-format control

The proposed control changes the action protocol of the existing GLM LOW episodes, not their teacher or task set. This is feasible for a restricted class of shell actions, but faithful conversion of every episode is not yet established. A converter should report unsupported actions rather than silently omit them or invent terminal feedback. No converted rows have been built and no run is authorized by this note.[28]

[27] Team decision history from 2026-10-23 and `paper/harness-formats.html`. The withdrawn premise was that contributor rows matched the evaluation action protocols while GLM rows did not; the harness comparison showed the reverse. Exact decision timestamps were not independently verified.

[28] Builder `experiments/post_training/sft_shootout_rows/traces.py` at `8c1d3fb11b`; format examples and mask boundaries in `paper/harness-formats.html`. This is a read-only feasibility assessment, not a validated converter specification.

Table 40: Required mappings. Terminal interaction and observation semantics, not JSON syntax alone, determine whether an episode can be preserved.

Family	Source and target	Mapping needed	Not automatically equivalent
SWE	mini-swe-agent bash calls to Terminus-2 JSON	Put the original shell command in a commands entry; escape it as JSON; map the explicit submit sentinel to task_complete; supply target-format instructions	Stateless bash execution versus a persistent terminal; execution timeout versus wait duration; shell output versus a terminal-screen observation
Terminal	Terminus-2 JSON to mini-swe-agent bash calls	Parse commands and completion; render shell actions as native bash tool calls; attach correctly identified tool responses	Multiple commands per reply; empty polling commands; control keys; interactive programs; persistent shell state; one aggregate screen for several actions

11.6.1 What a faithful builder must preserve

The starting point should be the selected canonical messages identified by the frozen LOW row index, not a regular-expression rewrite of tokenized rows. The converter must verify the source trial identity and preserve the task instruction, recorded reasoning, command strings, action order and observations. Target-format system instructions and tool schemas must replace the source protocol instructions, or training would contain contradictory instructions. This changes contextual text even if the task is unchanged.

For SWE, one native bash command can be represented as one Terminus-2 command string. An appended newline must implement submission to the terminal, not alter a quoted shell string. The source timeout cannot simply become a Terminus wait duration: one limits execution, whereas the other can return before a command finishes. A recorded exit code and stdout/stderr are not a terminal screen. Wrapping those values in a target observation makes a synthetic transcript; it does not reproduce what the target harness would have observed during execution.

For terminal tasks, combining multiple commands into one shell call can change failure handling, working directory, shell lifetime and observation timing. Splitting them into separate calls needs intermediate observations that may not exist. Empty polls, control keys and interactions with an already running process have no general equivalent in a single stateless bash call. Completion without a shell action would require a synthetic submit call. These cases need explicit accounting before a same-episode claim is made.

Reasoning should remain in the parent's thinking delimiters. Existing Terminus analysis and plan fields should not be deleted or reclassified as reasoning without documenting the change. Conversely, a native reply supplies no independent Terminus analysis/plan fields: filling them with generated prose introduces new content. An empty-field policy would need a parser check and would still change the target style.

The renderer should regenerate the loss mask from message roles and authors after conversion. Teacher replies remain supervised; instructions and tool observations remain context only. Masking newly invented wrappers avoids teaching synthetic text but also changes the objective. Supervising them teaches the new protocol but changes supervised token counts. Either policy must be declared. Reusing old token positions after re-tokenization is invalid. Replay should retain its exact original rows and order.

11.6.2 Effort and cost

The estimated builder effort is 8–16 engineering hours: inspect every selected action class, implement deterministic mappings, test the two target parsers, re-render and validate masks, and compare task/command/observation inventories. This is not measured implementation time. It excludes solving unsupported interactive histories; if those require execution or new generation, the proposed no-generation control is not faithful.

The recorded LOW plain v0.1 successful training allocation was 53.60 H100-h, and its main screen serve was approximately 42 H100-h before retries. Their roughly 96 H100-h sum is a reference, not a forecast. About 125 H100-h for training plus screens remains a planning allowance; a provisional range is 100–150 H100-h, not a cap. Conversion can change tokenization, context-limit eligibility and row counts, so a frozen-manifest plan readback is required before refining that estimate. Extra retries are unbounded by this estimate. No GPU test run is needed to decide basic mapping feasibility.

11.6.3 What the matched pair can answer

The matched pair fixes task identities but changes teacher, action protocol, episode content and token exposure together. It can test whether task-pool differences suffice to explain the source gap; it cannot identify format as the cause. A crossed-format control would be more discriminating only if the mapping audit establishes that action content and observations are preserved. Otherwise it is another bundled intervention, and must be reported as such. Excluding unsupported episodes would also require an unchanged-format control restricted to exactly those episodes.

A Pins and hashes

Commit ids are in `marin-community/marin` unless noted. "…" marks a hash shortened in the source.

A.1 Models

Table 41: Parent models: the HF revision used for every launch and the bucket copy that was read.

Parent	HF repository and revision	Durable bucket copy
v0	open-athena/Grug-67B-A2B-Merge-Campaign3-DARE-Linear-02-2026.10.12 @ 4aeb756b404db25bc04e4daf49dfb9817bceb741	s3://marin-us-east-02a/marin/models/open-athena--Grug-67B-A2B-Merge-Campaign3-DARE-Linear-02-2026-10-12--4aeb756b404db25bc04e4daf49dfb9817bceb741/
v0.1	open-athena/Snowball-Smartie-RL-2026.10.15.32-step14 @ 6783e9743478bd25a9ce03ab3460e84d6a9814b0	s3://marin-us-east-02a/marin/models/open-athena--Snowball-Smartie-RL-2026-10-15-32-step14--6783e9743478bd25a9ce03ab3460e84d6a9814b0/

Teacher: GLM 5.3 on the internal inference service (no public revision). Shared tokenizer and chat template: `chat_template.jinja` sha256 prefix `b3f20bc21498407f`.

A.2 Tasks and inputs

Table 42: Task pools, screen task lists and the replay slice. Every builder and screen checks these.

Item	Pin
Pool v5 manifest	`7f3d6f7dbef1ed80d302f251540c72a5c548edb871fd0afed729653d7b6def68`
Pool v5 SWE-smith packages	`swesmith-packages.parquet@sha256:f8572f803f3427413bedd71db012b8e2d7a497f0a5f783f96899ba75e73be2cb`
TMax packages	`allenai/TMax-SFT-16.5K@848e2182…` + `hamishivi/tmax@6d3d606c`
Pool HF copy	`open-athena/Snowball-SFT-Shootout-Pool-2026.10` @ `d30b3063`
v5 GLM traces on HF (partial)	`open-athena/GLM-5.3-tmax-swesmith-k8-traces-2026.10` @ `7a94d72a`
Contributor pool	`pool-t202-contrib/` manifest `ecebf85a…`; exclusions `06a99e03…`
Pilot sweep rows	`task202-sweep-prepared-20261016d`, manifest `2fbe5fe1…`
Replay slice (continuation v2)	manifest `32e6e20b…`
ENOSPC trial exclusions (v7)	`exclusions-enospc-20261020.jsonl`, `8fa67c25…`
Screen task ids	`screen-task-ids.json`, `baf99a92ded65a32175e69b6aa0e3e4147f804eff93cc57c5b6cad004598ea52`

A.3 Row manifests

Row roots are under `s3://marin-us-east-02a/marin/experiments/sft-shootout/rows/`.

The v5 rows (`plain-v5-20261018/`, `rs-multi-v5-20261018/`, `recovery-v5-20261018/`) were built at `6f74d672` (recovery at `0673b18b`); their manifest hashes are not in our notes (unknown).

A.4 Code

Draft pull requests under marin #10213:

- pool and decontamination: #10258;
- sweep data: #10252;
- row builder: #10253;

Table 43: Row manifest sha256 per arm. HIGH roots are `v7-v01-20261021/` (v0.1) and `v7-v0-20261021/` (v0).

Row set	Manifest sha256
HIGH validation, v0.1	`422fa71fdf96aefde86b0c15f24df1e392bd7191761952b8d0169431731edff3`
HIGH plain, v0.1	`dbb6d80d5412c3f30e5daf8b6819c6754264429d65d5681923365602bcf0c9cb`
HIGH RS-multi, v0.1	`2ac2c7febc286f8a8ede366f9aed3277c3f2e14a97b134b51a9b57f1fab91414`
HIGH relay, v0.1	`86addcdacd0829f298413fa7581aef0d9b3b8f63713e051b9ae433b9d30b49ec`
HIGH recovery, v0.1	`89fba21d0bfaf53bb91ef0ecc867c4157143df93e3e12a490bbd9d96e55c4542`
HIGH plain-matched-relay, v0.1	`16fd0765f9b90ed4db22fc49863651758f4592ff21d9db0ab1ca6c2e60a99487`
HIGH validation, v0	`acea62933dcab1d81a327aba698b478aa80e560256ee17c8cca0e551b4687b52`
HIGH plain, v0	`a1516b0adede45e2897d69e90448ed633d11104466c55287b641092db6011547`
HIGH RS-multi, v0	`5f4b35f7022a8d772602e7e62fc32078720e2b1644c2b61d906dfe57dfab81be`
HIGH relay, v0	`6c11f63a5c42d29b2732a9e927005c1579634650eb1a170693563be0b200975f`
HIGH recovery, v0	`5311601eb0270941eb57a106f217edfa5ca2c9dd9b20550bb70ce27d32a3becf`
HIGH plain-matched-relay, v0	`fa456a5695393da7e8335e39660aa2624363caa3bbf4a0f619c04cdd3648eab5`
LOW plain, v0.1	`33705e55...`
LOW plain, v0	`2598c43c...`
LOW relay, v0.1	`0de958b10a567b4e67d4bcfba34f7339731c2044d94b55c36720779407a4e70a`
LOW relay, v0	`ac1309f5eec5b2d09a49a79702e40221aba03aa6525b3b6f685b158b37d3dd52`
LOW RS-multi, v0.1	`2f9e06df...`
LOW recovery, v0.1	`23fe246c...`
Contributor plain, v0.1	`507e73f0d27068e85eddc4e1146e1df2d317cee0c07f4beec0696cb3ec7d075d`
Contributor plain, v0	`8773d29e58b7b99e864b18ae9938f87f06621de9512c297646029973f9fa5fc6`
Contributor RS-multi, v0.1	`f944d33f284e6a29fad2e0383ff22611a7d2a03a780eda3cf623a8c2b4054961`
Contributor recovery, v0.1	`2a097ce8bcb9efe452fb8cb09163f4d61aadad845ffd750c4b50c3bb6f9d4eb1`
Contributor validation, v0.1	`da051da0cc05dad93ed7ee3e04b540e95170ad07c8c5b65c4eea806b93d36679`
Contributor validation, v0	`af6115af2ed2bfb0f3e26c4b4b26cf70c69f03028cd62016474a3a555da9ea8f`
Shared tasks, LOW rows	`ba03f0f0b3df50929b770f01219502bb9a7b0d699d05105c12398e5a7519ed8c`
Shared tasks, contributor rows	`db79d475595e4cac22513660b0f441e731e9633729ec5ac9cf5abf0ddb6e4573`

Table 44: Code pins. Branch names are in `marin-community/marin`. The eval pins did not change across any screen. The trainer head changed between rounds; the recipe settings did not (Section 3).

Component	Pin
Pool build	`task/206` (`ba8d65085e` v1 ... `566547be5a` v5)
Generation driver, relay address fix	`task/212` @ `174906474a`
Generation, v7 lanes	`task/effort-pilot` @ `bc858393`; Harbor `016dd78b`
Row builder, v7 production	`d95d9b40` (rows), `77319c29` (verifier with widened RS-multi tolerance)
Row builder, contributor	`task/t202-arms` @ `8c1d3fb11b` (recovery commit `d57cecdfec`)
Row builder, matched pair	`f1ae2762e6`
Arm trainer, HIGH plain and LOW relay v0.1	`5ffddba708` / `11d41241ec` (production band)
Arm trainer, five HIGH v0 arms	`519568ab74`
Arm trainer, LOW grid and LR check	`01d104874e`
Arm trainer, contributor runs	`task/235-v7` @ `d23b8c9a94`
Arm trainer, matched pair and plateau on contributor rows	`264434a6ce` (on main `4a0b967c24`)
Eval policy	`4d45932e39` (with NCCL WARN override, non-preemptible serve)
Harbor (eval)	`38742b0f` (mini-swe-agent with the litellm fix)
Evalchemy (GSM8K)	`c1d5c62c`
Eval worktree, v5 screens	`task/238` @ `9af01aa66e`
Eval worktree, v0.1 reference and v7	`dbb92ac80b`, then `3d9fb7544a` (#10475), then `caba0fa38b` (#10351)

- generation driver: #10251;
- plateau dropping: #10255;
- sweep trainer: #10324;
- screen configs and launcher: #10325;
- arm trainer: #10327.

B Every screened model

Checkpoint paths are under `s3://marin-us-east-02a/marin/checkpoints/karan/` unless shown in full. Each is the final HF export of its run unless marked. "Group" is the screen group id on the Iris hub; retries ran as separate groups over the same records. Six exports are on the Hugging Face Hub as private repositories under `open-athena/`; the others exist only in the bucket.

B.1 Hugging Face repositories

Table 45: Exports uploaded to the Hub (private, license OpenMDW 1.1), verified file by file.

Model	Repository (=open-athena/=...)	Revision
HIGH plain v0.1	`Snowball-67B-A2B-SFT-Shootout-HIGH-Plain-v0.1-2026.10.21`	`c21e50f9`
HIGH plain v0	`Snowball-67B-A2B-SFT-Shootout-HIGH-Plain-v0-2026.10.21`	`a2e453ea`
LOW plain v0.1	`Snowball-67B-A2B-SFT-Shootout-LOW-Plain-v0.1-2026.10.20`	`052411a6`
v5 plain	`Snowball-67B-A2B-SFT-Shootout-v5-Plain-2026.10.18`	`fa9f10ff`
v5 RS-multi	`Snowball-67B-A2B-SFT-Shootout-v5-RS-Multi-2026.10.18`	`59e8c0dc`
v5 recovery	`Snowball-67B-A2B-SFT-Shootout-v5-Recovery-2026.10.19`	`cf1ee476`

B.2 Screened checkpoints

Table 46: Every model screened in this experiment, with its checkpoint and main screen group. Use the group id to find the records; results are in Sections 5 and 6.

Model	Checkpoint	Main screen group
Parent v0	HF `4eeb756b`	sweep reference `20261018-000259-task208-v0-reference-2d25` and others; SWE rerun `20261019-031658-task208-v0-reference-2fd0`
Parent v0.1	HF `6783e974` (durable copy)	`20261019-213306-v01-reference-smartie-step14-e4d0`
Pilot sweep, 3e-7, 1e-6, 3e-6, 1e-5 (epochs 2 and 3)	`experiments/task203-sweep/checkpoints/karan/v0-agentic-pilot-lr<rate>-hf/step-<N>` (under `s3://marin-us-east-02a/marin/`)	`20261018-184032-task204-lr1e-06-step1643-df6c`; `20261019-013320-task204-lr1e-06-step1643-df6c`; full list in `mopd2/sweep/screens-status-20261018.md`
v5 plain	`arm-plain-lr1e-06-3ep/2026.10.18.1/hf/step-791`	`20261018-230736-task238-arm-plain-lr1e-06-step791-eda0`; SWE rerun `20261019-055330-...-f7c6`
v5 RS-multi	`arm-rs-multi-lr1e-06-3ep/2026.10.18.1/hf/step-791`	`20261018-230802-task238-arm-rs-multi-lr1e-06-step791-298c`; SWE rerun `20261019-080620-...-f688`
v5 recovery	`arm-recovery-lr1e-06-3ep/2026.10.18.1/hf/step-791`	`20261019-025403-task238-arm-recovery-lr1e-06-step791-04d5`; rerun `20261019-144650-...-f739`
LOW plain v0.1	`plain-low-v0.1-v7-20261020-lr1e-06-3ep/2026.10.20/hf/step-674`	`20261020-233725-v7-plain-low-v01-20261020-1289`
HIGH plain v0.1	`plain-v0.1-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-704`	`20261021-084900-v7-plain-high-v01-20261021-eb6f`
Dose check: HIGH plain v0.1, epoch 2	`plain-v0.1-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-469`	`20261022-165719-v7-plain-high-v01-ep2-dose-20261021-5aec`
HIGH plain v0	`plain-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-704`	`20261021-120421-v7-plain-high-v0-20261021-d8e2`
HIGH plain-matched-relay v0	`plain-matched-relay-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-617`	`20261022-020525-v7-comparator-v0-20261021-6e5c`
HIGH RS-multi v0	`rs-multi-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-704`	`20261022-020750-v7-rs-multi-v0-20261021-448d`
HIGH recovery v0	`recovery-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-704`	`20261022-021014-v7-recovery-v0-20261021-ccf2`
HIGH relay v0	`relay-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-923`	`20261022-021238-v7-relay-v0-20261021-1082`
HIGH plateau v0	`plateau-v0-v7-20261021-lr1e-06-3ep/2026.10.21/hf/step-704`	`20261022-021501-v7-plateau-v0-20261021-3444`
LOW relay v0.1	`relay-low-v0.1-v7-20261021-lr1e-06-3ep/2026.10.21.4/hf/step-962`	`20261022-020259-v7-relay-low-v01-20261021-dbd1`
LOW plateau v0.1	`plateau-low-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-674`	`20261022-202855-v7-plateau-low-v01-20261022-e8c5`
LOW plateau v0	`plateau-low-v0-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-674`	`20261022-202212-v7-plateau-low-v0-20261022-e710`
LOW plain v0	`plain-low-v0-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-674`	`20261022-203149-v7-plain-low-v0-20261022-c723`
LOW relay v0	`relay-low-v0-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-932`	`20261022-220418-v7-relay-low-v0-20261022-f081`
LOW RS-multi v0.1	`rs-multi-low-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-674`	`20261022-204035-v7-rs-multi-low-v01-20261022-a2e3`
LOW recovery v0.1	`recovery-low-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-674`	`20261022-203743-v7-recovery-low-v01-20261022-35d6`
LR check: LOW plain v0 at 3e-7	`plain-low-v0-v7-20261022-lr3e-07-3ep/2026.10.22/hf/step-674`	`20261022-203450-v7-plain-low-v0-lr3e07-20261022-0897`
Contributor plain v0.1	`plain-contributor-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-617`	`20261022-204400-v7-contrib-plain-v01-20261022-56de`
Contributor plain v0	`plain-contributor-v0-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-617`	`20261022-220645-v7-contrib-plain-v0-20261022-03ea`
Contributor RS-multi v0.1 (TMax only)	`rs-multi-contributor-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-617`	`20261022-221052-v7-contrib-rsmulti-tmax-v01-20261022-5f79`
Contributor recovery v0.1	`recovery-contributor-v0.1-v7-20261022-lr1e-06-3ep/2026.10.22/hf/step-617`	`20261022-221546-v7-contrib-recovery-v01-20261022-9f87`

Continued on next page

Continued from previous page

Model	Checkpoint	Main screen group
Plain on LOW rows, shared tasks (matched pair), v0.1	`plain-lowshared-v0.1-v7-20261023-lr1e-06-3ep/2026.10.23/hf/step-3206`	`20261023-203224-v7-plain-lowshared-v01-20261023-1ec7`
Plain on contributor rows, shared tasks (matched pair), v0.1 (the `-r3` run)	`plain-contribshared-v0.1-v7-20261023-lr1e-06-3ep/2026.10.23.3/hf/step-3206`	`20261023-225922-v7-plain-contribshared-v01-20261023-be54`
Plateau on contributor rows, v0.1	`plateau-contributor-v0.1-v7-20261023-lr1e-06-3ep/2026.10.23/hf/step-1673`	`20261023-225752-v7-plateau-contrib-v01-20261023-ec60`

Retry and relaunch groups for the night of 2026-10-22/23 are listed per run in `sft-shootout/data-inventory.md`. The v0.1 TBLite checks at 5,400 s ran as `20261020-164700-v01-tblite-5400-28a5`; the 64k-context check's group id is not in our notes (unknown). The GLM 5.3 teacher row on the two agentic sets (not a screen of a trained model) is under `experiments/sft-shootout/traces-eval-glm-high/` (Harbor `38742b0f`).

B.3 Training jobs of the runs added on 2026-10-23

The three runs added on 2026-10-23 trained as these jobs:

- `/karan/arm-train-plateau-contributor-v0p1-v7-20261023-cw-rno2a-r2` (678 updates);
- `/karan/arm-train-plain-lowshared-v0p1-v7-20261023-cw-rno2a-r2` (306 updates);
- `/karan/arm-train-plain-contribshared-v0p1-v7-20261023-cw-rno2a-r3` (321 updates).

The pair ran at head `548bd67c41`; each first root failed before any GPU was allocated, and the contributor-shared `-r2` run failed at its epoch-2 save (8). None of the three exports is on the Hugging Face Hub.

C Glossary

Terms are grouped by topic. Names we coined for this experiment are marked (coined).

C.1 Models and data

Parent the model a training run starts from: Snowball v0 or v0.1 (Section 1).

Student a model trained in this experiment, or the parent when it acts in relay generation.

Teacher the model whose traces are the training targets: GLM 5.3, or for the contributor rows Kimi-2.5 (SWE-smith) and Qwen3.6-27B (TMax).

Task family SWE-smith (repository bug fixes, under mini-swe-agent) or TMax (terminal tasks, under Terminus-2).

Harness the agent program that runs a model on a task and talks to the sandbox: mini-swe-agent 2.1.0 or Terminus-2. Harbor runs harnesses and verifiers; Daytona provides the sandboxes.

Trial (attempt) one run of a harness on one task, ending with a verifier reward.

k attempts per task in generation (k = 8).

Episode all model steps of one trial.

Step, turn one model reply in an episode, usually one tool call. Used interchangeably.

Submit the mini-swe-agent command (`COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`) that ends an episode on the agent's own decision. "Submits" counts episodes whose exit status is `Submitted`. Unsubmitted episodes (for example those cut off at the 32k context) are still scored on their final diff, so solved can exceed submits.

`task_complete` Terminus-2's finish signal, the TMax counterpart of the submit.

Clean pass a passing trial that ended with the agent's own finish (submit or `task_complete`). The v7 and contributor rows use only clean passes.

Max, HIGH, LOW (coined: HIGH, LOW) GLM 5.3 reasoning-effort settings used to generate traces. Max is the template default and produced the v5 pool; HIGH (effort "high") and LOW (effort "low") produced the two v7 pools. "HIGH rows" and "LOW rows" are rows built from those pools.

v5, v7 (coined) version labels of the trace pools and their rows: v5 at max effort, v7 at HIGH and LOW.

Pilot sweep, pilot rows (coined) the learning-rate sweep run before generation, on existing Kimi-2.5 SWE-smith and Qwen3.6-27B TMax traces.

Contributor rows (coined) rows rebuilt from the pilot traces under the v7 rules, for the contributor-data experiment. Named after the collaborators' experiments, which used these teachers.

Row one training example, capped at 32,768 tokens.

Supervised (loss-bearing) tokens tokens that carry the training loss.

B (coined) plain's agentic supervised-token count per epoch; every arm is matched to it.

R, replay slice general (non-agentic) SFT rows mixed into every arm, with a target of $R = \text{round}(B/4)$ supervised tokens; the last row is kept whole, so the total is slightly above R.

allkimi an SFT run on relay traces whose weights were merged into v0; its data includes 1,012 of the contributor SWE-smith trajectories.

ATIF Harbor's per-step trajectory format, used to count steps and tokens in eval episodes.

Held-out tasks 5% of each family, never trained on.

C.2 Methods and runs

Arm one training run of one method on one row set from one parent.

Plain SFT the first clean pass per task, one row per task.

RS-multi (coined) rejection sampling across epochs: plain's tasks, but each later epoch draws a different passing attempt.

Relay the student starts the task and the teacher takes over at a trigger; only the teacher's turns are trained.

RS-multi-TMax (coined) RS-multi on the contributor rows, resampling TMax attempts only, because the contributor SWE-smith set has one trajectory per task.

Pilot student the model trained in the pilot sweep at 1e-6 for 3 epochs.

Plain SFT matched to the relay budget (run name `plain-matched-relay`) plain rows cut to the same size as a short relay pool, so relay can be compared at equal tokens.

Recovery plain rows with synthetic failed turns inserted into about 20% of them; the inserted turns carry no loss.

Plateau dropping plain rows, with tokens whose loss has stopped changing removed from the gradient.

LOW grid (coined) the set of method runs on the LOW rows for both parents (2026-10-22/23).

LR check (coined) plain SFT on LOW v0 rows at learning rate 3e-7 against the 1e-6 twin.

Dose check (coined) a screen of the HIGH plain v0.1 run's epoch-2 export against its final export, to test whether fewer updates help.

Matched pair (coined) two runs on the same 1,432 tasks, one on LOW rows and one on contributor rows (Section 11).

Twin, plain twin two runs that differ in one factor only (for example the same rows on two parents). A method's *plain twin* is plain SFT on the same rows from the same parent.

C.3 Evaluation

Screen one of the four fixed evaluations run on every model.

GSM8K grade-school math, 1,319 problems, 0-shot; a retention check.

BFCL-Parity 123 function-calling tasks from the Berkeley Function Calling Leaderboard as packaged by the eval policy; a retention check.

SWE-bench Verified r100 (coined short form "SWE r100") a fixed random 100 of the 500 SWE-bench Verified tasks, run under mini-swe-agent.

TBLite-100 (coined short form) the 100-task OpenThoughts TBLite set, run under Terminus-2 with an 1,800 s agent timeout.

Composite (coined) (SWE solved + TBLite solved)/2, out of 100. Shown as a range when trials are unscored: unscored counted as failed, then as passed.

Unscored a trial with no verifier outcome because of an infrastructure error.

Timeout a TBLite trial that hit the 1,800 s limit; scored as a failure.

Sensitivity range the range of a paired difference when one run's unscored trials are counted first as failed and then as passed.

Paired comparison a per-task difference between two models on the same tasks, with a 95% interval from resampling tasks (task bootstrap).

Serve the 8-GPU vLLM server that hosts a model during its screens.

Group, record a screen group is the hub job that runs a model's screens; a record is the stored result of one screen.

C.4 Infrastructure and cost

Band the scheduling priority of a job (batch, interactive, production, from lowest).

Eviction (preemption) the scheduler stopping a running job to place a higher-priority one.

Gang the group of GPU tasks of one training run (4 tasks of 8 H100s).

Native checkpoint the trainer's own checkpoint, with optimizer state, used to resume.

Export the HF-format BF16 copy of the weights used for serving and release.

H100-h H100 GPUs held times hours held.

Retained lineage, overhead ledger terms: allocation on attempts whose results are kept, and the rest (Section 7).

D Per-task paired tables

Method: Section 4.3. Every paired comparison in this report was run on the frozen SWE-bench r100 and TBLite-100 task ids, from each record's final archive after retries. The 30 comparisons of the initial experiment are in Table 21. This appendix gives, in Table 47, the 44 comparisons run after all retries of the night of 2026-10-22 (contributor arms, LOW grid, learning-rate check), and points to the per-task transition tables, which list for each pair how many tasks moved between solved, failed after a submit, failed at the context limit, and unscored.

D.1 Night of 2026-10-22, all comparisons

D.2 Transition tables and raw outputs

- Initial experiment (30 comparisons): transition tables in `mopd2/arms/screens.md`, section "Transition tables (reference outcome -> student outcome)"; raw output `t238/paired-out.md`; driver `t238/run_paired.sh`.
- Night of 2026-10-22 (44 comparisons): `t238/pairs-night/` holds 22 Markdown files, one per arm and screen (e.g. `contrib-plain-v0-1-swe.md`); each file holds that arm's comparisons against all its references (one to three), with summary lines and transition tables; pair list `t238/pairs_night.txt`.
- Final per-record counts: `t238/final_counts.py` (same loader as the paired test).
- Parents, per task: `mopd2/arms/parent-per-task-20261022.csv` (200 rows; solved, submitted and timeout flags per task for v0 and v0.1, plus the 5,400 s and 64k TBLite checks).

The `t238/` scripts and outputs live in the screens operator's scratch area and are copied into `mopd2/arms/` where noted; they are not in the Marin repository.

E Training rows and evaluation transcripts, side by side

This appendix shows what *harness format* means with real data. An agent harness is the program that runs the model on a task: it writes the prompt, fixes how an action is expressed, runs the action in a sandbox, returns the result and decides when the episode ends. mini-swe-agent (the SWE-bench screen) takes a `bash` tool call per turn and ends on `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`; Terminus-2 (the TBLite screen) takes a JSON object of keystrokes per turn and ends on `"task_complete": true`. Each training row is one recorded episode rendered with the model's chat template; spans marked TRAINED ON carry loss, spans marked CONTEXT do not. Container host names are replaced by `<container>`; cuts are marked [...].

Measured from these rows: our GLM rows are in the evaluation harness's own format for each family (SWE-smith rows in mini-swe-agent format, TMax rows in Terminus-2 format). The contributor rows are the reverse (Kimi SWE-smith rows in Terminus-2 format, Qwen TMax rows in a mini-swe-agent-style tool-call format). The eval panel (c) is from a different task, since screened tasks are not in training. Whether this format difference explains the results is a hypothesis (6.7).

Table 47: Paired per-task differences, night of 2026-10-22 (A minus B, tasks scored in both). Look at the "Excludes 0" column: it marks the established differences. Pilot-sweep pairs use the 2026-10-18 screen of the sweep's 1e-6 final, run on an earlier Harbor install (Section 4).

Arm	vs	Screen	Solved (A vs B)	Difference [95%]	Excludes 0	Sensitivity
contrib plain v0.1	HIGH plain v0.1	SWE	29 vs 13	+16 [+7, +25]	yes	+16 to +16
contrib plain v0.1	HIGH plain v0.1	TBLite	28 vs 8	+19 [+12, +27]	yes	+18 to +21
contrib plain v0.1	LOW plain v0.1	SWE	29 vs 17	+12 [+3, +22]	yes	+12 to +12
contrib plain v0.1	LOW plain v0.1	TBLite	28 vs 12	+13 [+5, +21]	yes	+9 to +17
contrib plain v0.1	parent v0.1	SWE	29 vs 26	+3 [-6, +12]		+3 to +3
contrib plain v0.1	parent v0.1	TBLite	28 vs 23	+4 [-6, +14]		+4 to +6
contrib plain v0	HIGH plain v0	SWE	35 vs 17	+18 [+10, +26]	yes	+18 to +18
contrib plain v0	HIGH plain v0	TBLite	26 vs 9	+16 [+9, +24]	yes	+9 to +17
contrib plain v0	parent v0	SWE	35 vs 33	+2 [-8, +12]		+2 to +2
contrib plain v0	parent v0	TBLite	26 vs 20	+6 [-2, +14]		+5 to +6
contrib plain v0	pilot sweep 1e-6 final	SWE	35 vs 33	+2 [-6, +10]		+1 to +2
contrib plain v0	pilot sweep 1e-6 final	TBLite	26 vs 29	-3 [-11, +5]		-4 to -3
contrib RS-multi-TMax v0.1	contrib plain v0.1	SWE	29 vs 29	+0 [-8, +8]		+0 to +0
contrib RS-multi-TMax v0.1	contrib plain v0.1	TBLite	24 vs 28	-4 [-14, +6]		-5 to -4
contrib RS-multi-TMax v0.1	parent v0.1	SWE	29 vs 26	+3 [-5, +11]		+3 to +3
contrib RS-multi-TMax v0.1	parent v0.1	TBLite	24 vs 23	+1 [-8, +9]		+0 to +1
contrib recovery v0.1	contrib plain v0.1	SWE	29 vs 29	+1 [-8, +10]		+0 to +1
contrib recovery v0.1	contrib plain v0.1	TBLite	29 vs 28	+1 [-8, +10]		+0 to +1
contrib recovery v0.1	parent v0.1	SWE	29 vs 26	+4 [-4, +12]		+3 to +4
contrib recovery v0.1	parent v0.1	TBLite	29 vs 23	+6 [-4, +17]		+5 to +6
LOW plateau v0.1	LOW plain v0.1	SWE	18 vs 17	+1 [-7, +9]		+1 to +1
LOW plateau v0.1	LOW plain v0.1	TBLite	15 vs 12	+3 [-5, +11]		-4 to +6
LOW recovery v0.1	LOW plain v0.1	SWE	24 vs 17	+7 [-1, +15]		+7 to +7
LOW recovery v0.1	LOW plain v0.1	TBLite	15 vs 12	+4 [-3, +11]		-4 to +5
LOW RS-multi v0.1	LOW plain v0.1	SWE	17 vs 17	+0 [-7, +7]		+0 to +0
LOW RS-multi v0.1	LOW plain v0.1	TBLite	11 vs 12	-1 [-7, +5]		-8 to +3
LOW plain v0	HIGH plain v0	SWE	18 vs 17	+1 [-6, +8]		+1 to +1
LOW plain v0	HIGH plain v0	TBLite	13 vs 9	+2 [-5, +9]		-4 to +7
LOW plain v0	parent v0	SWE	18 vs 33	-15 [-23, -7]	yes	-15 to -15
LOW plain v0	parent v0	TBLite	13 vs 20	-7 [-16, +2]		-8 to -4
LOW relay v0	HIGH relay v0	SWE	24 vs 13	+11 [+3, +19]	yes	+11 to +12
LOW relay v0	HIGH relay v0	TBLite	18 vs 14	+5 [-1, +11]		+3 to +5
LOW relay v0	LOW plain v0	SWE	24 vs 18	+6 [-2, +14]		+6 to +7
LOW relay v0	LOW plain v0	TBLite	18 vs 13	+4 [-3, +11]		+2 to +6
LOW relay v0	parent v0	SWE	24 vs 33	-8 [-16, +0]		-9 to -8
LOW relay v0	parent v0	TBLite	18 vs 20	-2 [-10, +6]		-3 to -1
LOW plateau v0	HIGH plateau v0	SWE	25 vs 14	+11 [+4, +19]	yes	+11 to +11
LOW plateau v0	HIGH plateau v0	TBLite	14 vs 13	+1 [-4, +6]		+1 to +3
LOW plateau v0	LOW plain v0	SWE	25 vs 18	+7 [-1, +15]		+7 to +7
LOW plateau v0	LOW plain v0	TBLite	14 vs 13	+1 [-6, +9]		-2 to +3
LOW plateau v0	parent v0	SWE	25 vs 33	-8 [-17, +2]		-8 to -8
LOW plateau v0	parent v0	TBLite	14 vs 20	-6 [-14, +2]		-7 to -4
LR 3e-7 plain v0	LR 1e-6 plain v0	SWE	11 vs 18	-7 [-13, -1]	yes	-7 to -7
LR 3e-7 plain v0	LR 1e-6 plain v0	TBLite	7 vs 13	-6 [-14, +1]		-9 to -6

E.1 SWE-smith task `swesmith-07306`

E.1.1 (a) Contributor row (Kimi-2.5, Terminus-2 format)

`rows/t202-contrib-v01-20261022/plain`, epoch 0, row 815: 8 model turns, 1,861 trained tokens

```
---- [CONTEXT, NOT TRAINED] ----
<|begin_of_text|><|start_header_id|>system<|end_header_id|>Reasoning: /think<|eot_id|><|start [...]
You are an AI assistant tasked with solving command-line tasks in a Linux environment. You wi [...]

Format your response as JSON with the following structure:

{
  "analysis": "Analyze the current state based on the terminal output provided. What do you s [...]
  "plan": "Describe your plan for the next steps. What commands will you run and why? Be spec [...]
  [...]
[... ~40 lines: JSON schema, keystroke rules, then the task ...]
---- [CONTEXT, NOT TRAINED] ----
Task Description:
ProxyCommand.close() uses SIGKILL instead of SIGTERM after recent changes

  [...]
[...]
Current Terminal Screen:
(testbed) root@<container>:/testbed#
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
---- [TRAINED ON] ----
<|start_think|>Let me analyze the task. The user is reporting an issue with `ProxyCommand.clo [...]

First, I need to explore the codebase to understand the current state and find where the issu [...]

Let me start by exploring the directory structure and finding the relevant files.<|end_think|>{
  "analysis": "I'm in the /testbed directory which likely contains the paramiko codebase. The [...]
  "plan": "First, I'll explore the directory structure to find the paramiko source files, spe [...]
  "commands": [
    {
      "keystrokes": "ls -la\n",
      "duration": 0.5
    },
    {
      "keystrokes": "find . -name 'proxy.py' 2>/dev/null\n",
      "duration": 0.5
    }
  ]
}
---- [CONTEXT, NOT TRAINED] ----
<|eot_id|><|start_header_id|>user<|end_header_id|>
New Terminal Output:

(testbed) root@<container>:/testbed# ls -la
  [...]
[... 5 more turns ...]
---- [CONTEXT, NOT TRAINED] ----
<|eot_id|><|start_header_id|>user<|end_header_id|>
Current terminal state:
New Terminal Output:

(testbed) root@<container>:/testbed#


Are you sure you want to mark the task as complete? This will trigger your solution to be gra [...]
```

```
  [...]
---- [TRAINED ON] ----
<|start_think|>Yes, I'm confident the task is complete. Let me summarize what was done:

1. Found the issue in paramiko/proxy.py where the close() method was using signal.SIGKILL ins [...]
2. Fixed it by changing `signal.SIGKILL` to `signal.SIGTERM`
3. Verified the fix works by running the test code from the issue description, which confirme [...]

The fix is minimal, targeted, and correct. I'm ready to submit.<|end_think|>{
  "analysis": "The fix has been verified and confirmed:\n1. Changed `signal.SIGKILL` to `sign [...]
  "plan": "Task is complete. The one-line fix resolves the issue described in the task.",
  "commands": [],
  "task_complete": true
}
```

E.1.2 (b) Our GLM LOW row (mini-swe-agent format)

`rows/v7-low-20261020/plain`, epoch 0, row 2247: 10 model turns, 1,308 trained tokens

```
---- [CONTEXT, NOT TRAINED] ----
<|begin_of_text|><|start_header_id|>system<|end_header_id|>Reasoning: /think
### Tools

You may call one or more functions to assist with the user query.
You are provided with function signatures within <tools> </tools> tags:

<tools>
{"type": "function", "function": {"name": "bash", "description": "Execute a bash command", "p [...]

</tools>

For each function call, pass a json object with function name and arguments within <tool_call [...]
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>

  [...]
[... system rules: one bash call per response, submit with echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT
↪  ...]
---- [CONTEXT, NOT TRAINED] ----
<|start_header_id|>user<|end_header_id|>
Please solve this issue: ProxyCommand.close() uses SIGKILL instead of SIGTERM after recent ch [...]
[... same issue text as (a), then the workflow and command rules ...]
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
---- [TRAINED ON] ----
<tool_call>
{"name": "bash", "arguments": {"command": "grep -n \"kill\\|SIGKILL\\|SIGTERM\" paramiko/prox [...]
</tool_call>
---- [CONTEXT, NOT TRAINED] ----
<|eot_id|><|start_header_id|>tool<|end_header_id|>
<tool_response name="bash">{
  "returncode": 0,
  "output": "122:        os.kill(self.process.pid, signal.SIGKILL)\n"
}</tool_response><|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
---- [TRAINED ON] ----
<tool_call>
{"name": "bash", "arguments": {"command": "sed -i 's/signal.SIGKILL/signal.SIGTERM/' paramiko [...]
</tool_call>
---- [CONTEXT, NOT TRAINED] ----
```

```
<|eot_id|><|start_header_id|>tool<|end_header_id|>
<tool_response name="bash">{
  "returncode": 0,
  "output": "122:        os.kill(self.process.pid, signal.SIGTERM)\n"
}</tool_response><|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
[... 6 more turns ...]
---- [TRAINED ON] ----
The fix is in place: `ProxyCommand.close()` now sends `SIGTERM` instead of `SIGKILL`, allowin [...]
{"name": "bash", "arguments": {"command": "echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT"}}
</tool_call>
```

E.1.3 (c) Evaluation: SWE-bench r100 under mini-swe-agent (a different task)

record `...-swebench-verified-100-3268` (contributor plain v0.1), trial `psf__requests-2317`

```
---- [SENT BY THE HARNESS] ----
system: You are a helpful assistant that can interact with a computer.
---- [SENT BY THE HARNESS] ----
user: Please solve this issue: method = builtin_str(method) problem
In requests/sessions.py is a command:

  [...]
[... same workflow and submit rules as (b) ...]
tools: [bash(command: string)]   (sent as the API's tools field)
---- [GENERATED BY THE MODEL] ----
assistant content:
<|start_think|>Let me start by analyzing the codebase and understanding the issue. The user i [...]

Let me first explore the repository structure and find the relevant file.
<|end_think|>I'll start by exploring the repository structure and understanding the issue des [...]
tool_calls: [{"name": "bash", "arguments": {"command": "find . -name \"*.py\" | head -20"}}]
---- [RETURNED BY THE HARNESS] ----
tool: {
  "returncode": 0,
  "output": "./build/lib/requests/__init__.py\n./build/lib/requests/adapters.py\n./build/lib/ [...]
  [...]
[... 136 more messages ...]
---- [RETURNED BY THE HARNESS] ----
episode ends: exit_status = ContextLimit
"Prompt has 32791 tokens; context window is 32768"
```

E.2 Terminal (TMax) task `task_000170_2b09f3ad`

E.2.1 (a) Contributor row (Qwen3.6-27B, tool-call format)

`rows/t202-contrib-v01-20261022/plain`, epoch 0, row 281: 6 model turns, 1,970 trained tokens

```
---- [CONTEXT, NOT TRAINED] ----
<|begin_of_text|><|start_header_id|>system<|end_header_id|>Reasoning: /think
### Tools

You may call one or more functions to assist with the user query.
  [...]
[... bash tool schema; 'persistent shell' ...]
---- [CONTEXT, NOT TRAINED] ----
<|start_header_id|>system<|end_header_id|>
You are a helpful assistant that can interact with a computer.

Your response must include a THOUGHT section before your action [...]
```

```
<|start_header_id|>user<|end_header_id|>
Please solve this task:

You are helping a developer organize a messy folder of legacy project files. [...]
[... task, then: 6. Submit ... `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` ...]
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
---- [TRAINED ON] ----
<|start_think|>I'll start by exploring the directory structure and understanding what files a [...]
<|end_think|><tool_call>
{"name": "bash", "arguments": {"command": "ls -la /home/user/legacy_code/"}}
</tool_call>
---- [CONTEXT, NOT TRAINED] ----
<|eot_id|><|start_header_id|>tool<|end_header_id|>
<tool_response name="bash">total 20
drwxr-xr-x 2 root root 101 May 10 19:21 .
drwxr-xr-x 3 root root  76 May 10 19:21 ..
  [...]
---- [TRAINED ON] ----
<|start_think|>

<|end_think|><tool_call>
{"name": "bash", "arguments": {"command": "cat /home/user/legacy_code/prog_a.json && echo \"- [...]
</tool_call>
[... 3 more turns ...]
---- [TRAINED ON] ----
<|start_think|>Everything looks correct:
- The 3 programs with R0 values summing to 100 are prog_b.json (23), prog_c.json (31), and pr [...]
  [...]
4. **Created `/home/user/organized_files/`** and copied the 3 matching files there
5. **Original files remain untouched** in `/home/user/legacy_code/`<tool_call>
{"name": "bash", "arguments": {"command": "echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT"}}
</tool_call>
```

E.2.2 (b) Our GLM LOW row (Terminus-2 format)

`rows/v7-low-20261020/plain`, epoch 0, row 1463: 4 model turns, 1,482 trained tokens

```
---- [CONTEXT, NOT TRAINED] ----
<|begin_of_text|><|start_header_id|>system<|end_header_id|>Reasoning: /think<|eot_id|><|start [...]
You are an AI assistant tasked with solving command-line tasks in a Linux environment. You wi [...]

Format your response as JSON with the following structure:
  [...]
[... the same Terminus-2 JSON schema and keystroke rules as SWE (a) ...]
---- [CONTEXT, NOT TRAINED] ----
Task Description:
You are helping a developer organize a messy folder of legacy project files. [...]
[... same task text as (a) ...]
Current terminal state:
Current Terminal Screen:
root@<container>:~#
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
---- [TRAINED ON] ----
<|start_think|>We need respond JSON commands batches. Need solve via terminal. Current at roo [...]
  "analysis": "The terminal is idle at the root home directory. No files have been inspected  [...]
  "plan": "List the legacy directory and all JSON files, then print a short sample of their c [...]
  "commands": [
    {
      "keystrokes": "find /home/user/legacy_code -maxdepth 1 -type f -name '*.json' -printf ' [...]
      "duration": 0.1
```

```
    },
    {
      "keystrokes": "python3 - <<'PY'\nimport json, glob\nfiles=sorted(glob.glob('/home/user/ [...]
      "duration": 0.2
    }
  ],
  "task_complete": false
}
---- [CONTEXT, NOT TRAINED] ----
<|eot_id|><|start_header_id|>user<|end_header_id|>
New Terminal Output:
root@<container>:~# find /home/user/legacy_code -maxdepth 1 -type f -name '*.json' -printf '% [...]
prog_a.json
prog_b.json
  [...]
[... 2 more turns ...]
---- [TRAINED ON] ----
{
  "analysis": "Yes. The program simulation was completed correctly, and the unique matching c [...]
  "plan": "No further commands are required.",
  "commands": [],
  "task_complete": true
}
```

E.2.3 (c) Evaluation: TBLite-100 under Terminus-2 (a different task)

record `...-openthoughts-tblite-100-f60f` (contributor plain v0.1), trial `jsonl-aggregator`

```
---- [SENT BY THE HARNESS] ----
user: You are an AI assistant tasked with solving command-line tasks in a Linux environment.  [...]

Format your response as JSON with the following structure:

{
  "analysis": "Analyze the current state based on the terminal output provided. What do you s [...]
  [...]
[... same JSON schema and keystroke rules as (b), then the task ...]
---- [GENERATED BY THE MODEL] ----
model reply, as recorded after parsing:
Analysis: I'm in the /app directory and need to explore the JSONL files to understand their s [...]
Plan: First, I'll list the files in the /app directory to see what JSONL files exist. Then I' [...]
commands: [{'tool_call_id': 'call_0_1', 'function_name': 'bash_command', 'arguments': {'keyst [...]
---- [RETURNED BY THE HARNESS] ----
next user message: {'results': [{'content': 'Previous response had warnings:
WARNINGS: - Extra text detected before JSON object

New Terminal Output:
root@<container>:/app# ls -la /app
total 79500
  [...]
[... 11 more steps ...]
---- [GENERATED BY THE MODEL] ----
last reply: Analysis: The task is complete. The /app/aggregates.json file has been successful [...]
- top_5_users_by_amount with 5 users sorted by amount (descending)
  [...]
("task_complete": true; or the 1,800 s limit ends the episode)
```

Revision history

2026-10-23, first draft assembled from the section parts. Two independent cold reads by a reader new to the project (120 and 67 findings); every finding applied by the author of its section.

2026-10-23, review round 1 (coordinator) 1. The earlier coined name for the relay-matched plain run removed from all prose (owner's rule). The run is "plain SFT matched to the relay budget" (run name `plain-matched-relay`), defined at first use. One screen-group identifier that contains the old name is kept, because it is a job name.

2. Appendix E checked: every excerpt with Markdown-like text is inside an example block, exported verbatim with line wrapping.
3. Captions checked: 44 tables, one caption each (multi-line captions join into one).
4. Terms checked at first use; LOW, TMax, SWE-smith, TBLite, "turns" and Evalchemy now glossed where they first appear.
5. "Composite" labelled as our screening mean at its definition and in the first composite caption of the results and follow-up sections.

2026-10-23, review round 2 (coordinator, after an independent fact check) 1. Pilot student against its parent hedged: above in point estimate. A paired interval was computed for this round: SWE-bench 0 [-7, +7], TBLite +8 [-2, +18], both containing 0.

2. Contributor rows: "226 steps per epoch (678 updates over 3 epochs)".
3. Costs: relay-generation split given (HIGH 1,357, LOW 721, at least 59 evicted; about 2,137).
4. Storage: the terminal native save is written twice, so a run's native state is about 3.22 TB rather than about 2.4 TB.
5. ShellSim incident: failure window 02:56:52-03:08:51 UTC, one cancel at 03:11:09; endpoint times of the relaunch added.
6. LOW plain v0.1 BFCL loop: `VerifierTimeoutError`, as named in the record.
7. `# source:` comments added above every table and numeric paragraph that lacked one.
8. Parent lineage: the shared pretraining base (the 10T cooldown, native step 156,000) cited from the provenance atlas; Step12's SFT line marked as inferred.

The fact check also confirmed the paper's cost and storage figures where an earlier summary differed (night training 769.24 H100-h within the 1,282.33 total; storage freed 88.6 TB for the shootout, 92.3 TB with the MOPD batch).

2026-10-24, results addendum 1. Section 11 retitled. It now reports the matched pair and plateau on the contributor rows: training, the export audits, screens (Table 38) and paired differences (Table 39). Measured results are kept separate from the remaining hypothesis, and the pair's limits (1.27 times the tokens, matched on tasks only, single runs) are in the same paragraph as its headline numbers. The decision options before the owner are restated.

2. New subsection 11.6: what a faithful cross-format conversion must preserve, and its estimated effort and cost.
3. New subsection 2.7.4: the trajectory-text decontamination scan, with three generic-code TBLite flags and no row changed.
4. Abstract, rounds table, conclusions and Appendix B updated for the three runs. Section 8 adds the contributor-shared checkpoint-save timeout and its rerun.

5. Costs: the three runs add 184.91 H100-h (documented subtotal about 5,960). Storage: batch 5 adds 10,464,967,266,119 bytes, so the shootout has freed about 99.1 TB (102.7 TB with the MOPD batch).

2026-10-24, addendum review (coordinator) launcher commits split by run (plateau `264434a6ce`, matched pair `548bd67c41`); the plateau-state location after the native checkpoints were deleted; GSM8K shown to one decimal in Table 38; sources repointed to `mopd2/arms/pairs-night/` and `screens.md`; contamination wording updated.

2026-10-24, costs screens of the three added runs, 106.4 H100-h, added to Table 33 (documented subtotal about 6,070).